ARMENU

TIZIM ARXITEKTURASI

I — Asoslar

Modulli monolit · Multi-tenant · 1 mln foydalanuvchiga tayyor
Django 5 · PostgreSQL+pgvector · Redis · 8 AI provayder (MCP)

Qismlar: 9 ta
Hajm: 19 594 qator · 900 KB
ADR: 15 ta arxitektura qarori
Kod namunalari: 944 blok
Jadvallar: 2 969 qator
Versiya: 1.0 · 2026-09-04

✔ **Kim uchun:** tizim arxitektori · texnik direktor · investor (texnik due diligence) · dasturchi.
Har qaror sababi bilan yozilgan: nega shunday, muqobili nima edi, qachon qayta ko'riladi.

MUNDARIJA — I — Asoslar

0. Umumiy ko'rinish

C4 kontekst, sifat talablari, texnologiya tanlovi va sabablari, modulli monolit qarori, fizik arxitektura S0/S3/S5, buyurtma oqimi vaqt byudjeti, 5 oltin qoida, 5 yillik evolyutsiya

1. Printsiplar va qarorlar

12 arxitektura printsipi (kod namunasi + buzilsa nima bo'ladi), 15 ta ADR qat'iy formatda, trade-off tahlili, 18 anti-pattern 3 darajada

2. Komponentlar

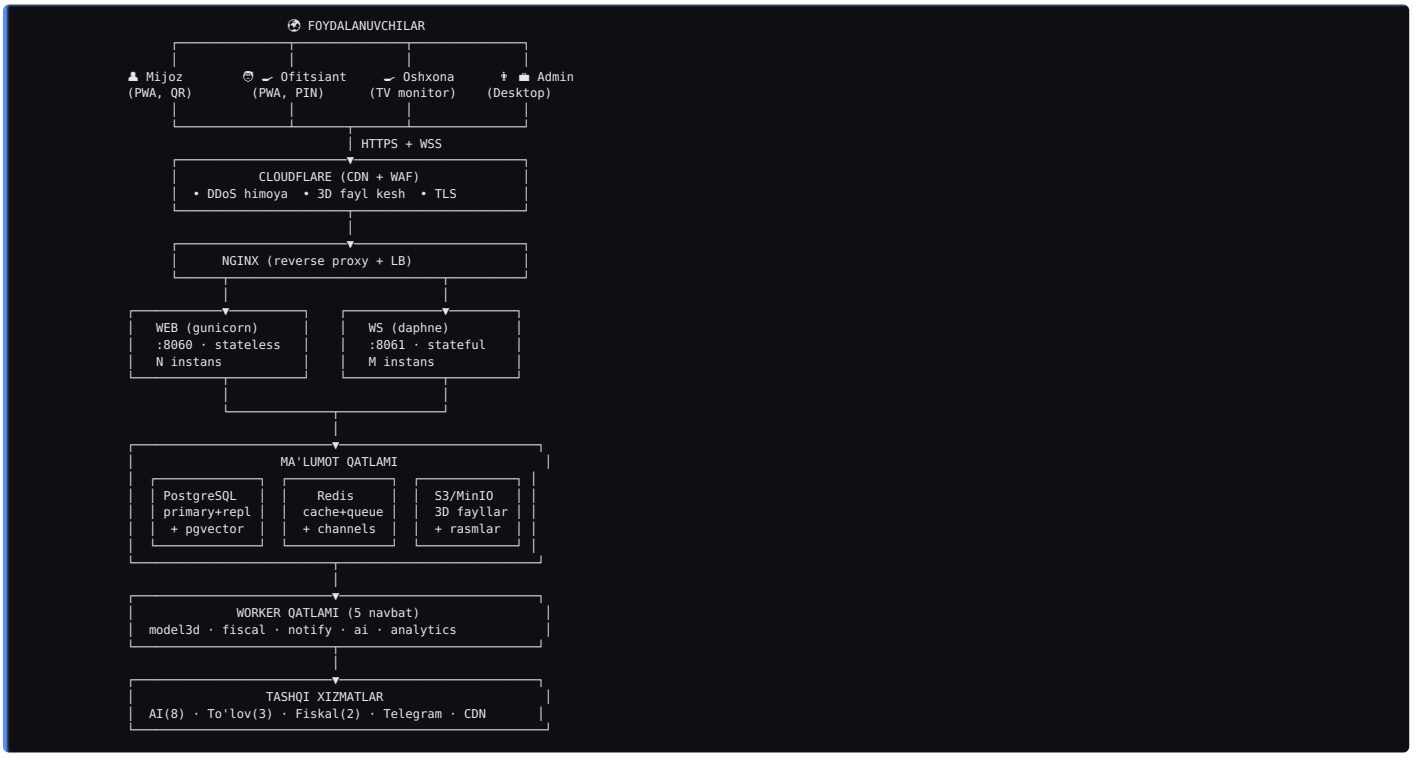
C4 Level 2 (10 konteyner) va Level 3 (13 Django app), modullar bog'liqlik grafi + siklsizlik AST testi, core/ qatlami 17 fayl, 4 frontend ilova, 5 worker navbati

QISM 0

Umumiy ko'rinish

Versiya: 1.0 ·**Sana:** 2026-09-04
Kim uchun: tizim arxitektori · texnik direktor · investor (texnik due diligence) · dasturchi

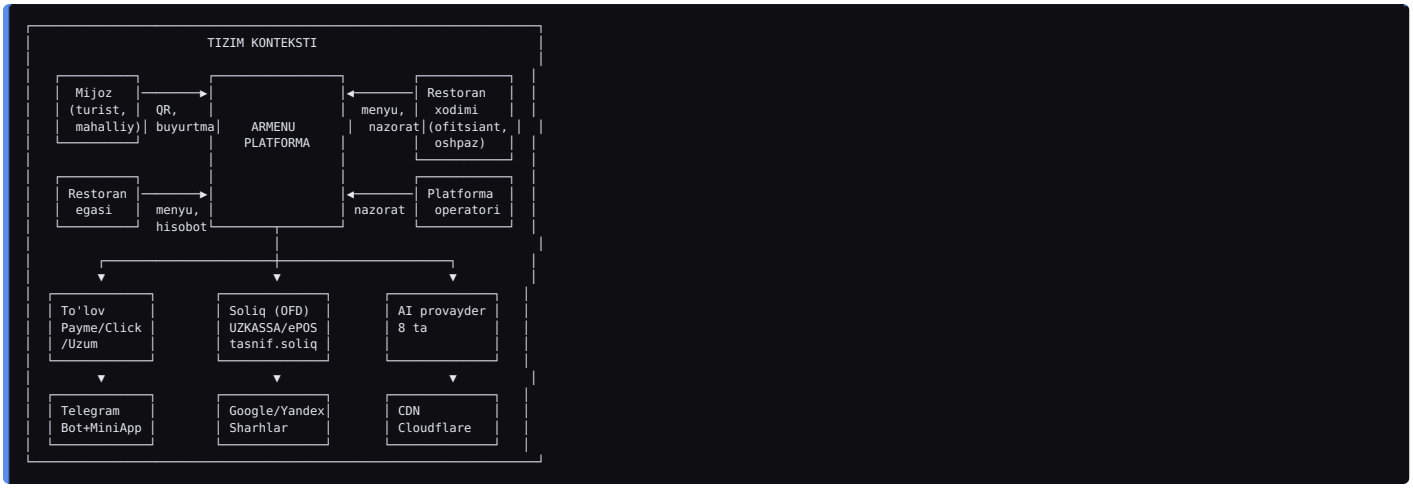
1. ARXITEKTURA BIR QARASHDA



2. ARXITEKTURA HUJJATLARI XARITASI

Qism	Fayl	Nima haqida
0	`00_UMUMIY_KORINISH.md`	← siz shu yerdasiz
1	`01_PRINTSIPLAR_VA_QARORLAR.md`	12 printsip, 15 ADR (nega shunday qilingan)
2	`02_KOMPONENTLAR.md`	Har komponent: vazifasi, texnologiyasi, chegaralari
3	`03_MALUMOT_ARXITEKTURASI.md`	DB sxema, indeks, partition, sharding, kesh qatlamlari
4	`04_MASSHTABLASH.md`	S0→S5, 1 mln foydalanuvchi, sig'im hisoblari
5	`05_ISHONCHLILIK.md`	Fault-tolerance, self-healing, SLA, falokatdan tiklash
6	`06_XAVFSIZLIK.md`	Tahdid modeli, multi-tenant izolyatsiya, to'lov, GDPR
7	`07_INTEGRATSIYALAR.md`	AI/MCP, to'lov, fiskal, Telegram — adapter naqshlari
8	`08_DEPLOY_VA_OPERATSIYA.md`	Infra, CI/CD, monitoring, runbook

3. LOYIHA KONTEKSTI (C4 — Level 1)



4. ASOSIY SIFAT TALABLARI (quality attributes)

Arxitektura shu 8 ta talab atrofida qurilgan. Prioritet tartibida:

#	Talab	Maqsad	Qanday erishiladi
1	**Multi-tenant izolyatsiya**	0 ta sizish	Middleware + Mixin + CI test
2	**Mavjudlik**	99,9% (buyurtma)	Stateless + replica + auto-failover
3	**Ishlash**	p95 < 250 ms	Kesh 4 qatlam + CDN + indeks
4	**Masshtablanuvchanlik**	1 mln bir vaqtda	Gorizontal + WS optimizatsiya
5	**Chidamlilik**	Qism yiqilsa ishlaydi	Circuit breaker + fallback + bulkhead
6	**Tiklanuvchanlik**	Avtomatik	Self-healing daemon (15 mexanizm)
7	**Kengaytiriluvchanlik**	5 yil eskimaydi	Adapter + plugin + MCP qatlam
8	**Kuzatiluvchanlik**	5 daqiqada muammo topiladi	Metric + log + trace + alert

Ataylab qurbon qilinganlar (trade-off):

Nima qurbon qilindi	Nima uchun
Mikroservis (monolit tanlandi)	1-2 dasturchi, tezlik muhim; modulli monolit yetarli
Kubernetes (systemd tanlandi)	S0-S3 gacha kerak emas; murakkablik ortadi
GraphQL (REST tanlandi)	Oddiy CRUD; kesh REST'da osonroq
Event sourcing (CRUD tanlandi)	Murakkablik; audit log yetarli
Real-time hamma joyda	Mijozga WS faqat buyurtmadan keyin (1 mln uchun kritik)

5. TEXNOLOGIK STEK VA SABABLARI

Qatlam	Tanlov	Nima uchun	Muqobil (rad etilgan)
Backend	**Django 5**	Jamoada tajriba, ORM, admin, ekotizim	FastAPI (tajriba yo'q), Node (tip xavfsizligi)
API	**DRF**	Django bilan bir butun, serializer, auth	GraphQL (kesh murakkab)
Real-time	**Channels + Daphne**	Django bilan integratsiya	Socket.io (alohida servis)
DB	**PostgreSQL 16**	JSONB, GIN, partition, pgvector, ishonchli	MySQL (JSONB zaif), Mongo (tranzaksiya)
Vektor	**pgvector**	DB ichida, alohida servis kerak emas	Pinecone (pulli), Qdrant (alohida)
Kesh/navbat	**Redis 7**	Tez, channels layer, navbat, lock	Memcached (pub/sub yo'q), RabbitMQ (ortiqcha)
Fayl	**S3/MinIO**	3D fayllar, versiyalash, CDN origin	Lokal disk (masshtablanmaydi)
Frontend	**Vanilla JS + PWA**	Yengil, tez, framework yuki yo'q	React (200KB+, SEO, murakkablik)
AR	`<model-viewer>`	iOS+Android bitta kod, Google qo'llab-quvvatlaydi	Three.js (AR o'zi yo'q), 8thWall (pulli)
To'lov	**paytechuz**	3 provayder bitta kutubxona	Har biriga alohida SDK
Deploy	**systemd + nginx**	Oddiy, ishonchli, S3 gacha yetadi	k8s (S4 dan keyin), Docker Swarm

6. MODULLI MONOLIT — ASOSIY QAROR

> **Qaror:** Mikroservis EMAS, **modulli monolit**.

Nima uchun

Sabab	Tushuntirish
Jamoa hajmi	1-2 dasturchi. Mikroservis 5+ dasturchi uchun
Tranzaksiya	Buyurtma + to'lov + fiskal — bir tranzaksiyada bo'lishi kerak
Tezlik	15 haftada v1. Mikroservis 2× uzoq
Operatsiya	1 ta deploy, 1 ta log, 1 ta monitoring
Narx	1 ta server yetadi (S0-S1)

Lekin — mikroservisga tayyor

Modullar **qat'iy chegaralangan**, kelajakda ajratish oson:

```
apps/
├── tenants/      ← chegara: restaurant_id
├── menu/         ← chegara: faqat menu modellari
├── orders/       ← chegara: buyurtma hayotiy sikli
├── ar3d/         ← ⚡ birinchi ajratiladigan (og'ir, mustaqil)
├── payments/     ← chegara: to'lov, tashqi API
├── fiscal/       ← chegara: OFD, IKPU
└── ai/           ← ⚡ ikkinchi ajratiladigan (tashqi API, sekin)
```

Ajratish qoidasi:
- Modullar bir-birini **to'g'ridan-to'g'ri chaqirmaydi** → servis funksiyasi orqali
- Modullar orasida **event** (signal) ishlatiladi
- Har modul **o'z jadvallariga** egalik qiladi

Qachon ajratiladi:

Modul	Sharoit
`ar3d`	3D generatsiya > 500/kun yoki GPU kerak bo'lsa
`ai`	AI so'rov > 100k/oy
`analytics`	Event > 10 mln/oy

---

7. FIZIK ARXITEKTURA (deployment view)

7.1 S0 — Start (1-20 restoran)

```
┌──────────────────────────────────┐
│  1 × VPS (4 CPU, 8 GB, 100 GB NVMe)   │
│  ┌────────────────────────────┐  │
│  │ nginx                      │  │
│  ├────────────────────────────┤  │
│  │ armenu.service     (gunicorn ×4)  │  │
│  │ armenu-ws.service  (daphne ×1)    │  │
│  │ armenu-worker@×5                  │  │
│  │ armenu-healer.service             │  │
│  ├────────────────────────────┤  │
│  │ PostgreSQL 16              │  │
│  │ Redis 7                    │  │
│  │ MinIO (3D fayllar)         │  │
│  └────────────────────────────┘  │
│  Narx: ~$40/oy                   │
└──────────────────────────────────┘
```

7.2 S3 — Respublika (400-1500 restoran)

```
┌──────────┐  ┌──────────┐  ┌──────────┐
│ LB × 2   │  │ WEB × 4  │  │ WS × 2   │
│ (nginx)  │  │ 4CPU/8GB │  │ 4CPU/8GB │
└──────────┘  └──────────┘  └──────────┘

┌──────────┐  ┌──────────┐  ┌──────────┐
│ PG primary│  │ PG replica×2 │ Redis × 3 │
│ 8CPU/32GB │  │ 4CPU/16GB │  │ (sentinel) │
└──────────┘  └──────────┘  └──────────┘

┌──────────┐  ┌──────────┐
│ WORKER × 3 │  │ S3 + CDN │
└──────────┘  └──────────┘

Narx: ~$700/oy
```

7.3 S5 — 1 mln bir vaqtda

Batafsil: `04_MASSHTABLASH.md` §5

```
40 × WEB · 3 × WS · 6 × WORKER
1 × PG primary (16CPU/64GB) + 3 × replica
6 × Redis cluster · 2 × ClickHouse
Narx: ~$4 400/oy (daromadning 1,9%)
```

---

8. MA'LUMOT OQIMI — ASOSIY SSENARIY

Buyurtma yaratish (eng kritik yo'l)

```
[1] Mijoz "Buyurtma" bosadi
     │
     ▼
[2] POST /v1/orders/  +  X-Idempotency-Key
     │
     ▼
[3] nginx → gunicorn worker
     │
     ▼
[4] TenantMiddleware → restaurant_id aniqlanadi
     │
     ▼
[5] Idempotency tekshiruvi (Redis)
     │  ├─ mavjud → mavjud buyurtma qaytariladi ✓ TUGADI
     │  └─ yo'q → davom
     ▼
[6] Validatsiya (7 bosqich)
     │  • stol shu restoranmi?
     │  • taomlar shu restoranmi?
     │  • stop-listda emasmi?
     │  • ⚡ NARX SERVERDA HISOBLANADI
     ▼
[7] transaction.atomic():
     │  Order.objects.create()
     │  OrderItem.objects.bulk_create()
     │  Table.status = 'ordering'
     ▼
[8] transaction.on_commit():
     │  ├─ WS → rest_<id>_kds       (oshxona)
     │  ├─ WS → rest_<id>_waiters   (ofitsiant)
     │  ├─ WS → rest_<id>_admin     (admin)
     │  ├─ WS → guest_<session>     (mijoz)
     │  ├─ enqueue('notify', ...)   (Telegram push)
     │  ├─ enqueue('notify', ...)   (oshxona printeri)
     │  └─ enqueue('analytics',...)  (event)
     ▼
[9] 201 Created  {id, number: "#047", eta_min: 18}

Vaqt byudjeti (p95):
  [3]-[4]  nginx + middleware      ≤  15 ms
  [5]      Redis idempotency        ≤   3 ms
  [6]      validatsiya (2 so'rov)   ≤  40 ms
  [7]      transaksiya (3 yozuv)    ≤  80 ms
  [8]      on_commit (async)        ≤   5 ms
  ──────────────────────────────────────────
  JAMI                              ≤ 143 ms   (maqsad 200 ms) ✓
```

9. ARXITEKTURANING 5 TA "OLTIN QOIDA"SI

> Bu qoidalar buzilsa — arxitektura buziladi. CI'da tekshiriladi.

#	Qoida	Buzilsa nima bo'ladi	Tekshiruv
1	**Har so'rov `restaurant_id` bilan filtrlanadi**	Ma'lumot sizishi (kritik)	`test_tenant_isolation.py`
2	**Narx faqat serverda hisoblanadi**	Moliyaviy zarar	`test_price_calculation.py`
3	**Har yozuv operatsiyasi idempotent**	Dublikat buyurtma/to'lov	`test_payment_idempotency.py`
4	**Uzoq ish (>2s) → worker**	Web bloklanadi, timeout	Kod ko'rigi + `tekshir_kod.sh`
5	**Tashqi chaqiruv → circuit breaker**	Kaskadli yiqilish	Kod ko'rigi

10. ARXITEKTURA EVOLYUTSIYASI (5 yil)

```
YIL 1 — MODULLI MONOLIT
├── 1 VPS → 4 node
├── PostgreSQL + Redis
└── 400 restoran

YIL 2 — GORIZONTAL MASSHTAB
├── Read replica × 2
├── ar3d moduli ajratiladi (GPU node)
├── ClickHouse (analitika)
└── 1 500 restoran, 2 mamlakat

YIL 3 — REGIONAL
├── Multi-region (UZ + KZ)
├── ai moduli ajratiladi
├── DB partition → sharding tayyorgarligi
└── 4 000 restoran, 4 mamlakat

YIL 4 — PLATFORMA
├── Mijoz marketplace ilovasi
├── Ta'minotchi moduli
├── Sharding (restaurant_id hash)
└── 8 000 restoran

YIL 5 — EKOTIZIM
├── Moliyalash moduli (skoring)
├── Ochiq API (3rd party)
├── k8s (agar kerak bo'lsa)
└── 15 000+ restoran
```

Har bosqichda arxitektura o'zgarmaydi — kengayadi. Sabab:
- Stateless → node qo'shiladi
- Adapter → yangi provayder qo'shiladi
- Modul chegarasi → ajratish oson
- MCP qatlam → AI provayder almashadi

11. RISK VA CHEKLOVLAR

Arxitekturaviy risklar

Risk	Ehtimol	Ta'sir	Yumshatish
Monolit o'sib ketadi	O'rta	O'rta	Modul chegarasi qat'iy; ajratish rejasi bor
PostgreSQL yozish cheklovi	Past	Yuqori	Partition → replica → sharding yo'li tayyor
WS 1 mln ulanish	Past	Yuqori	⚡ Mijozga WS faqat buyurtmadan keyin
3D fayl hajmi (storage)	O'rta	O'rta	Kutubxona (nusxa emas, havola) + CDN
Tashqi API bog'liqlik	Yuqori	O'rta	8 AI, 3 to'lov, 2 fiskal — fallback zanjiri
Vendor lock-in	Past	O'rta	Ochiq texnologiyalar; S3 API standart

Ma'lum cheklovlar (bilib qabul qilingan)

Cheklov	Sabab	Qachon hal qilinadi
iOS'da WebXR yo'q	Apple cheklovi	Apple ochganda (USDZ ishlaydi)
Gaussian Splatting yo'q	AR Quick Look qo'llab-quvvatlamaydi	Standart chiqqanda
Offline buyurtma yo'q	Fiskal + narx serverda	v2 da navbat bilan
Real-time hamma joyda emas	1 mln uchun optimizatsiya	Ataylab

QISM 1

Printsiplar va qarorlar (ADR)

Versiya: 1.0 · **Sana:** 2026-09-04
Kim uchun: tizim arxitektori · texnik direktor · yangi dasturchi (onboarding) · kod ko'ruvchi

> Bu hujjat **"nega shunday qilingan"** degan savolga javob beradi.
> `00_UMUMIY_KORINISH.md` — nima qurilgan. Bu yerda —**nima uchun aynan shunday**.
> Har qaror sanasi, konteksti va **qaytarish narxi** bilan yozilgan.

MUNDARIJA

Bo'lim	Mazmuni
§1	12 ta arxitektura printsipi (o'zgarmas qoidalar)
§2	15 ta ADR — arxitektura qarorlari yozuvi
§3	Trade-off tahlili — nima qurbon qilindi
§4	Anti-pattern'lar — bu loyihada QILINMAYDI
§5	Printsip ↔ ADR ↔ CI test bog'lanish matritsasi

---

§1. ARXITEKTURA PRINTSIPLARI

> ⚡ Bu 12 printsip — **muzokara qilinmaydi**. Yangi kod ularga mos bo'lishi kerak.
> Har birida: ta'rif · nima uchun · amalda (kod) · buzilsa nima bo'ladi.

Printsiplar xaritasi:



P1 — Stateless Application (holatsiz ilova)

Ta'rif: Web va WS process'lar **hech qanday holatni xotirada yoki diskda saqlamaydi**.
Barcha holat tashqi saqlagichda: Redis (session, kesh, lock), PostgreSQL (ma'lumot), S3 (fayl).

Nima uchun: Gorizontal masshtablash faqat shunda ishlaydi. Node qo'shish/olib tashlash foydalanuvchiga sezilmaydi. Deploy — zero-downtime (eski process o'ladi, holat yo'qolmaydi).
1 mln bir vaqtda uchun 40 ta web instans kerak — ular bir-biriga teng bo'lishi shart.

Amalda:

```
# config/settings.py
SESSION_ENGINE = 'django.contrib.sessions.backends.cache'
SESSION_CACHE_ALIAS = 'default'          # Redis db=5

CACHES = {
    'default': {
        'BACKEND': 'django_redis.cache.RedisCache',
        'LOCATION': 'redis://127.0.0.1:6379/5',
        'OPTIONS': {'CLIENT_CLASS': 'django_redis.client.DefaultClient'},
    }
}

# Fayl — hech qachon lokal diskka emas
DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
```

```
# ✖ NOTO'G'RI — process xotirasida holat
_menu_cache = {}                         # 2-instansda yo'q!

def get_menu(slug):
    if slug not in _menu_cache:
        _menu_cache[slug] = build_menu(slug)
    return _menu_cache[slug]

# ✔ TO'G'RI — umumiy saqlagich
def get_menu(slug, lang, version):
    key = f'menu:{slug}:{lang}:v{version}'
    data = cache.get(key)
    if data is None:
        data = build_menu(slug)
        cache.set(key, data, 60)
    return data
```

Buzilsa nima bo'ladi:
🔴 Foydalanuvchi 1-instansda savatga taom qo'shadi, keyingi so'rov 2-instansga tushadi — savat bo'sh. Sticky session qo'yishga majbur bo'lasiz → LB balansi buziladi → bir node 90% yuklanadi, qolganlari bo'sh turadi → auto-scale ishlamaydi.

P2 — Shared-Nothing (hech narsa umumiy emas)

Ta'rif: Har instans **mustaqil ishlaydi**. Node'lar bir-biriga to'g'ridan-to'g'ri murojaat qilmaydi, umumiy disk (NFS) yo'q, umumiy xotira yo'q. Faqat umumiy **ma'lumot qatlami** (PG/Redis/S3) bor.

Nima uchun: Bitta node yiqilsa — qolganlari sezmaydi. Deploy'da node'larni birma-bir almashtirish mumkin (rolling). Node'ni klonlash — bir xil image + env.

Amalda:

```
# deploy/systemd/armenu@.service — har instans bir xil, faqat port farq qiladi
[Service]
ExecStart=/opt/armenu/venv/bin/gunicorn config.wsgi:application \
    --bind 127.0.0.1:80%i \
    --workers 8 --threads 4 \
    --max-requests 2000 --max-requests-jitter 200 \
    --timeout 30 --graceful-timeout 20
Restart=always
```

```
upstream armenu_web {
    least_conn;
    server 127.0.0.1:8060 max_fails=2 fail_timeout=9s;
    server 127.0.0.1:8062 max_fails=2 fail_timeout=9s;
    server 10.0.0.5:8060  max_fails=2 fail_timeout=9s;   # boshqa node
    keepalive 64;
}
```

Buzilsa nima bo'ladi:
🔴 NFS umumiy papka qo'ysangiz — u SPOF (single point of failure) bo'ladi va yozish qulflari (lock) barcha node'ni bo'g'adi. Node A node B'ga to'g'ridan murojaat qilsa — B yiqilganda A ham yiqiladi (kaskad).

P3 — Graceful Degradation (bosqichma-bosqich pasayish)

Ta'rif: Tizimning bir qismi yiqilsa —**tizim to'liq o'lmaydi**, imkoniyat pasayadi. Har xizmat uchun **fallback zanjiri** oldindan belgilangan.

Nima uchun: Restoran kechqurun ishlaydi. AI yiqilgani uchun mijoz taom buyurtma qila olmasa — bu biznes yo'qotish. AI yo'q bo'lsa ham menyu va buyurtma ishlashi shart.

Amalda — zanjirlar:

```
👁 AR:        WebXR/QuickLook → 3D viewer → 360° rasm → oddiy rasm
🤖 AI:        Gemini → Haiku → GPT-mini → Groq → DeepSeek → Sonnet → FAQ + "Ofitsiantni chaqiring"
📡 Real-time: WebSocket → SSE → polling 10s → qo'lda yangilash tugmasi
🧾 Fiskal:    OFD-1 → OFD-2 → navbat (keyin) → qo'lda chek
💳 To'lov:    Payme → Click → Uzum → naqd (ofitsiantga)
🖼 3D fayl:   CDN → S3 origin → lokal kesh → rasm
💾 O'qish:    Redis kesh → read replica → primary → statik snapshot
```

```
# core/resilience.py
def with_fallback_chain(providers, fn_name, *args, **kwargs):
    """Zanjir bo'ylab yuradi, birinchi muvaffaqiyatli javobni qaytaradi."""
    last_error = None
    for p in providers:
        breaker = get_breaker(p.name)
        if breaker.state == 'OPEN':
            metrics.inc(f'chain.skip.{p.name}')
            continue
        try:
            return breaker.call(getattr(p, fn_name), *args, **kwargs)
        except Exception as e:
            last_error = e
            logger.warning('chain: %s yiqildi → keyingisi', p.name)
    raise AllProvidersDown(fn_name) from last_error
```

Buzilsa nima bo'ladi:
🔴 Bitta tashqi API (masalan OFD) yiqilsa — butun buyurtma oqimi to'xtaydi.
Restoran kassasi ishlamaydi, mijozlar ketadi. Bir kechada 5 000 restoran zarar ko'radi.

P4 — Idempotency (takroriy bajarish xavfsiz)

Ta'rif: Har **yozuv** operatsiyasi (buyurtma, to'lov, chek) idempotency key bilan
келади. Bir xil key bilan 2-marta chaqirilsa — **yangi yozuv yaratilmaydi**, mavjudi qaytariladi.

Nima uchun: Mobil tarmoq beqaror. Mijoz "Buyurtma" tugmasini 3 marta bosadi.
Retry mexanizmi ham takror yuboradi. Idempotency yo'q bo'lsa — 3 ta buyurtma,
3 marta pul yechilishi.

Amalda:

```
# core/idempotency.py
IDEM_TTL = 24 * 3600

def idempotent(scope):
    """
    X-Idempotency-Key sarlavhasi bo'yicha takrorni bloklaydi.
    Redis SET NX — atomik, race condition yo'q.
    """
    def deco(view_fn):
        @wraps(view_fn)
        def wrapper(request, *a, **kw):
            key = request.headers.get('X-Idempotency-Key')
            if not key:
                raise ValidationError({'X-Idempotency-Key': 'majburiy'})

            rid = request.restaurant_id
            rkey = f'idem:{scope}:{rid}:{key}'

            # 1) Tugallangan javob bormi?
            cached = cache.get(rkey + ':done')
            if cached is not None:
                return Response(cached, status=200, headers={'X-Idempotent-Replay': '1'})

            # 2) Qulf ol (parallel dublikat uchun)
            if not cache.add(rkey + ':lock', '1', timeout=30):
                raise Conflict('Shu so\'rov hozir bajarilmoqda')

            try:
                resp = view_fn(request, *a, **kw)
                if 200 <= resp.status_code < 300:
                    cache.set(rkey + ':done', resp.data, IDEM_TTL)
                return resp
            finally:
                cache.delete(rkey + ':lock')
        return wrapper
    return deco


@api_view(['POST'])
@idempotent('order.create')
def create_order(request): ...
```

DB darajasida ham himoya (ikki qatlam):

```
class Order(models.Model):
    idempotency_key = models.CharField(max_length=64, db_index=True)
    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=['restaurant', 'idempotency_key'],
                name='uniq_order_idem',
            )
        ]
```

Buzilsa nima bo'ladi:
🔴 Dublikat buyurtma → oshxona 2 ta taom tayyorlaydi (moddiy zarar).
Dublikat to'lov → mijozdan 2 marta pul → chargeback → provayder hisobini yopadi.
Bu **eng qimmat** xato turi.

P5 — Event-Driven (hodisaga asoslangan bog'lanish)

Ta'rif: Modullar bir-birini **to'g'ridan-to'g'ri chaqirmaydi**. Domain event chiqariladi,
qiziqqan modul obuna bo'ladi. Event `transaction.on_commit()` **ichida** yuboriladi.

Nima uchun: `orders` moduli `notifications`, `analytics`, `loyalty`, `fiscal` haqida
bilmasligi kerak. Yangi modul qo'shilsa — `orders` kodi o'zgarmaydi. Bu ADR-001
(modulli monolit) ni **haqiqiy** qiladi: mikroservisga ajratishda event → message bus.

Amalda:

```
# core/events.py
order_created   = django.dispatch.Signal()   # sender=Order
order_paid      = django.dispatch.Signal()
order_completed = django.dispatch.Signal()
ar_viewed       = django.dispatch.Signal()

# apps/orders/services.py
@transaction.atomic
def create_order(*, restaurant, table, items, session):
    order = Order.objects.create(restaurant=restaurant, table=table, ...)
    OrderItem.objects.bulk_create(build_items(order, items))
    table.status = Table.Status.ORDERING
    table.save(update_fields=['status'])

    # ⚡ Faqat COMMIT'dan keyin — rollback bo'lsa event chiqmaydi
    transaction.on_commit(lambda: order_created.send(sender=Order, order=order))
    return order

# apps/notifications/handlers.py — orders bu faylni bilmaydi
@receiver(order_created)
def on_order_created(sender, order, **kw):
    enqueue('notify', 'telegram_new_order', order_id=order.id)
    enqueue('notify', 'kitchen_print',      order_id=order.id)

# apps/analytics/handlers.py
@receiver(order_created)
def track_order(sender, order, **kw):
    enqueue('analytics', 'order_created', order_id=order.id)
```

⚠ **Qat'iy qoida:** handler ichida **og'ir ish qilinmaydi** — faqat `enqueue()`.
Signal sinxron ishlaydi; ichida `requests.post()` bo'lsa web so'rov bloklanadi.

Buzilsa nima bo'ladi:
🔴 `orders/services.py` 800 qatorga o'sadi va 9 ta modulni import qiladi → aylanma import (circular import) → modul chegarasi yo'qoladi → mikroservisga ajratish imkonsiz → ADR-001 ning "kelajakda ajratamiz" va'dasi yolg'onga aylanadi.

P6 — CQRS-lite (o'qish va yozishni ajratish, yengil variant)

Ta'rif: To'liq CQRS emas (alohida read-model DB emas), **yengil** variant: SELECT → read replica, INSERT/UPDATE → primary. Og'ir hisobotlar uchun materialized view. Read-your-writes kafolati saqlanadi.

Nima uchun: Yuklama profili nosimmetrik: 60% menyu o'qish, 5% buyurtma yozish. O'qishni alohida masshtablash **10× arzon** — replica qo'shish sharding'dan oson.

Amalda:

```
# core/db_router.py
class ReadWriteRouter:
    """SELECT → replica, yozish → primary. Read-your-writes saqlanadi."""

    def db_for_read(self, model, **hints):
        # Yozgandan keyin 5s ichida shu so'rov primary'dan o'qiydi
        if getattr(_local, 'force_primary', False):
            return 'default'
        if model._meta.app_label in ('analytics',):
            return 'replica_analytics'
        return 'replica'

    def db_for_write(self, model, **hints):
        _local.force_primary = True          # shu so'rov oxirigacha
        return 'default'


# core/middleware.py
class ReadYourWritesMiddleware:
    """POST/PUT/PATCH/DELETE — butun so'rov primary'dan o'qiydi."""
    def __call__(self, request):
        _local.force_primary = request.method not in ('GET', 'HEAD', 'OPTIONS')
        try:
            return self.get_response(request)
        finally:
            _local.force_primary = False
```

```
-- Menyu-injiniring hisoboti — har so'rovda hisoblanmaydi
CREATE MATERIALIZED VIEW mv_dish_performance AS
SELECT d.restaurant_id, d.id AS dish_id,
       COUNT(oi.id)                        AS order_count,
       SUM(oi.qty * oi.price)              AS revenue,
       AVG(EXTRACT(EPOCH FROM o.ready_at - o.created_at))/60 AS avg_cook_min
FROM menu_dish d
LEFT JOIN orders_orderitem oi ON oi.dish_id = d.id
LEFT JOIN orders_order o      ON o.id = oi.order_id AND o.status = 'completed'
WHERE o.created_at >= now() - interval '30 days'
GROUP BY d.restaurant_id, d.id;

CREATE UNIQUE INDEX ON mv_dish_performance (restaurant_id, dish_id);
-- REFRESH MATERIALIZED VIEW CONCURRENTLY — har 15 daqiqada (cron)
```

Buzilsa nima bo'ladi:
🟠 Barcha o'qish primary'ga tushadi → primary CPU 100% → yozish (buyurtma) sekinlashadi → buyurtma p95 200ms dan 2s ga chiqadi → mijoz tugmani qayta bosadi → yuklama yana ortadi.
🔴 Read-your-writes buzilsa: ofitsiant buyurtmani tasdiqlaydi, ro'yxatda ko'rmaydi (replica lag) → qayta kiritadi → dublikat.

P7 — Cache-First (avval kesh)

Ta'rif: Issiq yo'l (menyu, taom, restoran) **avval keshdan** o'qiladi.
4 qatlam: CDN → nginx → Redis → DB. Invalidatsiya **versiya kaliti** orqali (TTL emas).

Nima uchun: Menyu 60% trafik va **kamdan-kam o'zgaradi**. 12 000 RPS menyu so'rovidan DB'ga 1 200 RPS ham tushmasligi kerak (>90% hit).

Amalda — versiyalangan kalit (eng muhim naqsh):

```
# core/cache.py
def menu_version(restaurant_id: int) -> int:
    """Menyu o'zgarganda bu raqam oshadi → barcha eski kalit o'z-o'zidan o'lik."""
    v = cache.get(f'menuver:{restaurant_id}')
    if v is None:
        v = Restaurant.objects.values_list('menu_version', flat=True).get(pk=restaurant_id)
        cache.set(f'menuver:{restaurant_id}', v, 3600)
    return v

def bump_menu_version(restaurant_id: int):
    Restaurant.objects.filter(pk=restaurant_id).update(menu_version=F('menu_version') + 1)
    cache.delete(f'menuver:{restaurant_id}')


# apps/menu/services.py
def get_menu_payload(restaurant_id, lang):
    v = menu_version(restaurant_id)
    key = f'menu:{restaurant_id}:{lang}:v{v}'
    data = cache.get(key)
    if data is None:
        data = build_menu(restaurant_id, lang)
        cache.set(key, data, 300)
        metrics.inc('cache.menu.miss')
    else:
        metrics.inc('cache.menu.hit')
    return data


# apps/menu/handlers.py — dish/category o'zgarganda
@receiver([post_save, post_delete], sender=Dish)
@receiver([post_save, post_delete], sender=Category)
def invalidate_menu(sender, instance, **kw):
    transaction.on_commit(lambda: bump_menu_version(instance.restaurant_id))
```

Kesh qatlamlari:

Qatlam	Nima	TTL	Invalidatsiya
CDN (Cloudflare)	3D, rasm, JS/CSS	30 kun	fayl nomida hash
CDN	menyu JSON	60s	`Cache-Tag` purge
nginx `proxy_cache`	menyu API	30s	versiya kaliti URL'da
Redis	menyu payload, taom, restoran	300s	`bump_menu_version()`
Django ORM	—	—	keshsiz (ataylab)

⛔ **Stop-list keshlanmaydi:** taom tugagan bo'lsa mijoz buyurtma bermasligi kerak. Stop-list alohida, TTL=5s va WS orqali darhol push qilinadi.

Buzilsa nima bo'ladi:
🔴 Har menyu so'rovi DB'ga → 12 000 RPS × 8 ta JOIN → primary o'ladi.
🟡 Invalidatsiya noto'g'ri bo'lsa: admin narxni o'zgartiradi, mijoz eski narxni ko'radi, server yangi narx bilan hisoblaydi → mijoz "aldadingiz" deydi.

P8 — Circuit Breaker (elektr avtomati)

Ta'rif: Har **tashqi** chaqiruv breaker orqali o'tadi. Xizmat N marta yiqilsa — breaker OPEN bo'ladi va so'rovlar **umuman yuborilmaydi** (darhol fallback). Timeout'dan keyin HALF_OPEN — sinov so'rovi.

Nima uchun: Yiqilgan xizmatga so'rov yuborish — 30 soniya kutish. 100 parallel so'rov = 100 ta band gunicorn thread = butun web qatlami bloklanadi. Bu **kaskadli yiqilish**.

Amalda:

```
# core/resilience.py
class CircuitBreaker:
    """CLOSED → (N xato) → OPEN → (timeout) → HALF_OPEN → (M muvaffaqiyat) → CLOSED"""

    def __init__(self, name, fail_threshold=5, timeout=60, success_threshold=2):
        self.name, self.fail_threshold = name, fail_threshold
        self.timeout, self.success_threshold = timeout, success_threshold
        self._k = f'cb:{name}'

    def call(self, fn, *args, fallback=None, **kwargs):
        state = cache.get(f'{self._k}:state', 'CLOSED')

        if state == 'OPEN':
            if cache.get(f'{self._k}:opened_at', 0) + self.timeout < time.time():
                cache.set(f'{self._k}:state', 'HALF_OPEN', 300)
            else:
                metrics.inc(f'cb.{self.name}.rejected')
                if fallback: return fallback()
                raise ServiceUnavailable(self.name)

        try:
            result = fn(*args, **kwargs)
            self._on_success()
            return result
        except Exception:
            self._on_failure()
            metrics.inc(f'cb.{self.name}.failure')
            if fallback: return fallback()
            raise


# Har tashqi xizmat uchun alohida breaker + o'z chegarasi
BREAKERS = {
    'ofd_uzkassa': CircuitBreaker('ofd_uzkassa', fail_threshold=3,  timeout=120),
    'payme':       CircuitBreaker('payme',       fail_threshold=5,  timeout=60),
    'gemini':      CircuitBreaker('gemini',      fail_threshold=3,  timeout=30),
    'meshy_3d':    CircuitBreaker('meshy_3d',    fail_threshold=2,  timeout=300),
    'telegram':    CircuitBreaker('telegram',    fail_threshold=10, timeout=60),
}
```

⚠ **Timeout majburiy — breaker'dan oldin:**

```
# ✖ Timeout yo'q — breaker ham qutqarmaydi
requests.post(url, json=data)

# ✔
requests.post(url, json=data, timeout=(3, 10))   # (connect, read)
```

Buzilsa nima bo'ladi:
🔴 OFD 30s javob bermaydi → 8 worker × 4 thread = 32 thread band → yangi so'rov navbatda → nginx 502 → **butun platforma o'ladi**, sababi bitta tashqi API.

P9 — Bulkhead (kema bo'lmalari)

Ta'rif: Resurslar **ajratilgan bo'lmalarga** bo'lingan. Bir bo'lma to'lca — qolganlari ishlaydi. Amalda: alohida navbat, alohida worker pool, alohida DB connection limit.

Nima uchun: 3D generatsiya 5 daqiqa davom etadi. Agar u fiskal chek bilan bir navbatda bo'lsa — 200 ta 3D job navbatda turganda chek 16 soatdan keyin yuboriladi (soliq buzilishi).

Amalda:

```
┌──────────────────────────────────────────────────────┐
│                   REDIS NAVBATLARI                   │
│                                                      │
│  queue:model3d   ████▒▒▒▒▒▒  3 worker  · og'ir, 2-8 daqiqa    │
│  queue:fiscal    ██▒▒▒▒▒▒▒▒  5 worker  · KRITIK, <5s           │
│  queue:notify    █▒▒▒▒▒▒▒▒▒  8 worker  · yengil, <2s           │
│  queue:ai        ██▒▒▒▒▒▒▒▒  4 worker  · tashqi API kutadi     │
│  queue:analytics ▒▒▒▒▒▒▒▒▒▒  2 worker  · past prioritet       │
│                                                      │
│  ⚡ model3d 5000 job bilan to'lca — fiscal ta'sirlanmaydi     │
└──────────────────────────────────────────────────────┘
```

```
# deploy/systemd/armenu-worker@.service
[Service]
ExecStart=/opt/armenu/venv/bin/python manage.py run_worker --queue=%i
MemoryMax=2G                  # bo'lma chegarasi: RAM
CPUQuota=150%                 # bo'lma chegarasi: CPU
Restart=always
RestartSec=5
```

```
# core/queue.py
QUEUE_LIMITS = {
    'model3d':   {'workers': 3, 'max_depth': 5000,  'job_timeout': 900},
    'fiscal':    {'workers': 5, 'max_depth': 500,   'job_timeout': 30},
    'notify':    {'workers': 8, 'max_depth': 20000, 'job_timeout': 20},
    'ai':        {'workers': 4, 'max_depth': 2000,  'job_timeout': 60},
    'analytics': {'workers': 2, 'max_depth': 50000, 'job_timeout': 120},
}

def enqueue(queue, task, **payload):
    limit = QUEUE_LIMITS[queue]
    if redis.llen(f'queue:{queue}') >= limit['max_depth']:
        metrics.inc(f'queue.{queue}.rejected')
        if queue in ('fiscal', 'notify'):      # kritik → DLQ, yo'qolmasin
            return move_to_dlq(queue, task, payload)
        raise QueueFull(queue)                 # yengil → tashlanadi
    redis.rpush(f'queue:{queue}', json.dumps({'task': task, **payload}))
```

DB connection bulkhead ham bor:

Iste'molchi	PgBouncer pool	Sabab
web (mijoz)	120	Asosiy trafik
ws	20	Kam so'rov
worker:fiscal	15	Kafolatlangan joy
worker:model3d	10	Cheklangan — bo'g'masin
admin/hisobot	15	Og'ir so'rov izolyatsiyasi

Buzilsa nima bo'ladi:
🟠 Yagona navbat + yagona pool: bitta restoran 500 ta taomga 3D generatsiya buyursa — butun platformada chek, bildirishnoma va AI to'xtaydi.

P10 — Observability (kuzatiluvchanlik)

Ta'rif: Har so'rov **trace_id** bilan belgilanadi. Metric (Prometheus) + log (Loki) + trace (OpenTelemetry) + xato (Sentry). Muammoni **5 daqiqada** topish mumkin.

Nima uchun: 1 mln foydalanuvchida "sekin ishlayapti" degan shikoyat — 40 ta node, 5 ta navbat, 12 ta tashqi API ichida. Ko'r bo'lsangiz — kunlar ketadi.

Amalda:

```
# core/middleware.py
class ObservabilityMiddleware:
    def __call__(self, request):
        trace_id = request.headers.get('X-Trace-Id') or uuid.uuid4().hex[:16]
        request.trace_id = trace_id
        _local.trace_id = trace_id
        _local.restaurant_id = None            # TenantMiddleware to'ldiradi

        t0 = time.perf_counter()
        try:
            response = self.get_response(request)
        finally:
            dur = time.perf_counter() - t0
            metrics.observe(
                'armenu_http_request_duration_seconds', dur,
                endpoint=request.resolver_match.route if request.resolver_match else 'unknown',
                method=request.method,
                status=getattr(response, 'status_code', 500),
            )
            if dur > 1.0:
                logger.warning('SEKIN %.2fs %s %s trace=%s rest=%s',
                               dur, request.method, request.path, trace_id, _local.restaurant_id)
        response['X-Trace-Id'] = trace_id
        return response
```

```
# core/logging.py — har log qatorida trace_id va restaurant_id
class ContextFilter(logging.Filter):
    def filter(self, record):
        record.trace_id      = getattr(_local, 'trace_id', '-')
        record.restaurant_id = getattr(_local, 'restaurant_id', '-')
        return True

# format: %(asctime)s %(levelname)s [%(trace_id)s r=%(restaurant_id)s] %(name)s: %(message)s
```

⚡ **Qoida:** `enqueue()` trace_id ni payload ichida **worker'ga uzatadi** — bitta buyurtmaning web → navbat → worker → tashqi API yo'lini uchidan-uchiga ko'rish mumkin.

Buzilsa nima bo'ladi:
🟡 Xato 3 kun aniqlanmaydi. MTTR (o'rtacha tiklash vaqti) 5 daqiqadan 4 soatga chiqadi. SLO 99,9% (yiliga 8,8 soat) — bitta hodisa yillik byudjetni yeb qo'yadi.

P11 — Fail-Fast (tez xato ber)

Ta'rif: Noto'g'ri holat aniqlansa — **darhol** xato qaytariladi, "balki ishlar" deb davom etilmaydi. Validatsiya chegarada (serializer), tashqi chaqiruvda timeout, konfiguratsiya xatosi — start paytida.

Nima uchun: Sekin xato — yashirin xato. `restaurant_id` topilmasa `None` bilan davom etish = barcha restoran ma'lumotini ko'rsatish (xavfsizlik hodisasi).

Amalda:

```
# core/middleware.py
class TenantMiddleware:
    def __call__(self, request):
        rid = self._resolve(request)          # subdomain / QR token / staff session
        if rid is None and self._needs_tenant(request.path):
            raise TenantNotResolved()          # ⚡ 400, davom etmaydi
        request.restaurant_id = rid
        _local.restaurant_id = rid
        return self.get_response(request)


# core/mixins.py
class TenantScopedMixin:
    def get_queryset(self):
        rid = getattr(self.request, 'restaurant_id', None)
        if rid is None:
            raise PermissionDenied('Restoran aniqlanmadi')   # ⚡ hech qachon .all()
        return super().get_queryset().filter(**{self.tenant_field: rid})
```

```
# config/settings.py — start paytida tekshiruv, ishga tushmasdan
REQUIRED_ENV = ['DATABASE_URL', 'REDIS_URL', 'SECRET_KEY', 'S3_BUCKET', 'PAYME_MERCHANT_ID']
missing = [k for k in REQUIRED_ENV if not os.getenv(k)]
if missing:
    raise ImproperlyConfigured(f'.env da yo\'q: {missing}')   # ⚡ deploy shu yerda to'xtaydi
```

```
# ✖ Sekin xato — eng xavfli naqsh
try:
    price = calc_price(items)
except Exception:
    price = 0                    # buyurtma 0 so'mga o'tadi!

# ✔ Tez xato
price = calc_price(items)        # xato bo'lsa 500, buyurtma yaratilmaydi
```

Buzilsa nima bo'ladi:
🔴 Yashirilgan xato ma'lumot buzilishiga olib keladi. `except: pass` — 0 so'mlik buyurtmalar, `restaurant_id=None` — cross-tenant sizish. Bularni **oylardan keyin** buxgalteriya topadi.

P12 — Defense-in-Depth (ko'p qatlamli himoya)

Ta'rif: Har muhim invariant **kamida 3 qatlamda** himoyalangan: kod (mixin) + DB (constraint) + CI (test). Bitta qatlam buzilsa boshqasi ushlaydi.

Nima uchun: Dasturchi `TenantScopedMixin` ni unutishi mumkin. Kod ko'rigi o'tkazib yuborishi mumkin. Uch qatlam bir vaqtda buzilishi ehtimoli juda past.

Amalda — misol: multi-tenant izolyatsiya (3 qatlam):

```
# QATLAM 1 — kod
class DishViewSet(TenantScopedMixin, ModelViewSet):
    tenant_field = 'category__restaurant_id'

# QATLAM 2 — DB (PostgreSQL Row-Level Security, S2 dan)
# ALTER TABLE menu_dish ENABLE ROW LEVEL SECURITY;
# CREATE POLICY tenant_isolation ON menu_dish
#   USING (restaurant_id = current_setting('app.restaurant_id')::int);

# QATLAM 3 — CI test (deploy bloklaydi)
@pytest.mark.parametrize('model,url', TENANT_MODELS)
def test_cross_tenant_403(client, model, url):
    r1, r2 = make_restaurant(), make_restaurant()
    obj = make_obj(model, restaurant=r2)
    resp = client.get(f'{url}{obj.id}/', HTTP_HOST=f'{r1.slug}.armenu.uz')
    assert resp.status_code in (403, 404), f'{model} SIZDI!'
```

Misol 2 — narx himoyasi (3 qatlam):

Qatlam	Chora
Kod	Serializer `price` maydonini **qabul qilmaydi** (`read_only`)
Servis	`calc_total()` faqat DB'dagi `Dish.price` dan hisoblaydi
CI	`test_price_from_client_ignored` — klient 1 so'm yuborsa server 45 000 hisoblaydi

Misol 3 — to'lov (4 qatlam): idempotency key → DB unique constraint → provayder webhook imzosi (HMAC) → summa moslik tekshiruvi.

Buzilsa nima bo'ladi:
🔴 Yagona qatlam = yagona xato nuqtasi. Bitta unutilgan `filter()` — GDPR hodisasi, jarima, litsenziya yo'qotish, obro' zarari. Qaytarib bo'lmaydi.

---

§1.1 Printsiplar qisqacha jadvali

#	Printsip	Asosiy amaliyot	Buzilish narxi	CI tekshiruvi
P1	Stateless	Session→Redis, fayl→S3	🔴 Auto-scale ishlamaydi	`test_no_local_state.py`
P2	Shared-nothing	Bir xil instans, NFS yo'q	🔴 Kaskad yiqilish	Deploy ko'rigi
P3	Graceful degradation	7 ta fallback zanjiri	🔴 Biznes to'xtaydi	`test_fallback_chain.py`
P4	Idempotency	X-Idempotency-Key + unique	🔴 Dublikat to'lov	`test_idempotency.py`
P5	Event-driven	Signal + on_commit	🟡 Modul chegarasi yo'qoladi	`test_no_cross_import.py`
P6	CQRS-lite	DB router + replica	🟡 Primary o'ladi	Yuklama testi
P7	Cache-first	Versiyalangan kalit	🔴 DB o'ladi	`test_cache_hit_ratio.py`
P8	Circuit breaker	Har tashqi API	🔴 Kaskad yiqilish	`test_breaker_opens.py`
P9	Bulkhead	5 navbat, ajratilgan pool	🔴 Fiskal to'xtaydi	`test_queue_isolation.py`
P10	Observability	trace_id + metric + log	🟡 MTTR 4 soat	Metric mavjudligi
P11	Fail-fast	Validatsiya chegarada	🔴 Ma'lumot buzilishi	`test_no_silent_except.py`
P12	Defense-in-depth	3 qatlam har invariant	🔴 Ma'lumot sizishi	`test_tenant_isolation.py`

§2. ARXITEKTURA QARORLARI (ADR)

> **ADR formati:** har qaror **sanasi**, **konteksti**, **muqobillari** va **qaytarish narxi** bilan yozilgan. Qaror o'zgarsa — yangi ADR yoziladi, eskisi "Almashtirildi" holatiga o'tadi (o'chirilmaydi).

ADR xaritasi:

```
ARXITEKTURA          TEXNOLOGIYA          MAHSULOT/JARAYON
─────────────        ─────────────        ─────────────────
001 Modulli monolit  002 PG+pgvector      011 3D kutubxona
005 WS strategiya    003 Vanilla JS       014 paytechuz
007 Tenant izolyatsiya 004 model-viewer   015 Faza-asosli
009 MCP qatlam       008 Redis navbat
013 ClickHouse       010 systemd
                     012 REST
XAVFSIZLIK
──────────
006 Narx serverda
```

ADR-001 — Modulli monolit (mikroservis emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
ARMENU 13 ta funksional sohaga bo'linadi (menyu, 3D, buyurtma, to'lov, fiskal, AI, ...).
Jamoa — 1-2 dasturchi. v1 muddati — 15 hafta. 5 yillik maqsad — 15 000 restoran.
Savol: har sohani alohida servis qilib, mustaqil deploy qilaylikmi, yoki bitta kod bazasida qat'iy modul chegaralari bilan qolaylikmi?

Qaror:
Modulli monolit. Bitta Django loyihasi, bitta deploy, `apps/` ichida 13 ta mustaqil app. Modullar bir-birini **faqat servis funksiyasi va event orqali** chaqiradi.
Har modul o'z jadvallariga egalik qiladi. Kelajakda ajratish uchun chegaralar hozirdan qat'iy.

Sabablar:

Sabab	Tushuntirish
Jamoa hajmi	1-2 dasturchi. Mikroservis Conway qonuni bo'yicha 5+ jamoa uchun
Tranzaksiya yaxlitligi	Buyurtma + stol holati + narx — **bitta ACID tranzaksiya**. Saga naqshi bu yerda ortiqcha murakkablik
Yetkazish tezligi	15 haftada v1. Mikroservisda ~2× uzoq (infra, kontrakt, kuzatuv)
Operatsiya	1 deploy, 1 log oqimi, 1 monitoring dashboard, 1 migratsiya
Narx	S0-S1 uchun 1 VPS ($40/oy). Mikroservis minimal 5 node
Debug	Uchidan-uchiga stack trace bitta process ichida

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Mikroservis (13 servis)	Mustaqil deploy/masshtab	1-2 dasturchi uchun operatsion yuk qotil; taqsimlangan tranzaksiya (saga) kerak; tarmoq kechikishi buyurtma p95 ni 200→600ms qiladi
Serverless (Lambda)	To'lov faqat ishlatilganda	Cold start 1-3s (menyu 100ms bo'lishi kerak); WS qo'llab-quvvatlash zaif; O'zbekistonda region yo'q (latency)
Yaxlit monolit (modulsiz)	Eng tez yozish	2-yilda "katta loy uyumi"; ajratish imkonsiz; testlash qiyin
Makroservis (3 servis: web/3d/ai)	O'rta yo'l	Hozirdan 3 ta deploy; foydasi hali yo'q (ADR-001 bu yo'lni **kelajakda** ochiq qoldiradi)

Oqibatlari:
- ✅ v1 muddatiga yetish real; 1 VPS'da to'liq ishlaydi
- ✅ Buyurtma yaratish bitta tranzaksiyada — dublikat/nomuvofiqlik yo'q
- ✅ Yangi dasturchi 1 kunda butun tizimni lokal ko'taradi
- ⚠ Bitta modulning og'ir kodi butun process RAM'ini yeydi (bulkhead P9 yumshatadi)
- ⚠ Deploy — hammasi birga; bitta modul xatosi butun tizimni to'xtatishi mumkin
- ⚠ Kod bazasi o'sib ketishi xavfi — modul chegarasi **CI'da tekshiriladi** (`test_no_cross_import.py`)
- ♻ **Qaytarish narxi:** past-o'rta. Chegaralar qat'iy bo'lgani uchun `ar3d` yoki `ai` ni ajratish ~2 hafta (servis funksiyasi → HTTP/gRPC adapter, event → message bus).
Ajratish rejasi `00_UMUMIY_KORINISH.md` §6 da.

Qachon qayta ko'riladi:
- `ar3d`: 3D generatsiya > 500/kun **yoki** GPU node kerak bo'lganda
- `ai`: AI so'rov > 100 000/oy
- `analytics`: event > 10 mln/oy (ADR-013 bilan birga)
- Jamoa > 6 dasturchi bo'lganda (Conway qonuni)

ADR-002 — PostgreSQL 16 + pgvector (alohida vektor DB emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
Tizimga kerak: tranzaksion ma'lumot (buyurtma, to'lov — ACID majburiy), yarim strukturali ma'lumot (taom atributlari, modifikatorlar — JSONB), to'liq matnli qidiruv (menyu bo'ylab), va **semantik qidiruv** (AI chatbot: "achchiq bo'lmagan go'shtli taom" → taom ro'yxati). Vektor o'lchami 1536 (OpenAI/Gemini embedding), taxminan 15 000 restoran × 80 taom × 3 til = ~3,6 mln vektor (5-yil bashorati).

Qaror:
PostgreSQL 16 yagona ma'lumot bazasi + `pgvector` kengaytmasi. Alohida vektor bazasi (Pinecone/Qdrant/Weaviate) **ishlatilmaydi**. Analitik event'lar bundan mustasno (ADR-013 → ClickHouse).

Sabablar:

Sabab	Tushuntirish
Bitta saqlagich	Bitta backup, bitta failover, bitta monitoring, bitta connection pool
Tranzaksion yaxlitlik	Taom o'chirilsa vektori ham **shu tranzaksiyada** o'chadi. Alohida DB'da — sinxronizatsiya muammosi
JOIN imkoniyati	`WHERE restaurant_id=? AND is_active AND NOT in_stoplist ORDER BY embedding <=> ?` — bitta so'rovda. Vektor DB'da: 2 so'rov + qo'lda filtr
Multi-tenant filtri	`restaurant_id` bo'yicha filtrlash SQL'da tabiiy; vektor DB'da metadata filtri sekin
Hajm yetarli	3,6 mln vektor × 1536 dim — HNSW indeks ~12 GB. Bitta PG node bemalol
Narx	$0 qo'shimcha. Pinecone ~$70/oy dan boshlanadi, hajm o'sishi bilan $500+
Jamoa tajribasi	PostgreSQL — ma'lum; yangi DB = yangi operatsion bilim

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Pinecone	Boshqariladigan, tez	Pulli; tarmoq kechikishi; sinxronizatsiya muammosi; vendor lock-in; O'zbekistonga latency
Qdrant (self-hosted)	Tez, ochiq	+1 servis operatsiya; +1 backup; JOIN yo'q; hozircha foydasi yo'q
Elasticsearch	FTS + vektor	Og'ir (JVM, 4GB+ RAM); operatsiya murakkab; ACID yo'q
MySQL	Keng tarqalgan	JSONB zaif, `pgvector` yo'q, partition cheklangan, GIN indeks yo'q
MongoDB	Sxemasiz	Tranzaksiya zaif (to'lov uchun yaramaydi); JOIN yo'q

Oqibatlari:
- ✅ Bitta DB — operatsiya 3× oddiy (backup/failover/monitoring)
- ✅ Semantik qidiruv + SQL filtr bitta so'rovda: `<=>` operatori + `WHERE`
- ✅ `JSONB` + `GIN` indeks: taom atributlari sxema o'zgarishisiz kengayadi
- ⚠ Vektor indeks (HNSW) qurilishi sekin: 1 mln vektorga ~20 daqiqa. Yechim: `CONCURRENTLY`
- ⚠ Vektor so'rovlari primary CPU'ni yeydi → alohida read replica'ga yo'naltiriladi
- ⚠ pgvector ixtisoslashgan DB'dan ~2-3× sekinroq (bizning hajmda ahamiyatsiz: 15ms vs 5ms)
- 🔄 **Qaytarish narxi:** past. Embedding'lar qayta hisoblanmaydi — `vector` ustunini eksport qilib Qdrant'ga yuklash ~1 kun. Interfeys `apps/ai/search.py` da izolyatsiyalangan.

Qachon qayta ko'riladi:
- Vektor soni > 20 mln **yoki** semantik qidiruv p95 > 150 ms
- HNSW indeks RAM'ga sig'masa (>50 GB)

ADR-003 — Vanilla JS + PWA (React/Vue emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
Mijoz interfeysi — restoranda, stolda, **mobil tarmoqda** (ko'pincha 3G/sekin 4G) ochiladi. Foydalanuvchi QR skanerlaydi va menyuni **2 soniyada** ko'rishi kerak. Turist bo'lsa — roaming, sekin ulanish. Sahifa AR ko'rish uchun `<model-viewer>` (~250 KB) yuklaydi — bu allaqachon og'ir. Interfeys murakkabligi: menyu ro'yxati, taom kartochkasi, savat, buyurtma holati — o'rtacha, SPA murakkabligiga yetmaydi.

Qaror:
Vanilla JS (ES2022 modullar) + PWA. Framework yo'q. Django template'lar server-side render, JS faqat interaktivlik uchun. Holat boshqaruvi — oddiy observable store (~80 qator). Build qadam — minimal (`esbuild` bundle + minify).

Sabablar:

Sabab	Tushuntirish
Yuklanish hajmi	Vanilla ~45 KB gzip; React+ReactDOM+router+state ~140 KB gzip. 3G'da bu 1,5s farq
`<model-viewer>` byudjeti	AR kutubxonasi 250 KB. Framework qo'shilsa jami 400 KB+ — QR→menyu 2s maqsadi buziladi
SEO	Restoran menyusi Google'da indekslanishi kerak (organik trafik). SSR framework'siz tabiiy
Murakkablik darajasi	4 ta ekran + savat. Bu SPA emas — MPA + progressiv yaxshilash yetadi
Jamoa	Backend dasturchilar. Django template'lar allaqachon ishlatiladi
Uzoq umr	Vanilla JS 5 yildan keyin ham ishlaydi. React 16→17→18→19 migratsiyalari — texnik qarz
Build murakkabligi	Webpack/Vite konfiguratsiyasi, node_modules 300 MB, CI sekinlashadi

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
React + Next.js	Ekotizim, komponentlar	+140 KB; SSR uchun Node servis (+1 deploy); Django bilan ikkilik arxitektura
Vue 3	Yengilroq (~34 KB)	Baribir build tooling; foydasi bizning murakkablikda sezilmaydi
Svelte	Kichik bundle, compile-time	Build zarur; jamoa tajribasi yo'q; ekotizim kichik
HTMX + Alpine	Yengil, Django'ga mos	Jiddiy ko'rildi ✅. Savat va AR holati uchun imperativ JS baribir kerak → aralash yondashuv chalkash bo'ladi
Flutter Web	Bitta kod (mobil+web)	Bundle 2 MB+; SEO yo'q; AR integratsiyasi murakkab

> ⚠ **Istisno: admin panel** — Django admin + server-render + kichik JS.
> Admin uchun ham SPA qilinmaydi (ichki foydalanuvchi, tezlik talabi past).

Oqibatlari:
- ✅ Birinchi kontent (FCP) < 1,2s 4G'da; menyu to'liq < 2s
- ✅ Node.js runtime **kerak emas** — deploy sodda (ADR-010 bilan mos)
- ✅ Har sahifa mustaqil — bitta JS xatosi butun ilovani o'ldirmaydi
- ⚠ Komponent qayta ishlatish qo'lda — konvensiya bilan hal qilinadi (`static/js/components/`)
- ⚠ Murakkab holat (savat + modifikatorlar + real-time holat) qo'lda boshqariladi → `store.js` (observable naqsh) yozildi, testlar bilan qoplanadi
- ⚠ Yangi dasturchi "React yo'qmi?" deb hayron bo'ladi — onboarding hujjatida tushuntiriladi
- 🔄 **Qaytarish narxi:** o'rta-yuqori. Frontend qayta yozish ~4-6 hafta.
Lekin API (ADR-012, REST) o'zgarmaydi — faqat prezentatsiya qatlami.

Qachon qayta ko'riladi:
- Mijoz ekranlari > 15 ta **yoki** holat mantiqi `store.js` da 600 qatordan oshsa
- Jamoada 3+ frontend dasturchi paydo bo'lsa
- ⛔ **Marketplace mobil ilovasi** (yil 4) — u alohida loyiha, bu ADR unga tegishli emas

---

ADR-004 — `<model-viewer>` (Three.js emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
Mahsulotning yadro qiymati — mijoz **taomni stolda real o'lchamda** ko'rishi.
Bu iOS (Safari) va Android (Chrome) da ishlashi shart. iOS'da WebXR **yo'q** —
faqat **AR Quick Look** (USDZ formati) mavjud. Android'da**Scene Viewer** (GLB).
Ikkalasini bitta kod bilan qoplash kerak.

Qaror:
Google'ning `<model-viewer>` web-komponenti. 3D fayl 2 formatda saqlanadi:
`.glb` (Android/web) va `.usdz` (iOS). Fallback zanjiri: AR → 3D viewer → 360° → rasm.

Sabablar:

Sabab	Tushuntirish
iOS + Android bitta kod	`ar-modes="webxr scene-viewer quick-look"` — brauzer o'zi tanlaydi
Google qo'llab-quvvatlaydi	Faol rivojlanmoqda, Scene Viewer bilan birinchi darajali integratsiya
Deklarativ	`<model-viewer src="..." ar camera-controls>` — 1 qator. Three.js'da ~300 qator
Tayyor UX	Yuklanish indikatori, poster rasm, kamera nazorati, AR tugmasi — hammasi ichida
Ishlash	Ichida Three.js bor, lekin optimallashtirilgan va tree-shaken
Ta'mirlash	Brauzer AR API o'zgarsa — Google yangilaydi, biz emas

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Three.js (to'g'ridan)	To'liq nazorat, effektlar	AR o'zi **yo'q** — WebXR ni qo'lda yozish kerak; iOS'da baribir Quick Look'ga o'tish; ~600 KB; ta'mirlash bizda
8th Wall	Kuchli marker'siz AR	Pulli ($99+/oy per app); vendor lock-in; ortiqcha (bizga world-tracking yetadi)
Babylon.js	Kuchli dvigatel	Og'ir (~1 MB); AR qo'llab-quvvatlashi Three.js bilan bir xil muammo
Native ilova (ARKit/ARCore)	Eng yaxshi AR sifati	Mijoz ilova o'rnatmaydi ⛔ — QR→brauzer yagona real yo'l
Zappar / Blippar	Tayyor platforma	Pulli; brendlangan; menyu bilan integratsiya cheklangan

Oqibatlari:
- ✅ Bitta `<model-viewer>` tegi — iOS va Android'da AR ishlaydi
- ✅ Fallback avtomatik: AR yo'q qurilmada 3D aylantirish qoladi
- ✅ Kutubxona CDN'dan (`cdnjs`) — bizning bandwidth'imizga tushmaydi
- ⚠ **Har model 2 nusxada** (GLB + USDZ) → storage 2×, konversiya pipeline'i kerak (`worker:model3d` da `usdzconvert`)
- ⚠ Maxsus effektlar (soya sozlash, custom shader) cheklangan
- ⚠ Gaussian Splatting (yangi, fotorealistik) qo'llab-quvvatlanmaydi — AR Quick Look ham qo'llab-quvvatlamaydi, shuning uchun yo'qotish yo'q
- 🔄 **Qaytarish narxi:** past. `<model-viewer>` DOM qatlamida izolyatsiyalangan (`static/js/ar/viewer.js`, ~120 qator). Three.js'ga o'tish ~1 hafta, lekin AR'ni qayta yozish kerak (+3 hafta).

Qachon qayta ko'riladi:
- Apple Safari'da WebXR ochilsa → yagona format (GLB) ga o'tish mumkin
- Maxsus AR o'zaro ta'sir (taomni "yeyish" animatsiyasi, o'lcham o'zgartirish) talab qilinsa

---

ADR-005 — Mijozga WebSocket faqat buyurtmadan keyin

Holat: ✅ Qabul qilindi · 2026-09-04 · ⚡**Eng muhim masshtab qarori**

Kontekst:
5-yillik maqsad: 1 mln bir vaqtda faol foydalanuvchi. Sodda yondashuv — har mijozga sahifa ochilishida WebSocket ulanish. Hisob: 1 ulanish ≈ 20-40 KB RAM (Daphne) → 1 mln ulanish = **20-40 GB RAM** faqat ulanishlar uchun, plus Redis channel layer pub/sub yuklamasi, plus 20 ta Daphne instansi (~$1 600/oy).

Lekin **yuklama profili** shuni ko'rsatadi: mijozlarning ~60% faqat **menyu ko'radi** (hech narsa buyurtma qilmaydi), 15% AR'ga qaraydi, faqat **~5% buyurtma beradi**. Menyu ko'rayotgan mijozga real-time hech narsa kerak emas — menyu 60s kesh bilan ham yetarli.

Qaror:
Mijozga WebSocket ulanishi faqat buyurtma yaratilgandan KEYIN ochiladi.
Menyu ko'rish, AR, savat — WS'siz. Buyurtma holati (`accepted → cooking → ready`) uchun WS ochiladi va buyurtma yopilganda **yopiladi**.

Xodimlar (ofitsiant, KDS, admin) — WS**doim ochiq** (ular kam sonli).



Sabablar:

Sabab	Tushuntirish
RAM tejash	20-40 GB → 2-4 GB. WS node soni 20 → 3
Redis pub/sub	1 mln obunachi → 90 k. Channel layer yuklamasi 11× kam
Batareya	Mobil qurilmada ochiq WS — ping/pong har 20s, batareya yeyadi
Tarmoq beqarorligi	Restoran Wi-Fi/4G. Uzilib-ulanish sikli har mijozda — behuda yuk
Real ehtiyoj	Menyu ko'rayotgan mijozga push qilinadigan narsa yo'q (stop-list — 15s polling yetadi)
Narx	$1 600/oy → $240/oy (WS qatlami)

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Hammaga WS	Sodda, bir xil kod	20-40 GB RAM; $1 600/oy; batareya; 1 mln'da Redis pub/sub bo'g'ilishi
Faqat polling	Eng oddiy	Buyurtma holati 10s kechikadi — "Tayyor!" bildirishnomasi kech; UX zaif
SSE (Server-Sent Events)	Yengilroq, HTTP/2	Bir tomonlama (mijozdan server'ga yo'q); HTTP/2 stream limiti; ✅ **fallback sifatida ishlatiladi**
Long polling	Universal	Har so'rov gunicorn thread'ini band qiladi — WS'dan yomonroq
Push notification (FCM)	WS umuman kerak emas	Ruxsat so'rash kerak (mijoz bermaydi); iOS Safari cheklovlari; ✅ **qo'shimcha** sifatida

Oqibatlari:
- ✅ 1 mln bir vaqtda 3 ta WS node bilan ishlaydi (20 ta o'rniga)
- ✅ WS qatlami yiqilsa — menyu, AR, savat **ta'sirlanmaydi** (P3 bilan mos)
- ✅ Mijoz batareyasi tejaladi
- ⚠ Stop-list o'zgarishi mijozga darhol yetmaydi → **15s polling** bilan hal qilindi (`GET /v1/menu/<slug>/stoplist/` — juda yengil, faqat ID ro'yxati, ETag bilan)
- ⚠ Ikki xil real-time kod yo'li (WS va polling) — testlash hajmi ortadi
- ⚠ Buyurtma yaratilganda WS ulanish "portlash"i (5% mijoz bir vaqtda) — ulanish rate limiti va exponential backoff kerak
- 🔁 **Qaytarish narxi:** past. Frontend'da `realtime.js` abstraksiyasi bor — `connect()` chaqiruvini sahifa ochilishiga ko'chirish 1 qator. Lekin infra narxi 7× oshadi.

Qachon qayta ko'riladi:
- Agar mijozga real-time kerak bo'ladigan yangi funksiya paydo bo'lsa (masalan, jonli ovqat tayyorlash translyatsiyasi)
- WS texnologiyasi arzonlashsa (masalan WebTransport keng tarqalsa)

ADR-006 — Narx faqat serverda hisoblanadi

Holat: ✅ Qabul qilindi · 2026-09-04 · 🔒 **Xavfsizlik qarori — muzokara qilinmaydi**

Kontekst:
Mijoz PWA'da savat quradi: taom + modifikator (qo'shimcha pishloq +8 000) + miqdor + promo-kod + servis haqi (10%) + soliq. Klient qulaylik uchun jami summani ko'rsatadi. Savol: klient hisoblagan summani serverga yuborsinmi (tez, oddiy) yoki server qaytadan hisoblasinmi?

Klient kodi — **butunlay ishonchsiz muhit**. Har kim DevTools ochib `{"total": 1}` yuborishi mumkin.

Qaror:
Server barcha summani noldan hisoblaydi. Klientdan **faqat**: `dish_id`, `qty`, `modifier_ids`, `promo_code`. `price`, `total`, `discount` maydonlari serializer'da **qabul qilinmaydi** (yuborilsa — jim tashlanadi, aniqroq: `400` qaytariladi). Klient hisobi — faqat **ko'rsatish uchun**; yakuniy summa serverdan qaytadi va mijozga tasdiqlash oynasida ko'rsatiladi.

Sabablar:

Sabab	Tushuntirish
Moliyaviy xavfsizlik	Klientga ishonish = to'g'ridan-to'g'ri pul yo'qotish
Fiskal talab	OFD'ga yuboriladigan chek summasi **haqiqiy** bo'lishi shart (soliq javobgarligi)
Yagona haqiqat manbai	Narx `Dish.price` da. Promo mantiqi `apps/loyalty` da. Klient nusxasi eskiradi
Audit	Har buyurtmada narx qanday hisoblangani `Order.price_breakdown` (JSONB) da saqlanadi
Race condition	Mijoz savat ochiq turganda admin narxni o'zgartirishi mumkin — server joriy narxni oladi

Amalda:

```
# apps/orders/serializers.py
class OrderItemInSerializer(serializers.Serializer):
    dish_id      = serializers.IntegerField()
    qty          = serializers.IntegerField(min_value=1, max_value=50)
    modifier_ids = serializers.ListField(child=serializers.IntegerField(), required=False)
    note         = serializers.CharField(max_length=200, required=False, allow_blank=True)
    # ⛔ price, total, discount — YO'Q. Bo'lishi ham mumkin emas.


# apps/orders/pricing.py
def calc_order_total(*, restaurant, items_in, promo_code=None) -> PriceResult:
    """Yagona narx hisoblash nuqtasi. Boshqa hech qayerda narx hisoblanmaydi."""
    dishes = {d.id: d for d in Dish.objects.filter(
        id__in=[i['dish_id'] for i in items_in],
        category__restaurant_id=restaurant.id,      # ⚡ tenant tekshiruvi
        is_active=True,
    ).select_related('category')}

    lines, subtotal = [], Decimal('0')
    for it in items_in:
        dish = dishes.get(it['dish_id'])
        if dish is None:
            raise ValidationError({'dish_id': f"{it['dish_id']} — mavjud emas yoki faol emas"})
        if dish.id in stoplist_ids(restaurant.id):
            raise ValidationError({'dish_id': f'{dish.name} — hozircha yo\'q'})

        mods = Modifier.objects.filter(id__in=it.get('modifier_ids', []), dish=dish)
        mod_sum = sum(m.price for m in mods)
        line = (dish.price + mod_sum) * it['qty']    # ⚡ DB narxi, klient emas
        subtotal += line
        lines.append({'dish_id': dish.id, 'unit': str(dish.price),
                      'mods': str(mod_sum), 'qty': it['qty'], 'line_total': str(line)})

    discount = apply_promo(restaurant, promo_code, subtotal)
    service  = (subtotal - discount) * restaurant.service_fee_pct / 100
    total    = (subtotal - discount + service).quantize(Decimal('1'))   # so'm — butun son

    return PriceResult(subtotal=subtotal, discount=discount,
                       service=service, total=total, breakdown=lines)
```

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Klient hisoblaydi, server tekshiradi	Bitta hisob kamroq	Server baribir hisoblashi kerak (tekshirish uchun) — foyda yo'q, murakkablik ko'p
Klient narxni imzolaydi (HMAC)	Server hisoblamaydi	Kalit klientda bo'lishi kerak → o'g'irlanadi. Klassik anti-pattern
Narxli token (server bergan)	Tez	Token muddati, savat o'zgarishi, promo — murakkablik server hisobidan katta

Oqibatlari:
- ✅ Narx manipulyatsiyasi **texnik jihatdan imkonsiz**
- ✅ Fiskal chek summasi doim to'g'ri
- ✅ `Order.price_breakdown` — har buyurtmada to'liq audit izi
- ⚠ Klient ko'rsatgan summa serverdagidan farq qilishi mumkin (narx o'zgargan bo'lsa) → tasdiqlash oynasida "Narx yangilandi: 45 000 → 47 000" ko'rsatiladi
- ⚠ Har buyurtmada 2 qo'shimcha DB so'rovi (dishes + modifiers) — `select_related` va kesh bilan p95 byudjetiga (40 ms) sig'adi
- 🔁 **Qaytarish narxi: ⛔ qaytarilmaydi.** Bu Oltin qoida #2 (`00_UMUMIY_KORINISH.md` §9). CI testi `test_price_calculation.py` deploy'ni bloklaydi.

Qachon qayta ko'riladi: ⛔ Hech qachon.

ADR-007 — Multi-tenant izolyatsiya: Middleware + Mixin (schema/DB-per-tenant emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
Platforma 15 000 restoranga xizmat qiladi. Har restoran o'z menyusi, buyurtmalari, xodimlari va **moliyaviy ma'lumoti** bilan. Bir restoran boshqasining ma'lumotini ko'rmasligi — **mutlaq talab** (aks holda: GDPR/shaxsiy ma'lumot buzilishi, raqobat ma'lumoti sizishi, litsenziya yo'qotish).

Uch klassik yondashuv: (a) database-per-tenant, (b) schema-per-tenant, (c) shared schema + `restaurant_id` ustuni.

Qaror:
Shared schema + `restaurant_id` ustuni, uch qatlamli himoya bilan (P12):
1. `TenantMiddleware` — har so'rovda `restaurant_id` ni aniqlaydi (subdomain / QR token / staff session)
2. `TenantScopedMixin` — har ViewSet queryset'ni avtomatik filtrlaydi
3. CI testi — har model uchun cross-tenant urinish 403/404 qaytarishi tekshiriladi

S2 bosqichidan **PostgreSQL Row-Level Security** to'rtinchi qatlam sifatida yoqiladi.

Sabablar:

Sabab	Tushuntirish
Migratsiya	15 000 schema'da `migrate` — soatlar. Shared schema'da — daqiqalar
Connection pool	Schema-per-tenant'da har tenant uchun `SET search_path` → pooling buziladi
Platforma hisoboti	"Barcha restoran bo'yicha o'rtacha chek" — bitta `GROUP BY`. 15 000 schema'da — imkonsiz
Kesh	`menu:{rid}:{lang}:v{n}` — tabiiy. Schema'da kesh kaliti murakkablashadi
Resurs	15 000 DB = 15 000 connection slot, 15 000 backup. Real emas
MebelCity tajribasi	Platformada DB-per-tenant bor — **operatsion og'irligi ma'lum**. ARMENU'da restoran soni 100× ko'p → bu yo'l yaramaydi

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Database-per-tenant	Eng kuchli izolyatsiya; oson eksport/o'chirish	15 000 DB — migratsiya soatlar; connection pool portlaydi; backup 15 000 ta; narx
Schema-per-tenant	Yaxshi izolyatsiya, bitta DB	PostgreSQL'da >1 000 schema — `pg_catalog` sekinlashadi; migratsiya sekin; pooling muammosi
Shared schema, filtrsiz (qo'lda)	Eng oddiy	⛔ Bitta unutilgan `filter()` = ma'lumot sizishi. Inson xatosiga tayangan
Faqat RLS (mixin'siz)	DB darajasida kafolat	`current_setting()` ni har connection'da o'rnatish kerak; pooling bilan xavfli; xato qilingan sozlash = to'liq ochiq

Amalda:

```python
# core/middleware.py
class TenantMiddleware:
    """restaurant_id ni 3 manbadan aniqlaydi. Topilmasa — fail-fast (P11)."""

    PUBLIC_PATHS = ('/healthz', '/metrics', '/platform-admin/', '/static/')

    def __call__(self, request):
        rid = None

        # 1) QR token (mijoz): /t/<token>/ yoki X-Table-Token
        token = request.resolver_match.kwargs.get('token') if request.resolver_match else None
        token = token or request.headers.get('X-Table-Token')
        if token:
            rid = resolve_table_token(token)          # Redis kesh, 600s

        # 2) Subdomain: <slug>.armenu.uz
        if rid is None:
            host = request.get_host().split(':')[0]
            if host.endswith('.armenu.uz'):
                rid = resolve_slug(host.split('.')[0])

        # 3) Xodim sessiyasi (ofitsiant/admin)
        if rid is None and request.user.is_authenticated:
            rid = getattr(request.user, 'restaurant_id', None)

        if rid is None and not request.path.startswith(self.PUBLIC_PATHS):
            raise TenantNotResolved()                 # ⚡ 400, davom etmaydi

        request.restaurant_id = rid
        _local.restaurant_id  = rid                   # log/metrik uchun
        return self.get_response(request)


# core/mixins.py
class TenantScopedMixin:
    """
    tenant_field — model'dan restaurant'ga yo'l.
    Har ViewSet'da MAJBURIY. Yo'q bo'lsa — TypeError (fail-fast).
    """
    tenant_field: str = 'restaurant_id'

    def get_queryset(self):
        rid = getattr(self.request, 'restaurant_id', None)
        if rid is None:
            raise PermissionDenied('Restoran aniqlanmadi')
        return super().get_queryset().filter(**{self.tenant_field: rid})

    def perform_create(self, serializer):
        serializer.save(restaurant_id=self.request.restaurant_id)   # klient yubora olmaydi


# apps/menu/views.py
class DishViewSet(TenantScopedMixin, ModelViewSet):
    queryset     = Dish.objects.select_related('category')
    tenant_field = 'category__restaurant_id'
```

Oqibatlari:
- ✅ Bitta DB, bitta migratsiya, bitta backup — operatsiya sodda
- ✅ Platforma darajasidagi analitika tabiiy (`GROUP BY restaurant_id`)
- ✅ Yangi restoran ochish — bitta `INSERT` (schema yaratish emas)
- ⚠ Izolyatsiya **kodga bog'liq** — shuning uchun P12 (3 qatlam) majburiy
- ⚠ Bitta restoran "shovqinli qo'shni" bo'lishi mumkin (og'ir so'rov) → `statement_timeout` va PgBouncer bulkhead (P9) bilan yumshatiladi
- ⚠ Restoran ma'lumotini o'chirish (GDPR "unutilish huquqi") — `DELETE ... WHERE restaurant_id=?` har jadvalda; `apps/tenants/services.py:purge_restaurant()` da markazlashtirilgan
- 🔄 **Qaytarish narxi:** yuqori. Shared → schema-per-tenant migratsiyasi katta ish (~6 hafta). Lekin **kerak bo'lmaydi**: sharding yo'li (`restaurant_id` hash) ochiq va arzonroq.

Qachon qayta ko'riladi:
- Yirik korporativ mijoz **ajratilgan DB** talab qilsa → gibrid: shared + VIP tenant uchun alohida DB
- Regulyator ma'lumotni jismonan ajratishni talab qilsa

ADR-008 — Redis navbat (Celery emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
Fon vazifalar kerak: 3D generatsiya (2-8 daqiqa), fiskal chek yuborish (kritik), Telegram bildirishnoma, AI so'rov, analitika ETL. Django ekotizimida standart tanlov — Celery. Lekin MebelCity platformasida **Celery jimgina o'lgan** tajribasi bor (worker broker'ga ulanadi, task qabul qiladi, lekin bajarmaydi — 3 kun sezilmagan), va u yerda thread-asosli yechimga o'tilgan.

Bizga kerak: bulkhead (P9 — 5 alohida navbat), orphan job tiklash, DLQ, trace_id uzatish, va **sodda debug**.

Qaror:
O'z Redis-asosli navbat qatlami (`core/queue.py` + `manage.py run_worker --queue=X`). `Job` modeli PostgreSQL'da (holat, urinishlar, heartbeat), navbat Redis LIST'da. Har navbat uchun alohida systemd servis (`armenu-worker@model3d.service`).

Sabablar:

Sabab	Tushuntirish
Bulkhead tabiiy	Har navbat = alohida systemd unit = alohida `MemoryMax` / `CPUQuota`
Kuzatiluvchanlik	`Job` jadvali — SQL bilan so'rash mumkin: "fiskal navbatda nechta va nega turibdi"
Orphan tiklash	`heartbeat_at` ustuni + healer daemon (15 mexanizm) — Celery'da bu murakkab
Jim o'lim yo'q	Worker `heartbeat` yozadi; yozmasa healer 60s ichida qayta navbatga qo'yadi
Kod hajmi	`core/queue.py` ~250 qator. Celery — 50 000+ qator qora quti
Bog'liqlik	Celery + kombu + billiard + vine. Bizda faqat `redis-py` (allaqachon bor)
Tajriba	⚠ MebelCity'da Celery jimgina o'lgan — takrorlanmasin

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Celery	Standart, boy funksiyalar, beat	Jim o'lish tajribasi ⚠; debug qiyin; bulkhead uchun ko'p konfiguratsiya; ortiqcha funksiyalar (chord/chain) kerak emas
RQ (Redis Queue)	Sodda, Redis'da	Job holati faqat Redis'da (Redis yiqilsa yo'qoladi ⛔ — fiskal chek yo'qolmasligi kerak); heartbeat yo'q
Dramatiq	Zamonaviy, sodda	Yaxshi ✅, lekin baribir tashqi bog'liqlik; o'z qatlamimiz 250 qator va to'liq nazorat
Django-Q2	Django integratsiya	Kichik jamoa; ORM'ga og'ir yuk (poll har 4s)
threading.Thread	Eng oddiy	⛔ Web process qayta ishga tushsa vazifa yo'qoladi (MebelCity media_agent bug'i)
PostgreSQL LISTEN/NOTIFY	DB'da	Xabar yo'qoladi agar tinglovchi yo'q bo'lsa; navbat semantikasi yo'q

Amalda:

```
# apps/core/models.py
class Job(models.Model):
    class Status(models.TextChoices):
        QUEUED = 'queued'; RUNNING = 'running'
        DONE = 'done';     FAILED = 'failed';  DLQ = 'dlq'

    queue        = models.CharField(max_length=32, db_index=True)
    task         = models.CharField(max_length=64)
    payload      = models.JSONField(default=dict)
    status       = models.CharField(max_length=16, choices=Status.choices, default=Status.QUEUED)
    attempts     = models.PositiveSmallIntegerField(default=0)
    max_attempts = models.PositiveSmallIntegerField(default=3)
    heartbeat_at = models.DateTimeField(null=True, blank=True, db_index=True)  # ⚡ orphan
    trace_id     = models.CharField(max_length=32, blank=True)                 # ⚡ P10
    restaurant_id= models.IntegerField(null=True, db_index=True)
    error        = models.TextField(blank=True)
    created_at   = models.DateTimeField(auto_now_add=True, db_index=True)

    class Meta:
        indexes = [models.Index(fields=['status', 'queue', 'heartbeat_at'])]


# core/queue.py
def enqueue(queue: str, task: str, *, delay=0, **payload) -> int:
    job = Job.objects.create(
        queue=queue, task=task, payload=payload,
        trace_id=getattr(_local, 'trace_id', ''),
        restaurant_id=getattr(_local, 'restaurant_id', None),
        max_attempts=QUEUE_LIMITS[queue].get('max_attempts', 3),
    )
    if delay:
        redis.zadd(f'delayed:{queue}', {job.id: time.time() + delay})
    else:
        redis.rpush(f'queue:{queue}', job.id)
    metrics.inc(f'queue.{queue}.enqueued')
    return job.id


# apps/core/management/commands/run_worker.py
def handle(self, *args, queue, **opts):
    while not self._stop:
        job_id = redis.blpop(f'queue:{queue}', timeout=5)
        if not job_id:
            self._promote_delayed(queue)          # kechiktirilganlarni ko'chirish
            continue
        with self._heartbeat(job_id):             # har 10s heartbeat_at yangilanadi
            self._run(job_id)
```

Oqibatlari:
- ✅ Har navbat mustaqil masshtablanadi va cheklanadi (P9)
- ✅ `Job` jadvali — to'liq audit: kim, qachon, necha marta, qanday xato
- ✅ Jim o'lim imkonsiz: heartbeat yo'q → healer 60s ichida qayta navbatga
- ✅ `trace_id` web → worker → tashqi API bo'ylab uziluvsiz (P10)
- ⚠ Celery'ning tayyor funksiyalari (beat, chord, rate limit) qo'lda yoziladi — `beat` o'rniga systemd timer, `rate limit` o'rniga navbat worker soni
- ⚠ O'z kodimiz = o'z buglarimiz. Yumshatish: 250 qator, to'liq test qoplami
- 🔄 **Qaytarish narxi:** o'rta. `enqueue()` interfeysi Celery `.delay()` ga o'xshash — adapter yozish ~3 kun. Lekin `Job` jadvalidagi tarix migratsiya qilinishi kerak.

Qachon qayta ko'riladi:
- Murakkab workflow (chord, chain, canvas) kerak bo'lsa
- Navbat hajmi > 10 mln job/oy (unda Kafka/NATS ham ko'rib chiqiladi)

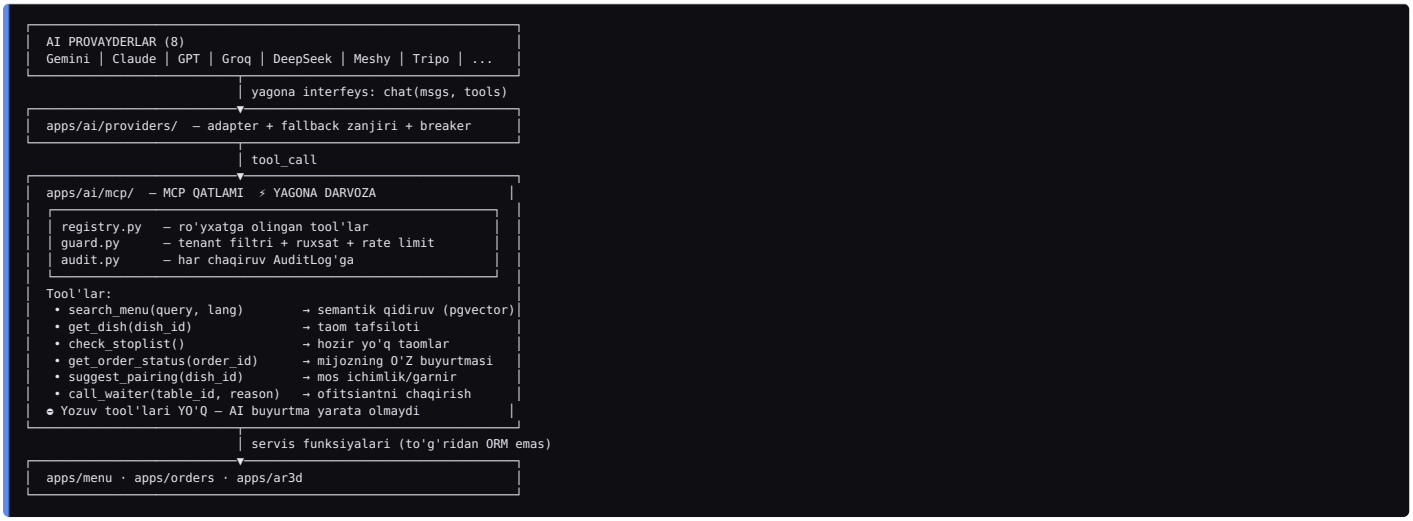
ADR-009 — MCP qatlami AI integratsiyasi uchun

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
Tizim 8 ta AI provayder bilan ishlaydi (Gemini, Claude, GPT, Groq, DeepSeek, Meshy, Tripo, ...) va AI 5 xil vazifa bajaradi: mijoz chatboti, menyu tarjimasi (uz/ru/en), taom tavsifini yozish, 3D model generatsiyasi, menyu-injiniring tahlili.

Muammo: har provayderning o'z SDK'si, o'z formati, o'z xato kodlari.
To'g'ridan-to'g'ri chaqirsak — 8 × 5 = 40 ta integratsiya nuqtasi.
Bundan tashqari, AI'ga **tizim ma'lumotiga kirish** kerak (menyu, buyurtma tarixi, stop-list) — lekin **hech qanday holatda** cross-tenant ma'lumot bermasligi kerak.

Qaror:
MCP (Model Context Protocol) qatlami — `apps/ai/mcp/`. AI provayderlar tizim ma'lumotiga faqat **ro'yxatga olingan MCP tool'lar** orqali kiradi.
Har tool: sxema (JSON Schema), tenant filtri, ruxsat tekshiruvi, rate limit, audit log.
Provayderlar `apps/ai/providers/` da adapter naqshi bilan (ADR-003'dagi kabi zanjir).



Sabablar:

Sabab	Tushuntirish
Yagona darvoza	AI ma'lumotga faqat shu yerdan kiradi → tenant tekshiruvi **bitta joyda**
Provayder almashtirish	MCP tool'lar standart; provayder o'zgarsa tool'lar o'zgarmaydi
Xavfsizlik	AI'ga ORM bermaymiz. `search_menu` tool'i majburiy `restaurant_id` filtri qo'yadi
Prompt injection himoyasi	Mijoz "barcha restoranlar menyusini ko'rsat" desa — tool baribir filtrlaydi
Audit	Har AI tool chaqiruvi log'da: kim, qaysi tool, qanday argument, natija hajmi
Standart	MCP — ochiq protokol; kelajakda tashqi AI agentlar ham shu darvozadan kiradi
MebelCity tajribasi	`mcp_api/` allaqachon shu naqsh bilan ishlaydi — tasdiqlangan

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
To'g'ridan-to'g'ri provayder SDK	Tez yozish	40 ta integratsiya nuqtasi; tenant tekshiruvi har joyda takrorlanadi; provayder almashtirish = qayta yozish
LangChain	Tayyor abstraksiyalar	Og'ir bog'liqlik; tez o'zgaradi (breaking changes); ortiqcha (bizga agent framework kerak emas)
Faqat prompt'ga ma'lumot solish (RAG'siz tool)	Sodda	Kontekst limiti; menyu 200 taom bo'lsa sig'maydi; real vaqt ma'lumoti (stop-list) eskiradi
AI'ga read-only DB user	Kuchli	⛔ Cross-tenant himoya yo'q; SQL generatsiya xavfli (prompt injection → `SELECT * FROM orders`)

Amalda:

```
# apps/ai/mcp/registry.py
@mcp_tool(
    name='search_menu',
    description='Restoran menyusidan taom qidirish (semantik)',
    schema={
        'type': 'object',
        'properties': {
            'query': {'type': 'string', 'maxLength': 200},
            'lang':  {'type': 'string', 'enum': ['uz', 'ru', 'en']},
            'limit': {'type': 'integer', 'minimum': 1, 'maximum': 10},
        },
        'required': ['query'],
    },
    rate_limit='10/5m',          # session bo'yicha
)
def search_menu(ctx: MCPContext, query: str, lang='uz', limit=5):
    # ⚡ ctx.restaurant_id — guard.py o'rnatadi, AI o'zgartira olmaydi
    emb = embed(query, lang)
    return (Dish.objects
            .filter(category__restaurant_id=ctx.restaurant_id,   # ⚡ majburiy
                    is_active=True)
            .exclude(id__in=stoplist_ids(ctx.restaurant_id))
            .order_by(CosineDistance('embedding', emb))[:limit]
            .values('id', 'name', 'price', 'description', 'image_url'))
```

Oqibatlari:
- ✅ AI provayderni almashtirish — 1 fayl (`providers/`), tool'lar tegilmaydi
- ✅ Cross-tenant sizish AI orqali **imkonsiz** (guard majburiy)
- ✅ Prompt injection zarari cheklangan — AI faqat ro'yxatdagi tool'larni chaqira oladi
- ✅ Har AI chaqiruv narxi hisoblanadi (`armenu_ai_cost_usd_total{provider,model}`)
- ⚠ Qo'shimcha abstraksiya qatlami — yangi tool qo'shish 20 qator kod
- ⚠ Tool chaqiruvi qo'shimcha kechikish (~10-30 ms) — AI javobiga nisbatan ahamiyatsiz
- ♻ **Qaytarish narxi:** past. Qatlam `apps/ai/` ichida to'liq izolyatsiyalangan.

Qachon qayta ko'riladi:
- MCP standarti jiddiy o'zgarsa (versiya migratsiyasi)
- AI'ga **yozuv** operatsiyasi kerak bo'lsa (masalan buyurtma yaratish) → yangi ADR yoziladi, tasdiqlash oqimi bilan

---

ADR-010 — systemd + nginx (Kubernetes emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
Deploy strategiyasi kerak: S0 (1 VPS) dan S5 (60+ node) gacha o'sish yo'li bilan. Jamoa — 1-2 dasturchi, alohida DevOps yo'q. Sanoat standarti — Kubernetes, lekin u o'z operatsion murakkabligini olib keladi (control plane, CNI, ingress, operator'lar, RBAC, Helm).

Qaror:
systemd unit'lar + nginx. Konteyner yo'q (S0-S2), keyin ixtiyoriy Docker. Deploy — `git pull` + `migrate` + `collectstatic` + `systemctl reload`. Ansible playbook bilan avtomatlashtiriladi. k8s — **S4'dan keyin ko'rib chiqiladi**.

Sabablar:

Sabab	Tushuntirish
Operatsion sodda	1-2 dasturchi. k8s to'liq vaqtli DevOps talab qiladi
Socket activation	`armenu.socket` portni ushlab turadi → `reload` da **zero-downtime**, port ochiq qoladi
Debug	`journalctl -u armenu -f` — darhol. k8s'da: pod → container → log driver
Resurs	k8s control plane 3 node × 2 CPU = boy resurs S0-S2 uchun
MebelCity tajribasi	systemd + socket activation ishlab turibdi, zero-downtime tasdiqlangan ✅
Narx	S0 $40/oy. Boshqariladigan k8s minimal ~$150/oy + node'lar

Amalda:

```
# deploy/systemd/armenu.socket  — ⚡ zero-downtime kaliti
[Socket]
ListenStream=127.0.0.1:8060
Backlog=2048
[Install]
WantedBy=sockets.target

# deploy/systemd/armenu.service
[Unit]
Requires=armenu.socket
After=armenu.socket postgresql.service redis.service

[Service]
Type=notify
User=armenu
WorkingDirectory=/opt/armenu
EnvironmentFile=/opt/armenu/.env
ExecStart=/opt/armenu/venv/bin/gunicorn config.wsgi:application \
    --workers 8 --threads 4 --worker-class gthread \
    --max-requests 2000 --max-requests-jitter 200 \
    --timeout 30 --graceful-timeout 20 --keep-alive 5
ExecReload=/bin/kill -HUP $MAINPID
Restart=always
RestartSec=5
MemoryMax=6G
[Install]
WantedBy=multi-user.target
```

```
DEPLOY OQIMI (zero-downtime):

  git pull
    │
    ▼
  pip install -r requirements.txt   (o'zgargan bo'lsa)
    │
    ▼
  python manage.py migrate          (⚡ orqaga mos migratsiya majburiy)
    │
    ▼
  python manage.py collectstatic --noinput
    │
    ▼
  systemctl reload armenu           ← socket port ushlab turadi, uzilish YO'Q
    │
    ▼
  systemctl restart armenu-ws       (WS consumer o'zgargan bo'lsa)
    │
    ▼
  smoke test: /healthz + /v1/menu/demo/
```

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Kubernetes	Auto-scale, self-heal, standart	Operatsion murakkablik 1-2 dasturchiga qotil; S0-S3 gacha foydasi yo'q; ✅ **S4'da qayta ko'riladi**
Docker Compose	O'rta yo'l	Bir node uchun; ko'p node'da orkestratsiya yo'q; systemd'dan ustunligi kam
Docker Swarm	Oddiy orkestratsiya	Amalda o'lgan loyiha (Docker Inc. qo'llab-quvvatlamaydi)
Nomad	Yengil k8s alternativi	Yana bir yangi texnologiya; ekotizim kichik
PaaS (Heroku/Render)	Deploy oson	O'zbekistonda region yo'q (latency); qimmat; DB nazorati yo'q; fiskal integratsiya uchun statik IP kerak

Oqibatlari:
- ✅ Deploy — 1 buyruq, ~40 soniya, uzilishsiz
- ✅ Log — `journalctl`, standart Linux vositalari
- ✅ Resurs chegarasi systemd'da (`MemoryMax`, `CPUQuota`) — bulkhead (P9) bilan mos
- ⚠ Auto-scaling qo'lda/skript bilan (k8s'da avtomatik) — S3 gacha yetarli
- ⚠ Ko'p node'da konfiguratsiya drift xavfi → **Ansible majburiy** (konfiguratsiya Git'da)
- ⚠ Rolling deploy qo'lda skript (node'larni birma-bir)
- ♻ **Qaytarish narxi:** o'rta. Ilova allaqachon 12-factor (stateless, env config, log stdout). Dockerfile + Helm chart yozish ~2 hafta. Kod **o'zgarmaydi**.

Qachon qayta ko'riladi:
- Node soni > 15 **yoki** kuniga 3+ deploy
- Multi-region kerak bo'lganda (yil 3)
- Alohida DevOps ishga olinganda

> ⛔ **Ogohlantirish (MebelCity tajribasidan):** systemd/nginx/socket konfiguratsiya fayllari — **infratuzilma**. Bir vaqtda ikki kishi tahrirlamasin. O'zgarish faqat Git orqali, Ansible bilan tarqatiladi.

ADR-011 — 3D kutubxona: nusxa emas, havola

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
3D model yaratish qimmat: AI generatsiya (Meshy/Tripo) ~$0,20-0,50 per model, professional skanerlash ~$5-15. Bir GLB+USDZ juftligi 3-12 MB.

Muhim kuzatuv: **taomlarning 60-70% takrorlanadi** — osh, somsa, lag'mon, manti, shashlik, choy, non... 15 000 restoran × 80 taom = 1,2 mln model.
Agar har restoran o'z nusxasini saqlasa: 1,2 mln × 8 MB = **9,6 TB** va 1,2 mln × $0,35 = **$420 000** generatsiya xarajati.

Qaror:
Markaziy 3D kutubxona (`DishLibrary`) — platforma darajasida umumiy modellar.
Restoran taomi kutubxona modeliga **havola qiladi** (FK), nusxa olmaydi.
Restoran xohlasa — o'z modelini yuklaydi (`DishModel3D`, restoranga tegishli).



Sabablar:

Sabab	Tushuntirish
Storage	9,6 TB → 0,9 TB. S3 narxi $230/oy → $22/oy
Generatsiya xarajati	$420 000 → $45 000 (bir marta yaratilgan model qayta ishlatiladi)
CDN kesh	Bir xil URL ko'p restoran uchun → Cloudflare edge hit ~94% (nusxada ~30%)
Yangi restoran onboarding	Menyu yuklaganda 70% taomga model **darhol** topiladi (semantik moslash) → "AR menyu 1 kunda"
Sifat	Markaziy model bir marta yaxshi qilinadi, hamma foydalanadi
Yangilash	Model yaxshilansa — barcha restoranda avtomatik yangilanadi

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Har restoranga nusxa	To'liq mustaqillik; restoran modelni o'zgartira oladi	10× storage; 9× xarajat; CDN kesh past; yangi restoran 80 ta model kutadi (kunlar)
Faqat kutubxona (o'z model yo'q)	Eng arzon	Restoran o'z taomini ko'rsata olmaydi — asosiy qiymat yo'qoladi ⛔
Copy-on-write	Havola + kerak bo'lsa nusxa	✅ Aslida shu tanlandi: `DishLibrary` (havola) + `DishModel3D` (o'z modeli)
Restoranlararo almashish (P2P)	Restoranlar bir-birining modelini ko'radi	⛔ Cross-tenant ma'lumot sizishi (ADR-007 buzilishi); raqobat masalasi

Amalda:

```
# apps/ar3d/models.py
class DishLibrary(models.Model):
    """Platforma kutubxonasi — restaurant_id YO'Q (umumiy)."""
    slug        = models.SlugField(unique=True)          # 'osh_toy'
    name_uz     = models.CharField(max_length=120)
    name_ru     = models.CharField(max_length=120)
    name_en     = models.CharField(max_length=120)
    tags        = ArrayField(models.CharField(max_length=32), default=list)
    embedding   = VectorField(dimensions=1536)           # semantik moslash uchun
    glb_url     = models.URLField()
    usdz_url    = models.URLField()
    poster_url  = models.URLField()
    quality     = models.CharField(max_length=8, choices=[('low','low'),('mid','mid'),('high','high')])
    usage_count = models.PositiveIntegerField(default=0)
    is_public   = models.BooleanField(default=True)


class DishModel3D(models.Model):
    """Restoranning O'Z modeli — kutubxona mos kelmasa."""
    restaurant = models.ForeignKey('tenants.Restaurant', on_delete=models.CASCADE)
    dish       = models.OneToOneField('menu.Dish', on_delete=models.CASCADE, related_name='own_model')
    glb_url    = models.URLField()
    usdz_url   = models.URLField()
    source     = models.CharField(max_length=16)   # 'ai' | 'scan' | 'upload'


class Dish(models.Model):
    # ...
    library_model = models.ForeignKey(DishLibrary, null=True, blank=True,
                                      on_delete=models.SET_NULL, related_name='+')

    @property
    def ar_model(self):
        """Ustuvorlik: o'z modeli → kutubxona → yo'q (rasm fallback)."""
        return getattr(self, 'own_model', None) or self.library_model
```

Oqibatlari:
- ✅ Yangi restoran: menyu yuklanadi → semantik moslash → 70% taomga AR **darhol**
- ✅ Storage va CDN xarajati 10× kam
- ✅ Kutubxona modeli yaxshilansa — hammaga foyda
- ⚠ "Bizning osh boshqacha ko'rinadi" e'tirozi → restoran o'z modelini yuklashi mumkin
- ⚠ Kutubxona modelini o'zgartirish — **340 restoranga ta'sir qiladi**. Shuning uchun o'zgarish versiyalanadi (`osh_toy_v2`), eski versiya saqlanadi
- ⚠ Semantik moslash xato qilishi mumkin ("manti" → "somsa") → restoran admini tasdiqlaydi (moslash **taklif** sifatida ko'rsatiladi, avtomatik qo'llanmaydi)
- 🔄 **Qaytarish narxi:** past. `Dish.ar_model` property — yagona kirish nuqtasi. Har restoranga nusxa qilish uchun ETL skripti ~2 kun (lekin xarajat 10× oshadi).

Qachon qayta ko'riladi:
- Restoranlar > 50% o'z modelini yuklasa (kutubxona foydasi kamayadi)
- Litsenziya masalasi chiqsa (kim yaratgan modelga kim egalik qiladi — hozir: platforma)

ADR-012 — REST API (GraphQL emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
API iste'molchilari: mijoz PWA, ofitsiant PWA, KDS ekrani, admin panel, Telegram MiniApp, kelajakda 3rd-party integratsiya. Ma'lumot shakllari oldindan ma'lum va barqaror (menyu, taom, buyurtma, to'lov).

Qaror:
REST (DRF), versiyalangan: `/v1/...`. GraphQL ishlatilmaydi.

Sabablar:

Sabab	Tushuntirish
HTTP kesh	`GET /v1/menu/<slug>/` → CDN va nginx keshlaydi. GraphQL POST — CDN keshlamaydi ⛔
Menyu = 60% trafik	Aynan shu endpoint keshlanishi kerak. Bu **hal qiluvchi sabab**
Rate limiting	Endpoint bo'yicha turli limit (ADR: menyu 120/daq, buyurtma 10/daq). GraphQL'da bitta endpoint → limit qo'yish murakkab
Kuzatuv	`armenu_http_request_duration_seconds{endpoint}` — tabiiy. GraphQL'da barcha so'rov `/graphql`
Xavfsizlik	GraphQL'da chuqurlik hujumi (nested query bomb), N+1, introspection sizishi — qo'shimcha himoya kerak
Sodda	DRF serializer + ViewSet + router — jamoa biladi
Over-fetching muammosi yo'q	Bizda javob shakllari kichik va aniq; `?fields=` parametri yetarli

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
GraphQL	Klient kerakligini so'raydi; bitta so'rov	⛔ CDN kesh yo'q (menyu 60% trafik!); rate limit murakkab; kuzatuv qiyin; chuqurlik hujumi
gRPC	Tez, tipli	Brauzerdan to'g'ridan ishlamaydi (grpc-web proxy kerak); debug qiyin
tRPC	Tip xavfsizligi	TypeScript'ga bog'liq; backend Python
JSON:API spetsifikatsiyasi	Standart	Ortiqcha verbose; foyda kam

Amalda:

```
# config/urls.py
urlpatterns = [
    path('v1/', include('apps.api_v1.urls')),
]

# apps/api_v1/urls.py — qat'iy versiyalash
router = DefaultRouter()
router.register('menu/dishes',  DishViewSet,      basename='dish')
router.register('orders',       OrderViewSet,     basename='order')
router.register('tables',       TableViewSet,     basename='table')

urlpatterns = [
    path('menu/<slug:slug>/',           MenuPayloadView.as_view()),   # ⚡ keshlanadi
    path('menu/<slug:slug>/stoplist/',  StoplistView.as_view()),      # ⚡ ETag, 15s
    path('ar/<int:dish_id>/',           ARModelView.as_view()),       # ⚡ keshlanadi
    *router.urls,
]
```

```
# Menyu API — nginx darajasida keshlanadi (GraphQL'da imkonsiz)
location ~ ^/v1/menu/[^/]+/$ {
    proxy_cache        armenu_menu;
    proxy_cache_key    "$scheme$host$request_uri$http_accept_language";
    proxy_cache_valid  200 30s;
    proxy_cache_use_stale error timeout updating http_500 http_502 http_503;
    add_header X-Cache-Status $upstream_cache_status;
    proxy_pass http://armenu_web;
}
```

Versiyalash siyosati:

Qoida	Tafsilot
URL'da versiya	`/v1/`, `/v2/` — sarlavhada emas (CDN kesh uchun)
Buzuvchi o'zgarish	Faqat yangi versiyada. `v1` kamida 12 oy qo'llab-quvvatlanadi
Qo'shuvchi o'zgarish	`v1` ichida mumkin (yangi maydon qo'shish — buzuvchi emas)
Deprecation	`Sunset` sarlavhasi + 6 oy ogohlantirish

Oqibatlari:
- ✅ Menyu API CDN va nginx'da keshlanadi → 12 000 RPS'dan DB'ga <1 200 RPS
- ✅ Har endpoint alohida kuzatiladi, alohida rate limit
- ✅ `curl` bilan debug — darhol
- ⚠ Klient bir nechta so'rov qilishi mumkin (menyu + stop-list + restoran) → `GET /v1/menu/<slug>/` **kompozit** javob qaytaradi (bitta so'rov, hammasi ichida)
- ⚠ Over-fetching — admin panelda ba'zi joyda ortiqcha maydon keladi (ahamiyatsiz)
- ♻ **Qaytarish narxi:** o'rta. GraphQL qatlamini REST ustiga qo'shish mumkin (`/graphql` — faqat admin panel uchun) REST'ni buzmasdan. ~2 hafta.

Qachon qayta ko'riladi:
- 3rd-party integratsiyalar ko'payib, har biri turli maydon so'rasa
- Admin panelda murakkab, chuqur bog'langan so'rovlar ko'paysa

---

ADR-013 — Analitik event'lar ClickHouse'ga (PostgreSQL'ga emas)

Holat: 🟢 Qabul qilindi, **S2'dan joriy etiladi** · 2026-09-04

Kontekst:
Tizim ko'p event yozadi: `menu_viewed`, `dish_viewed`, `ar_viewed`, `ar_duration`, `cart_add`, `cart_remove`, `order_created`, `order_completed`, `payment_*`, `review_*`. Bashorat: 1 mln bir vaqtda foydalanuvchida**~50 mln event/kun**.

Agar bular PostgreSQL'ga yozilsa:
- Yozish yuklamasi: 50 mln/kun ≈ 580 event/s doimiy — bu buyurtma yozish yuklamasidan **kattaroq**
- Jadval hajmi: 50 mln × 200 bayt × 365 kun = **3,6 TB/yil**
- VACUUM/autovacuum yuklamasi primary'ni bo'g'adi
- Analitik so'rovlar (`GROUP BY dish_id, date`) tranzaksion so'rovlar bilan raqobatlashadi

Qaror:
Analitik event'lar PostgreSQL'ga yozilmaydi. Ular:
- **S0-S1:** `analytics_event` jadvali, kun bo'yicha partition, 90 kun saqlanadi (hajm kichik — buni tanlash mumkin)
- **S2+: ClickHouse** ga yoziladi. PostgreSQL faqat**tranzaksion** ma'lumot (buyurtma, to'lov, menyu, foydalanuvchi) uchun qoladi.

Oqim: `enqueue('analytics', ...)` → `worker:analytics` → batch (1 000 ta / 5s) → ClickHouse.



Sabablar:

Sabab	Tushuntirish
Yozish tezligi	ClickHouse batch insert: ~1 mln qator/s bitta node'da. PostgreSQL: ~50 k/s
Siqilish	Ustunli saqlash + LZ4: 3,6 TB → ~250 GB (14×)
Analitik so'rov	`GROUP BY` 100 mln qator ustida: ClickHouse ~0,5s, PostgreSQL ~45s
Primary himoyasi	Event yozuvi tranzaksion DB'ga **umuman tegmaydi** (P9 bulkhead)
Retention	`TTL 730 DAY` — avtomatik o'chirish, VACUUM kerak emas
Narx	2 × ClickHouse node ($400/oy) vs PostgreSQL'ni 2× kattalashtirish ($800/oy)

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
PostgreSQL (partition bilan)	Bitta DB (ADR-002 bilan mos)	3,6 TB/yil; VACUUM primary'ni bo'g'adi; analitik so'rov 45s; ✅ **S0-S1 uchun yetarli**
TimescaleDB	PG kengaytmasi, bitta DB	Baribir PG resursini yeydi; siqilish ClickHouse'dan zaif; yozish tezligi past
S3 + Athena/DuckDB	Juda arzon	So'rov kechikishi 5-30s; real-time dashboard uchun yaramaydi
Elasticsearch	Qidiruv + analitika	Og'ir (JVM); yozish tezligi ClickHouse'dan past; qimmat
Event'larni umuman yozmaslik	Eng arzon	⛔ AR konversiya, menyu-injiniring — mahsulotning asosiy qiymati

Amalda:

```
# apps/analytics/services.py
def track(event_type: str, *, restaurant_id: int, **props):
    """
    Sinxron ish YO'Q — faqat navbatga qo'yiladi.
    Event yo'qolishi mumkin (at-most-once) — bu analitika uchun maqbul.
    """
    enqueue('analytics', 'ingest', event_type=event_type,
            restaurant_id=restaurant_id, ts=time.time(), props=props)


# apps/analytics/tasks.py — worker:analytics
BATCH_SIZE, BATCH_SECONDS = 1000, 5

def ingest_batch(jobs):
    rows = [(j.payload['restaurant_id'], j.payload['event_type'],
             datetime.fromtimestamp(j.payload['ts']),
             json.dumps(j.payload['props'])) for j in jobs]
    try:
        clickhouse.execute(
            'INSERT INTO events (restaurant_id, event_type, created_at, props) VALUES',
            rows)
    except ClickHouseError:
        move_to_dlq('analytics', rows)     # yo'qolmasin, keyin qayta
```

```
-- ClickHouse sxemasi
CREATE TABLE events (
    restaurant_id  UInt32,
    event_type     LowCardinality(String),
    created_at     DateTime,
    dish_id        Nullable(UInt32),
    session_id     String,
    props          String            -- JSON
) ENGINE = MergeTree
PARTITION BY toYYYYMM(created_at)
ORDER BY (restaurant_id, event_type, created_at)
TTL created_at + INTERVAL 730 DAY;
```

Oqibatlari:
- ✅ PostgreSQL primary event yuklamasidan **butunlay ozod**
- ✅ Menyu-injiniring hisoboti (100 mln qator) < 1 soniyada
- ✅ 2 yillik tarix saqlanadi, avtomatik tozalanadi
- ⚠ **Yangi texnologiya** — operatsion bilim, backup, monitoring kerak
- ⚠ ClickHouse'da JOIN zaif → analitik so'rovlarda `restaurant_id`, `dish_id` denormalizatsiya (taom nomi ham event ichida saqlanadi)
- ⚠ Event yo'qolishi mumkin (at-most-once) — moliyaviy hisobot uchun **ishlatilmaydi** (u PostgreSQL'dagi `Order` / `Payment` dan olinadi)

- ⚠ S0-S1'da ClickHouse ortiqcha → shuning uchun **S2'dan** joriy etiladi
- ♻ **Qaytarish narxi:** past. `track()` — yagona kirish nuqtasi. Backend almashtirish 1 fayl.

Qachon joriy etiladi / qayta ko'riladi:
- **Joriy etish sharti:** `analytics_event` jadvali > 100 mln qator**yoki** > 200 GB
- **Qayta ko'rish:** agar event hajmi kutilgandan 10× kam bo'lsa — PostgreSQL'da qolish

---

ADR-014 — `paytechuz` (har provayderga alohida SDK emas)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
O'zbekistonda 3 ta asosiy to'lov provayderi: **Payme**, **Click**, **Uzum Bank**.
Har biri o'z protokoli bilan:
- Payme — JSON-RPC 2.0, merchant API, `CheckPerformTransaction` / `CreateTransaction` / `PerformTransaction` / `CancelTransaction`
- Click — REST, `prepare` / `complete` ikki bosqichli, MD5 imzo
- Uzum — REST, o'z sxemasi

Har biriga alohida integratsiya = 3× kod, 3× test, 3× webhook handler, 3× xato mantiqi.

Qaror:
`paytechuz` **kutubxonasi** — O'zbekiston to'lov provayderlari uchun yagona Python abstraksiyasi. Uning ustiga o'z **adapter qatlami** (`apps/payments/gateways/`) — kutubxona interfeysini bizning domenimizga moslash va fallback zanjiri (P3).



Sabablar:

Sabab	Tushuntirish
Kod hajmi	3 integratsiya (~1 200 qator) → 1 adapter (~300 qator)
Protokol tafsilotlari	Payme JSON-RPC xato kodlari (-31001, -31050...) kutubxona ichida
Yangi provayder	4-provayder qo'shilsa (masalan Anor/Humo) — kutubxona qo'llasa, biz 0 qator yozamiz
Yangilanish	Provayder API o'zgarsa — kutubxona yangilanadi, biz emas
Django integratsiya	`paytechuz` Django uchun webhook view va model mixin beradi
Test	Bitta interfeys → mock qilish oson

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Har provayderga o'z SDK	To'liq nazorat, kutubxona bug'iga bog'liq emas	3× kod va test; har provayder yangilanishini kuzatish; xatolar takrorlanadi
Bitta provayder (faqat Payme)	Eng oddiy	⛔ Payme yiqilsa to'lov to'xtaydi (P3 buzilishi); mijozlar Click'ni afzal ko'radi
Agregator (Octobank/Atmos)	Bitta shartnoma	Qo'shimcha komissiya (0,3-0,5%); yana bir vositachi = yana bir yiqilish nuqtasi
Faqat naqd (to'lov integratsiyasisiz)	0 ish	⛔ Mahsulot qiymatining katta qismi — stolda to'lash

⚠ Risk va yumshatish:

Risk	Yumshatish
Kutubxona qo'llab-quvvatlanmay qolsa	Ochiq kod — fork qilish mumkin; adapter qatlami tufayli almashtirish lokal
Kutubxonada bug (pul masalasi!)	Har to'lov 4 qatlamda tekshiriladi (P12): idempotency → DB unique → webhook HMAC → summa moslik
Provayder API o'zgarishi kutubxonaga kech yetsa	Adapter'da override imkoniyati; monitoring alert

Amalda:

```
# apps/payments/gateways/base.py
class PaymentGateway(Protocol):
    def create(self, *, order_id: int, amount: Decimal, idem_key: str) -> PaymentIntent: ...
    def verify_webhook(self, request) -> WebhookEvent: ...
    def cancel(self, *, provider_tx_id: str, reason: str) -> None: ...


# apps/payments/services.py
PROVIDER_CHAIN = ['payme', 'click', 'uzum']

def create_payment(*, order, preferred=None):
    chain = ([preferred] if preferred else []) + [p for p in PROVIDER_CHAIN if p != preferred]
    for name in chain:
        gw = get_gateway(name)                      # paytechuz orqali
        breaker = BREAKERS[name]
        try:
            intent = breaker.call(
                gw.create,
                order_id=order.id,
                amount=order.total,                 # ⚡ ADR-006 — serverdan
                idem_key=f'order:{order.id}',       # ⚡ P4
            )
            Payment.objects.create(order=order, provider=name,
                                   amount=order.total, provider_tx_id=intent.tx_id)
            return intent
        except ServiceUnavailable:
            logger.warning('to\'lov: %s mavjud emas → keyingisi', name)
    raise AllProvidersDown('payment')               # → UI "naqd to'lash" taklif qiladi
```

Oqibatlari:
- ✅ 3 provayder bitta interfeys ostida; birortasi yiqilsa keyingisiga o'tadi
- ✅ To'lov kodi qisqa va testlanadigan
- ⚠ Tashqi kutubxonaga bog'liqlik — **pul masalasida**. Shuning uchun P12 (4 qatlam) majburiy
- ⚠ Kutubxona qo'llab-quvvatlamaydigan maxsus funksiya (masalan Payme fiskal integratsiyasi) uchun adapter'da to'g'ridan-to'g'ri chaqiruv yozilishi mumkin
- ♻ **Qaytarish narxi:** past-o'rta. `PaymentGateway` protokoli bor — har provayderga o'z adapterini yozish ~1 hafta per provayder.

Qachon qayta ko'riladi:
- Kutubxona 6 oydan ko'p yangilanmasa
- Provayderlarning kutubxona qamramaydigan funksiyalari kerak bo'lsa
- Xalqaro to'lov (Visa/Mastercard to'g'ridan) kerak bo'lsa → yangi ADR

ADR-015 — Faza-asosli rivojlanish (13 faza, 234 vazifa)

Holat: ✅ Qabul qilindi · 2026-09-04

Kontekst:
ARMENU — katta loyiha (~39 650 qator kod bashorati, 13 funksional soha).
Jamoa kichik. Katta loyihalarning klassik o'limi: hamma narsani parallel boshlash → hech narsa tugamaydi → 6 oydan keyin ishlaydigan mahsulot yo'q.

Bundan tashqari, `MEMORY.md` da yozilgan qat'iy qoida: **QONUN69 — 2 bosqich, ruxsatsiz kod yozilmaydi**; **TZ avval**; **faqat so'ralgan joy**.

Qaror:
13 faza, 234 vazifa — qat'iy ketma-ketlikda. Har faza:
- **Kirish sharti** (oldingi faza tugagan bo'lishi)
- **Chiqish sharti** (Definition of Done: testlar, hujjat, demo)
- **Mustaqil qiymat** — har faza oxirida ishlaydigan narsa bo'ladi

Faza ichida vazifalar parallel bo'lishi mumkin, fazalar orasida — yo'q.

```
FAZA YO'LI (har faza oxirida ishlaydigan mahsulot)

 F0  Asos          ── Django skelet, core/, CI, deploy       ── 12 vazifa
  │                    ✅ Chiqish: /healthz ishlaydi, CI yashil
  ▼
 F1  Tenant+Auth   ── Restaurant, Staff, middleware, mixin    ── 18 vazifa
  │                    ✅ Chiqish: cross-tenant test 403 beradi
  ▼
 F2  Menyu         ── Category, Dish, Modifier, stop-list     ── 22 vazifa
  │                    ✅ Chiqish: admin menyu kiritadi, API beradi
  ▼
 F3  Stol + QR     ── Hall, Table, QR token, guest session    ── 14 vazifa
  │                    ✅ Chiqish: QR skanerlanadi, menyu ochiladi
  ▼
 F4  Mijoz PWA     ── Menyu ko'rish, savat, i18n              ── 26 vazifa
  │                    ⚡ Chiqish: MVP DEMO — mijoz menyu ko'radi
  ▼
 F5  3D / AR       ── DishLibrary, model-viewer, pipeline     ── 24 vazifa
  │                    ⚡ Chiqish: AR ishlaydi — YADRO QIYMAT
  ▼
 F6  Buyurtma      ── Order, holat mashinasi, WS, KDS         ── 28 vazifa
  │                    ⚡ Chiqish: to'liq buyurtma sikli
  ▼
 F7  To'lov        ── paytechuz, 3 provayder, webhook         ── 16 vazifa
  │                    ✅ Chiqish: real to'lov (sinov rejimi)
  ▼
 F8  Fiskal        ── OFD, IKPU, chek                          ── 14 vazifa
  │                    ✅ Chiqish: qonuniy chek chiqadi
  ▼                   ⚡ SHU YERDA ISHLAB CHIQARISHGA CHIQISH MUMKIN
 F9  AI + MCP      ── chatbot, tarjima, tavsif                ── 20 vazifa
  ▼
 F10 Sodiqlik+Sharh ── bonus, keshbek, Google yo'naltirish    ── 16 vazifa
  ▼
 F11 Analitika     ── event, hisobot, menyu-injiniring        ── 14 vazifa
  ▼
 F12 Chidamlilik   ── healer, chaos test, monitoring          ── 10 vazifa
  ──────────────────────────────────────────────────────────
  JAMI: 234 vazifa
```

Sabablar:

Sabab	Tushuntirish
Bog'liqlik tabiiy	Buyurtma menyusiz bo'lmaydi; AR taomsiz bo'lmaydi. Tartib majburiy
Har faza — qiymat	F4'da demo ko'rsatish mumkin; F8'da real ishlatish mumkin
Risk erta	Eng noaniq qism (F5 — AR) o'rtada, kech emas
Kichik jamoa	Bir vaqtda bitta narsaga fokus
⚡ QONUN69	Har faza boshida TZ → savol → ruxsat → kod. Faza — tabiiy nazorat nuqtasi
O'lchash	"234 vazifadan 87 bajarildi" — aniq progress, "80% tayyor" emas

Muqobillar va nega rad etildi:

Muqobil	Ijobiy	Nega rad etildi
Parallel (hamma modul birga)	Nazariy tez	Kichik jamoada — hech narsa tugamaydi; integratsiya do'zaxi
Scrum sprintlari (2 hafta)	Standart	Sprint chegarasi texnik bog'liqlikka mos kelmaydi; ceremony yuki
Kanban (oqim)	Moslashuvchan	Prioritet uzluksiz o'zgaradi → arxitektura izchilligi buziladi
MVP birinchi, keyin qayta yozish	Eng tez demo	Qayta yozish deyarli hech qachon bo'lmaydi; texnik qarz qoladi

Faza chiqish sharti (Definition of Done) — har faza uchun bir xil:

#	Shart
1	Barcha vazifalar bajarilgan
2	Unit test qoplami ≥ 70% (kritik yo'llarda 100%)
3	Cross-tenant izolyatsiya testi yashil (`test_tenant_isolation.py`)
4	`tekshir_kod.sh` yashil (lint, N+1 detektor, `except: pass` qidiruvi)
5	API hujjati yangilangan (`04_DATA_MODEL_VA_API.md`)
6	Staging'da demo ko'rsatilgan va qabul qilingan
7	Yangi ADR yozilgan (agar arxitektura qarori qabul qilingan bo'lsa)

Oqibatlari:
- ✅ Har 2-3 haftada ko'rsatiladigan natija
- ✅ F8'dan keyin **real restoranda ishlatish mumkin** (v1 = F0-F8)
- ✅ Progress aniq o'lchanadi (234 vazifa, har biri ≤ 1 kun)
- ✅ QONUN69 tabiiy joylashadi: faza boshi = TZ + ruxsat nuqtasi
- ⚠ Kech fazalar (F9-F12) uzoq kutadi — AI va analitika 6+ oydan keyin
- ⚠ Faza chegarasi qat'iy → shoshilinch talab kelsa reja buziladi → yechim: "shoshilinch" vazifalar uchun faza ichida 10% zaxira vaqt
- ⚠ Katta reja — o'zgarishga qarshilik xavfi. Yumshatish: har faza oxirida qolgan fazalar **qayta ko'rib chiqiladi** (o'zgarishi mumkin)
- ♻ **Qaytarish narxi:** yo'q — bu jarayon qarori, kod emas.

Qachon qayta ko'riladi:
- Har faza oxirida (qolgan fazalar tartibi va hajmi)
- Jamoa 4+ dasturchiga o'ssa (parallel oqimlar mumkin bo'ladi)
- Bozor talabi keskin o'zgarsa (masalan yetkazib berish avvalroq kerak bo'lsa)

---

§2.1 ADR qisqacha jadvali

ADR	Qaror	Muqobil (rad etilgan)	Qaytarish narxi	Qayta ko'rish sharti
001	Modulli monolit	Mikroservis, serverless	🟡 O'rta (~2 hafta/modul)	ar3d >500/kun, jamoa >6
002	PG 16 + pgvector	Pinecone, Qdrant, ES	🟢 Past (~1 kun)	>20 mln vektor
003	Vanilla JS + PWA	React, Vue, Svelte	🟠 O'rta-yuqori (4-6 hafta)	>15 ekran, 3+ frontend dev
004	`<model-viewer>`	Three.js, 8thWall	🟢 Past (~1 hafta)	iOS WebXR ochilsa
005	WS faqat buyurtmadan keyin	Hammaga WS, polling	🟢 Past (1 qator)	Yangi real-time funksiya
006	Narx faqat serverda	Klient hisoblaydi	⛔ **Qaytarilmaydi**	⛔ Hech qachon
007	Middleware + Mixin tenant	DB/schema-per-tenant	🔴 Yuqori (~6 hafta)	Korporativ mijoz talabi
008	Redis navbat	Celery, RQ, Dramatiq	🟡 O'rta (~3 kun)	Murakkab workflow
009	MCP qatlam	To'g'ridan SDK, LangChain	🟢 Past	MCP standarti o'zgarsa
010	systemd + nginx	k8s, Compose, PaaS	🟡 O'rta (~2 hafta)	>15 node, 3+ deploy/kun
011	3D kutubxona (havola)	Har restoranga nusxa	🟢 Past (~2 kun)	>50% o'z modelini yuklasa
012	REST + DRF	GraphQL, gRPC	🟡 O'rta (~2 hafta)	Ko'p 3rd-party integratsiya
013	Event → ClickHouse	PG, TimescaleDB, S3	🟢 Past (1 fayl)	S2'da joriy; hajm 10× kam bo'lsa
014	paytechuz	Har provayderga SDK	🔴 Past-o'rta (~1 hafta/prov)	Kutubxona tashlab ketilsa
015	Faza-asosli (234 vazifa)	Parallel, Scrum, Kanban	🟢 Yo'q (jarayon)	Har faza oxirida

§3. TRADE-OFF TAHLILI

> Har arxitektura qarori — **almashuv**. Bu yerda: nima berildi, nima olindi,
> va qaytarish yo'li bormi.

3.1 Asosiy almashuvlar

#	✖ Qurbon qilindi	✔ Olindi	Narx (agar kerak bo'lsa)	ADR
1	Mustaqil deploy va masshtab (mikroservis)	Tranzaksiya yaxlitligi, 2× tez yetkazish, 1 VPS	2 hafta/modul ajratish	001
2	Ixtisoslashgan vektor DB tezligi (5ms→15ms)	Bitta DB, SQL+vektor bitta so'rovda, $0	1 kun eksport	002
3	Frontend framework qulayligi	45 KB vs 140 KB bundle, SEO, Node yo'q	4-6 hafta qayta yozish	003
4	AR effektlari ustidan to'liq nazorat	iOS+Android bitta kod, Google qo'llab-quvvatlaydi	4 hafta (Three.js+WebXR)	004
5	Mijozga jonli menyu yangilanishi	11× kam WS resurs, $1 360/oy tejash	1 qator (lekin 7× narx)	005
6	Klient tarafda tez narx ko'rsatish	Moliyaviy xavfsizlik, fiskal to'g'rilik	⛔ Qaytarilmaydi	006
7	Jismoniy tenant izolyatsiyasi	1 migratsiya, 1 backup, platforma analitikasi	6 hafta migratsiya	007
8	Celery ekotizimi (beat, chord)	Bulkhead, orphan tiklash, jim o'lim yo'q	3 kun adapter	008
9	AI'ga to'g'ridan ma'lumot kirishi (tez)	Cross-tenant himoya, audit, provayder almashtirish	—	009
10	Avtomatik auto-scaling (k8s)	Operatsion soddalik, $110/oy tejash	2 hafta Docker+Helm	010
11	Restoranning modelni erkin o'zgartirishi	10× storage, 9× xarajat, tez onboarding	2 kun ETL	011
12	Klient kerakligini so'rashi (GraphQL)	CDN kesh (60% trafik!), rate limit, kuzatuv	2 hafta qo'shimcha qatlam	012
13	Bitta ma'lumot bazasi soddaligi	PG primary event yukidan ozod, 14× siqilish	1 fayl	013
14	To'lov kodi ustidan to'liq nazorat	4× kam kod, provayder yangilanishi tashqarida	1 hafta/provayder	014
15	Moslashuvchan prioritetlash	Aniq progress, har faza — ishlaydigan mahsulot	—	015

3.2 Sifat atributlari o'rtasidagi almashuvlar



Ziddiyat	Qanday hal qilindi
Xavfsizlik ↔ Ishlash (ADR-006: narx serverda)	Xavfsizlik yutdi. +40 ms narx byudjetga sig'adi
Soddalik ↔ Masshtab (ADR-001: monolit)	Soddalik yutdi **hozircha**; chegaralar ajratish uchun tayyor
Nazorat ↔ Tezlik (ADR-014: paytechuz)	Tezlik yutdi; P12 (4 qatlam) risk yumshatadi
Izchillik ↔ Mavjudlik (ADR-013: event yo'qolishi mumkin)	Analitikada mavjudlik yutdi; moliyada — izchillik (PG)
UX ↔ Resurs (ADR-005: WS cheklovi)	Resurs yutdi; polling 15s UX'ga sezilarli ta'sir qilmaydi
Sifat ↔ Xarajat (ADR-011: umumiy 3D)	Xarajat yutdi; restoran o'z modelini yuklashi mumkin

3.3 Bilib qabul qilingan cheklovlar

Cheklov	Sabab	Ta'sir	Qachon hal bo'ladi
iOS'da WebXR yo'q	Apple cheklovi	AR Quick Look ishlaydi, "jonli" joylashtirish cheklangan	Apple ochganda
Gaussian Splatting yo'q	AR Quick Look qo'llamaydi	Fotorealizm biroz past	Standart chiqqanda
Offline buyurtma yo'q	Narx serverda (ADR-006) + fiskal	Tarmoq yo'q bo'lsa buyurtma yo'q	v2 — navbat + keyingi tasdiqlash
Menyu real-time emas (mijozga)	ADR-005	Stop-list 15s kechikadi	Ataylab, o'zgarmaydi
Analitik event yo'qolishi mumkin	ADR-013 (at-most-once)	Statistika ~0,1% noaniq	Moliyaviy hisobotda ishlatilmaydi
Auto-scaling qo'lda	ADR-010	Kutilmagan pikda 5-10 daq kechikish	S4 (k8s)
Restoranlar 3D modelni bo'lisha olmaydi	ADR-007 (izolyatsiya)	Faqat platforma kutubxonasi umumiy	Ataylab

§4. ANTI-PATTERN'LAR — bu loyihada QILINMAYDI

> ⛔ Quyidagilar **kod ko'rigida rad etiladi** va imkon qadar CI'da bloklanadi.
> Har biri: nima, nega yomon, o'rniga nima.

4.1 🔴 Kritik — deploy bloklanadi

⛔ AP-01 — Filtrsiz queryset (cross-tenant sizish)

```
# ❌ HECH QACHON
dish  = Dish.objects.get(pk=dish_id)
order = Order.objects.filter(status='cooking')
Table.objects.all()

# ✅ DOIM
dish  = Dish.objects.get(pk=dish_id, category__restaurant_id=request.restaurant_id)
order = Order.objects.filter(status='cooking', restaurant_id=request.restaurant_id)
# yoki ViewSet'da: TenantScopedMixin
```

Nega yomon: Ma'lumot sizishi — GDPR hodisasi, jarima, obro' yo'qotish.
Tekshiruv: `test_tenant_isolation.py` (har model uchun) + `tekshir_kod.sh` grep.

⛔ AP-02 — Klientdan kelgan narx/summa ishlatish

```
# ❌ HECH QACHON
total = request.data['total']
Order.objects.create(total=total)

# ✅ DOIM
result = calc_order_total(restaurant=..., items_in=..., promo_code=...)
Order.objects.create(total=result.total, price_breakdown=result.breakdown)
```

Nega yomon: To'g'ridan-to'g'ri moliyaviy zarar; fiskal chek noto'g'ri (soliq javobgarligi).
Tekshiruv: `test_price_calculation.py` — klient 1 so'm yuborsa server to'g'ri hisoblaydi.

⛔ AP-03 — Idempotency'siz yozuv operatsiyasi

```
# ❌ HECH QACHON
@api_view(['POST'])
def create_order(request): ...

# ✅ DOIM
@api_view(['POST'])
@idempotent('order.create')
def create_order(request): ...
```

Nega yomon: Dublikat buyurtma/to'lov. Mobil tarmoqda retry — kafolatlangan hodisa.
Tekshiruv: `test_idempotency.py` — bir xil key bilan 2 marta → 1 ta yozuv.

⛔ AP-04 — Web so'rov ichida sinxron tashqi API

```
# ❌ HECH QACHON
def create_order(request):
    order = Order.objects.create(...)
    requests.post(OFD_URL, json=...)          # 3 soniya! Thread bloklanadi
    send_telegram(order)                       # yana 1 soniya
    return Response(...)

# ✅ DOIM
def create_order(request):
    order = Order.objects.create(...)
    transaction.on_commit(lambda: (
        enqueue('fiscal', 'send_receipt', order_id=order.id),
        enqueue('notify', 'telegram_new_order', order_id=order.id),
    ))
    return Response(...)                       # darhol
```

Nega yomon: Gunicorn thread'lari band bo'ladi → kaskadli yiqilish (P8 buzilishi).
Tekshiruv: Kod ko'rigi + `tekshir_kod.sh` (view fayllarda `requests.` qidiruvi).

⛔ AP-05 — Jim yutilgan xato (`except: pass`)

```
# ❌ HECH QACHON
try:
    price = calc_price(items)
except Exception:
    price = 0                                # 0 so'mlik buyurtma!

try:
    send_fiscal(order)
except Exception:
    pass                                     # chek yo'qoldi, hech kim bilmaydi

# ✅ DOIM
price = calc_price(items)                    # xato bo'lsa 500 — buyurtma yaratilmaydi

try:
    send_fiscal(order)
except OFDError as e:
    logger.exception('fiskal xato order=%s', order.id)
    metrics.inc('fiscal.error')
    enqueue('fiscal', 'send_receipt', order_id=order.id, delay=300)   # qayta urinish
    raise                                    # yoki aniq fallback
```

Nega yomon: Xato oylardan keyin buxgalteriyada topiladi (P11 buzilishi).
Tekshiruv: `tekshir_kod.sh` — `except.*:\s*pass` va `except Exception:` grep.

4.2 🟠 Jiddiy — kod ko'rigida rad etiladi

⛔ AP-06 — N+1 so'rov

```
# ❌
for order in Order.objects.filter(restaurant_id=rid):
    print(order.table.number, order.guest.name)      # har iteratsiyada 2 so'rov

# ✅
for order in Order.objects.filter(restaurant_id=rid).select_related('table', 'guest'):
    print(order.table.number, order.guest.name)

# ✅ ko'p-ko'p uchun
Order.objects.filter(...).prefetch_related('items__dish')
```

Nega yomon: 100 buyurtma = 201 so'rov. 1 mln foydalanuvchida DB o'ladi.
Tekshiruv: `django-silk` staging'da; testda `assertNumQueries`.

⛔ AP-07 — Butun jadvalni xotiraga olish

```
# ❌
dishes = list(Dish.objects.filter(restaurant_id=rid))     # 15 000 restoranda ETL'da o'ldiradi
all_events = AnalyticsEvent.objects.all()

# ✅
for dish in Dish.objects.filter(restaurant_id=rid).iterator(chunk_size=500):
    ...
# yoki
Dish.objects.filter(...).values_list('id', 'price')       # kerakli ustunlar
```

⛔ AP-08 — Modullar orasida to'g'ridan-to'g'ri import

```
# ❌ orders moduli notifications'ni biladi — bog'liqlik sikli xavfi
# apps/orders/services.py
from apps.notifications.telegram import send_new_order
from apps.analytics.tracker import track_order

def create_order(...):
    ...
    send_new_order(order)
    track_order(order)

# ✅ event orqali (P5)
# apps/orders/services.py
from core.events import order_created
transaction.on_commit(lambda: order_created.send(sender=Order, order=order))

# apps/notifications/handlers.py
@receiver(order_created)
def on_order_created(sender, order, **kw):
    enqueue('notify', 'telegram_new_order', order_id=order.id)
```

Nega yomon: ADR-001 ning "kelajakda ajratamiz" va'dasi buziladi.
Tekshiruv: `test_no_cross_import.py` — ruxsat etilgan bog'liqlik grafi tekshiriladi.

⛔ AP-09 — `transaction.atomic()` ichida tashqi chaqiruv yoki WS

```
# ❌
with transaction.atomic():
    order = Order.objects.create(...)
    ws_send(f'rest_{rid}_kds', {...})          # tranzaksiya rollback bo'lsa — noto'g'ri xabar!
    requests.post(OFD_URL, ...)                 # tranzaksiya 3 soniya ochiq turadi — qulf!

# ✅
with transaction.atomic():
    order = Order.objects.create(...)
transaction.on_commit(lambda: ws_send(f'rest_{rid}_kds', {...}))
transaction.on_commit(lambda: enqueue('fiscal', 'send_receipt', order_id=order.id))
```

Nega yomon: Uzoq tranzaksiya = uzoq qulf = deadlock. Rollback bo'lsa — yolg'on xabar.

⛔ AP-10 — Kesh kalitida versiya/tenant yo'q

```
# ⛔
cache.set('menu', data, 60)                    # qaysi restoran? qaysi til?
cache.set(f'menu:{slug}', data, 60)            # o'zgarganda invalidatsiya qanday?

# ✅
key = f'menu:{restaurant_id}:{lang}:v{menu_version(restaurant_id)}'
cache.set(key, data, 300)
```

Nega yomon: Cross-tenant kesh sizishi (⛔ kritik!) yoki eskirgan menyu.

⛔ AP-11 — Timeout'siz tashqi chaqiruv

```
# ⛔
requests.get(url)
requests.post(url, json=data)

# ✅
requests.post(url, json=data, timeout=(3, 10))     # (connect, read)
# + circuit breaker (P8)
BREAKERS['ofd_uzkassa'].call(_send, order, fallback=lambda: enqueue_retry(...))
```

⛔ AP-12 — Migratsiyada `NOT NULL` DEFAULT'siz

```
# ⛔ Eski process (hali eski kod bilan ishlayotgan) INSERT qila olmaydi → 500
migrations.AddField('order', 'source', models.CharField(max_length=16))

# ✅ Ikki qadamli
# 1-migratsiya: null=True yoki default bilan
migrations.AddField('order', 'source',
                    models.CharField(max_length=16, default='qr', null=True))
# 2-migratsiya (keyingi deploy'da): NOT NULL qilish
```

Nega yomon: Rolling deploy paytida eski va yangi kod bir vaqtda ishlaydi.
⚠ MebelCity'da aynan shu bug bo'lgan (`NOT NULL` ustun eski WS process'ni buzgan).

4.3 🟠 O'rta — konvensiya buzilishi

⛔ AP-13 — Process xotirasida holat

```
# ⛔ 2-instansda yo'q (P1 buzilishi)
_active_sessions = {}
_rate_limits = defaultdict(int)

# ✅
cache.set(f'session:{sid}', data, 3600)
cache.incr(f'rl:{ip}:{minute}')
```

⛔ AP-14 — Signal handler ichida og'ir ish

```
# ⛔ Signal SINXRON — web so'rov bloklanadi
@receiver(order_created)
def on_order(sender, order, **kw):
    generate_3d_model(order)                   # 5 daqiqa!
    requests.post(TELEGRAM_URL, ...)           # 1 soniya

# ✅ Faqat navbatga
@receiver(order_created)
def on_order(sender, order, **kw):
    enqueue('notify', 'telegram_new_order', order_id=order.id)
```

⛔ AP-15 — Butun son bo'lishi kerak joyda `float`

```
# ⛔ float — pul va miqdor uchun
total = 45000.0 * 1.1                          # 49500.000000001
qty = 2.0

# ✅ Decimal (pul), int (miqdor)
from decimal import Decimal
total = (Decimal('45000') * Decimal('1.1')).quantize(Decimal('1'))   # so'm — butun
qty = int(math.ceil(raw_qty))
```

Nega yomon: Buxgalteriya farqi; fiskal chek summasi mos kelmaydi.
⚠ MebelCity qoidasi: raqamlar (quantity, stock) butun son — `Math.ceil()`.

⛔ AP-16 — Statik faylni cache-bust'siz yangilash

```
<!-- ❌ Brauzer eski JS'ni keshdan oladi -->
<script src="/static/js/guest/menu.js"></script>

<!-- ✅ ManifestStaticFilesStorage (hash) yoki qo'lda versiya -->
<script src="{% static 'js/guest/menu.js' %}"></script>   <!-- menu.a3f9c2.js -->
```

⚠ MebelCity qoidasi: static o'zgarsa `collectstatic` + cache-bust MAJBURIY.

⛔ AP-17 — Tenant'siz WebSocket guruh

```
# ❌ Barcha restoran bitta guruhda — cross-tenant sizish!
await self.channel_layer.group_add('kds', self.channel_name)

# ✅ Har restoran o'z guruhida
await self.channel_layer.group_add(f'rest_{self.restaurant_id}_kds', self.channel_name)
```

⚠ MebelCity qoidasi: WS guruhlari tenant-scoped bo'lishi SHART.

⛔ AP-18 — `display: none` bilan "o'chirish"

```
// ❌ Ma'lumot baribir yuborilgan — DevTools'da ko'rinadi
document.querySelector('#cost-price').style.display = 'none';

// ✅ Serverda bermaslik (serializer'da maydon yo'q)
// yoki ruxsat bo'lmasa API javobiga qo'shilmaydi
```

Nega yomon: Tannarx, boshqa restoran ma'lumoti — HTML/JSON ichida qoladi.

4.4 Anti-pattern jadvali (tez ma'lumotnoma)

#	Anti-pattern	Daraja	Buzilgan printsip	Tekshiruv
AP-01	Filtrsiz queryset	🔴	P12, ADR-007	`test_tenant_isolation.py`
AP-02	Klient narxi	🔴	ADR-006	`test_price_calculation.py`
AP-03	Idempotency yo'q	🔴	P4	`test_idempotency.py`
AP-04	Sinxron tashqi API	🔴	P8, P9	`tekshir_kod.sh`
AP-05	`except: pass`	🔴	P11	`tekshir_kod.sh`
AP-06	N+1 so'rov	🟠	P6	`assertNumQueries`
AP-07	Butun jadval xotiraga	🟠	P6	Kod ko'rigi
AP-08	Modullararo import	🟠	P5, ADR-001	`test_no_cross_import.py`
AP-09	atomic ichida tashqi	🟠	P5	Kod ko'rigi
AP-10	Kesh kaliti noto'g'ri	🟠	P7, P12	Kod ko'rigi
AP-11	Timeout yo'q	🟠	P8	`tekshir_kod.sh`
AP-12	NOT NULL DEFAULT'siz	🟠	—	Migratsiya ko'rigi
AP-13	Process xotirasida holat	🟡	P1	Kod ko'rigi
AP-14	Signal'da og'ir ish	🟡	P5	Kod ko'rigi
AP-15	`float` pul uchun	🟡	—	Lint qoidasi
AP-16	Cache-bust yo'q	🟡	—	Deploy skripti
AP-17	Tenant'siz WS guruh	🔴	P12	`test_ws_isolation.py`
AP-18	`display:none` bilan yashirish	🟠	P12	Kod ko'rigi

---

§5. PRINTSIP ↔ ADR ↔ TEST MATRITSASI

> Har printsip qaysi ADR bilan amalga oshiriladi va qaysi test uni himoya qiladi.

Printsip	Amalga oshiruvchi ADR	Himoya qiluvchi test	Anti-pattern
P1 Stateless	001, 010	`test_no_local_state.py`	AP-13
P2 Shared-nothing	001, 010	Deploy smoke test	—
P3 Graceful degradation	004, 009, 014	`test_fallback_chain.py`	—
P4 Idempotency	008, 014	`test_idempotency.py`	AP-03
P5 Event-driven	001, 008	`test_no_cross_import.py`	AP-08, AP-09, AP-14
P6 CQRS-lite	002, 013	Yuklama testi	AP-06, AP-07
P7 Cache-first	012	`test_cache_hit_ratio.py`	AP-10
P8 Circuit breaker	009, 014	`test_breaker_opens.py`	AP-04, AP-11
P9 Bulkhead	008, 013	`test_queue_isolation.py`	AP-04
P10 Observability	008, 010	Metrik mavjudligi testi	—
P11 Fail-fast	006, 007	`test_fail_fast.py`	AP-05
P12 Defense-in-depth	006, 007, 009	`test_tenant_isolation.py`	AP-01, AP-02, AP-17, AP-18

5.1 CI darvozasi (deploy bloklaydigan testlar)

```
# .github/workflows/ci.yml — barcha yashil bo'lmasa deploy YO'Q
pytest tests/architecture/ -v          # ⚡ arxitektura invariantlari
  ├── test_tenant_isolation.py         # 🔴 har model uchun cross-tenant
  ├── test_price_calculation.py        # 🔴 klient narxi e'tiborga olinmaydi
  ├── test_idempotency.py              # 🔴 dublikat yaratilmaydi
  ├── test_no_cross_import.py          # 🔴 modul bog'liqlik grafi
  ├── test_ws_isolation.py             # 🔴 WS guruh tenant-scoped
  └── test_no_local_state.py           # 🟡 global mutable holat yo'q

./tekshir_kod.sh                        # ⚡ statik tahlil
  ├── grep 'except.*:\s*pass'          # AP-05
  ├── grep 'requests\.(get|post)' apps/*/views.py   # AP-04
  ├── grep 'requests\..*(?!timeout)'   # AP-11
  └── grep 'objects\.all\(\)'          # AP-01 (oq ro'yxat bilan)
```

QISM 2

Komponentlar

Versiya: 1.0 ·**Sana:** 2026-09-04
Kim uchun: dasturchi · DevOps · tizim arxitektori · yangi jamoa a'zosi (onboarding)

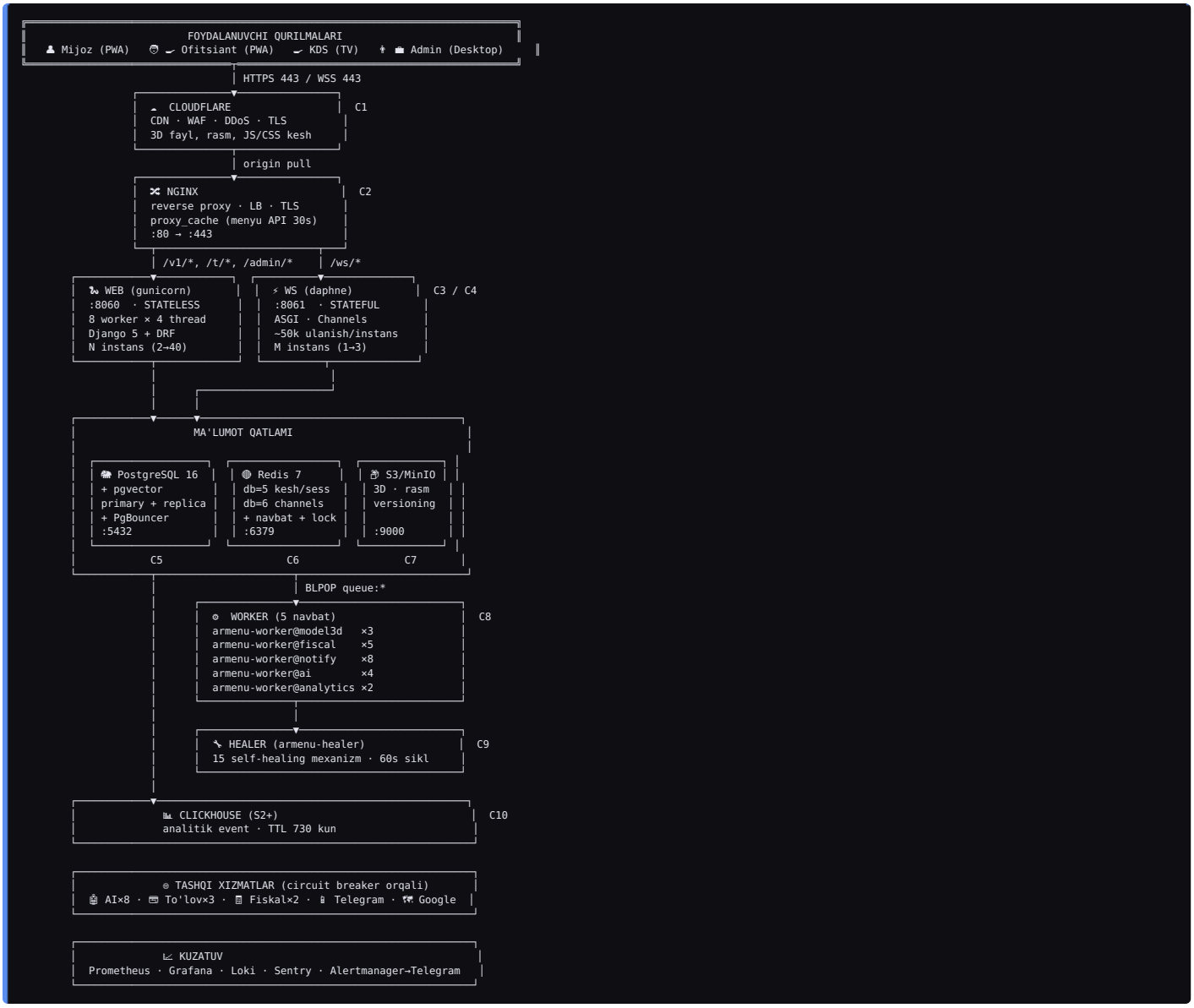
> Bu hujjat **"nimadan tuzilgan"** degan savolga javob beradi.
> §1-2 — konteynerlar (C4 L2: process, servis, saqlagich).
> §3-7 — komponentlar (C4 L3: Django modullar, core qatlami, frontend, worker).
> Qarorlarning sabablari: `01_PRINTSIPLAR_VA_QARORLAR.md`.

MUNDARIJA

Bo'lim	Mazmuni
§1	Konteyner diagrammasi (C4 L2)
§2	Har konteyner kartochkasi (8 ta)
§3	Django modullar — 13 app (C4 L3)
§4	Modullar bog'liqlik grafi (siklsiz)
§5	`core/` qatlami — har fayl
§6	Frontend komponentlari (4 ta ilova)
§7	Worker turlari (5 navbat)
§8	Komponent ↔ ADR ↔ Faza matritsasi

---

§1. KONTEYNER DIAGRAMMASI (C4 — Level 2)



1.1 Konteynerlar ro'yxati

ID	Konteyner	Texnologiya	Port	Holat	ADR
C1	Cloudflare	CDN + WAF	443	🟢 tashqi	—
C2	nginx	nginx 1.24	80/443	🟢 stateless	010
C3	web	gunicorn + Django 5	8060	🟢 stateless	001, 010
C4	ws	daphne + Channels	8061	🟡 stateful (ulanish)	005
C5	db	PostgreSQL 16 + pgvector	5432	🔵 stateful	002
C6	redis	Redis 7	6379	🟡 stateful (qayta quriladi)	008
C7	s3	MinIO / S3	9000	🔵 stateful	011
C8	worker	Python (run_worker)	—	🟢 stateless	008
C9	healer	Python (run_healer)	—	🟢 stateless	—
C10	clickhouse	ClickHouse (S2+)	8123	🔵 stateful	013

§2. KONTEYNER KARTOCHKALARI

C1 — ☁ CLOUDFLARE (CDN + WAF)

Maydon	Qiymat
Vazifasi	Statik va 3D fayllarni edge'dan berish · DDoS himoya · WAF · TLS termination · bot filtri
Texnologiya	Cloudflare Pro/Business plan
Portlar	443 (kirish), origin'ga 443
Resurs	— (boshqariladigan xizmat)
Masshtablash	Avtomatik, cheksiz (Cloudflare tomonidan)
Sog'liq tekshiruvi	Cloudflare o'z health-check'i; bizning `/healthz` origin uchun
Narx	~$20/oy (S0-S2) → ~$200/oy (S5)

Kesh siyosati:

Yo'l	TTL (edge)	TTL (brauzer)	Invalidatsiya
`/static/*` (hash bilan)	1 yil	1 yil	Fayl nomida hash
`/media/models/*.glb`	30 kun	7 kun	URL o'zgaradi (versiya)
`/media/models/*.usdz`	30 kun	7 kun	URL o'zgaradi
`/media/dishes/*.webp`	30 kun	7 kun	URL o'zgaradi
`/v1/menu/<slug>/`	60s	0	`Cache-Tag: menu-<rid>` purge
`/v1/orders/*`	⛔ NO CACHE	⛔	—
`/ws/*`	⛔ bypass	⛔	—

⛔ Yiqilsa nima bo'ladi:
🟡 **O'rta ta'sir.** Cloudflare "Always Online" bo'lmasa — trafik to'g'ridan origin'ga (nginx) tushadi. 3D fayllar S3'dan beriladi (sekinroq, ~3× kechikish, bandwidth xarajati oshadi). Menyu API nginx `proxy_cache` dan beriladi. DDoS himoyasi yo'qoladi — nginx rate limit qoladi.
Tiklanish: Cloudflare o'zi; DNS'ni "grey cloud"ga o'tkazish — 5 daqiqa (qo'lda).

C2 — 🔀 NGINX (reverse proxy + load balancer)

Maydon	Qiymat
Vazifasi	TLS termination · yuklamani taqsimlash · menyu API keshi · rate limit · statik fayl · WS upgrade
Texnologiya	nginx 1.24, `proxy_cache`, `limit_req_zone`
Portlar	80 (→443 redirect), 443
Resurs	1 CPU / 512 MB (S0) → 4 CPU / 8 GB × 2 (S3)
Masshtablash	Gorizontal: 2+ instans + DNS round-robin yoki tashqi LB
Sog'liq tekshiruvi	`/healthz` upstream'da har 3s; `max_fails=2 fail_timeout=9s`

Konfiguratsiya (asosiy qismlar):

```
upstream armenu_web {
    least_conn;                          # ⚡ eng kam ulangan node'ga
    server 127.0.0.1:8060 max_fails=2 fail_timeout=9s;
    server 10.0.0.5:8060  max_fails=2 fail_timeout=9s;
    keepalive 64;
}

upstream armenu_ws {
    ip_hash;                             # ⚡ WS uchun sticky (ulanish saqlanadi)
    server 127.0.0.1:8061 max_fails=2 fail_timeout=9s;
}

proxy_cache_path /var/cache/nginx/menu levels=1:2
                 keys_zone=armenu_menu:50m max_size=2g inactive=10m;

limit_req_zone $binary_remote_addr zone=qr:10m    rate=60r/m;
limit_req_zone $binary_remote_addr zone=menu:10m  rate=120r/m;
limit_req_zone $http_x_session_id  zone=order:10m rate=10r/m;

server {
    listen 443 ssl http2;
    server_name ~^(?<sub>[^.]+)\.armenu\.uz$;

    # Menyu API — keshlanadi (ADR-012 ning asosiy foydasi)
    location ~ ^/v1/menu/[^/]+/$ {
        limit_req          zone=menu burst=20 nodelay;
        proxy_cache        armenu_menu;
        proxy_cache_key    "$scheme$host$request_uri$http_accept_language";
        proxy_cache_valid  200 30s;
        proxy_cache_use_stale error timeout updating http_500 http_502 http_503;
        proxy_cache_lock   on;                  # ⚡ cache stampede himoyasi
        add_header X-Cache-Status $upstream_cache_status;
        proxy_pass http://armenu_web;
    }

    # Buyurtma — hech qachon keshlanmaydi
    location /v1/orders/ {
        limit_req zone=order burst=5 nodelay;
        proxy_cache off;
        proxy_pass http://armenu_web;
    }

    # WebSocket
    location /ws/ {
        proxy_pass http://armenu_ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade    $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout  3600s;              # uzoq ulanish
        proxy_send_timeout  3600s;
    }

    location / {
        proxy_pass http://armenu_web;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

⛔ Yiqilsa nima bo'ladi:
🟡 **To'liq to'xtash** (bitta instans bo'lsa) — hech narsa ishlamaydi.
Yumshatish: S1'dan 2 ta nginx + DNS failover; systemd `Restart=always` (5s).
Cloudflare "Always Online" statik sahifani ko'rsatadi.
Tiklanish: avtomatik 5s (systemd); konfiguratsiya xatosi bo'lsa — qo'lda `nginx -t` deploy'da majburiy).

C3 — 🐍 WEB (gunicorn + Django)

Maydon	Qiymat
Vazifasi	HTTP API (REST) · sahifa render · admin panel · webhook qabul qilish · biznes mantiq
Texnologiya	Python 3.12, Django 5, DRF, gunicorn (`gthread`)
Portlar	8060 (systemd socket activation)
Resurs	4 CPU / 8 GB per instans; 8 worker × 4 thread = 32 parallel
Sig'im	~300 RPS / instans (p95 250 ms da)
Masshtablash	⚡ **Gorizontal** — stateless (P1). S0: 1 → S5: 40 instans
Sog'liq tekshiruvi	`GET /healthz` (DB ping + Redis ping + 200 OK, < 50 ms)

Sog'liq endpoint'i:

```
# core/health.py
def healthz(request):
    """LB uchun. Yengil bo'lishi shart — har 3s chaqiriladi."""
    checks = {}
    try:
        connections['default'].cursor().execute('SELECT 1')
        checks['db'] = 'ok'
    except Exception as e:
        checks['db'] = f'fail: {e.__class__.__name__}'
    try:
        cache.set('_hz', '1', 5)
        checks['redis'] = 'ok'
    except Exception as e:
        checks['redis'] = f'fail: {e.__class__.__name__}'

    ok = all(v == 'ok' for v in checks.values())
    return JsonResponse({'status': 'ok' if ok else 'degraded', **checks},
                        status=200 if ok else 503)


def readyz(request):
    """Deploy uchun — migratsiya qo'llanganmi, kesh isitilganmi."""
    pending = MigrationExecutor(connections['default']).migration_plan(
        MigrationExecutor(connections['default']).loader.graph.leaf_nodes())
    return JsonResponse({'ready': not pending, 'pending_migrations': len(pending)},
                        status=200 if not pending else 503)
```

Gunicorn sozlamalari va sabablari:

Sozlama	Qiymat	Sabab
`--workers`	8	`2 × CPU` (4 CPU)
`--threads`	4	I/O kutish (DB, Redis) uchun
`--worker-class`	`gthread`	`sync` — thread yo'q; `gevent` — Django ORM bilan xavfli
`--timeout`	30	30s dan uzoq ish → worker'ga (P9)
`--graceful-timeout`	20	Reload'da joriy so'rovlar tugasin
`--max-requests`	2000	Xotira sizishiga qarshi (worker qayta tug'iladi)
`--max-requests-jitter`	200	Barcha worker bir vaqtda restart bo'lmasin
`--keep-alive`	5	nginx bilan keepalive

⛔ Yiqilsa nima bo'ladi:
- **1 instans:** 🟧 Ta'sir yo'q — nginx `max_fails` bilan chiqarib tashlaydi, boshqasi oladi.
systemd 5s ichida qayta ko'taradi. Socket activation tufayli port ochiq qoladi.
- **Barcha instans:** 🔴 To'liq to'xtash. Menyu — Cloudflare keshidan 60s davomida ko'rinadi.
Buyurtma, to'lov — yo'q.
Tiklanish: systemd `Restart=always`; healer C9 tekshiradi; alert P1 (Telegram + qo'ng'iroq).

C4 — ⚡ WS (daphne + Django Channels)

Maydon	Qiymat
Vazifasi	Real-time xabar: KDS buyurtma, ofitsiant chaqiruvi, admin dashboard, mijoz buyurtma holati
Texnologiya	Daphne (ASGI), Django Channels 4, `channels_redis` layer
Portlar	8061
Resurs	4 CPU / 8 GB per instans
Sig'im	~50 000 bir vaqtda ulanish / instans (~20-40 KB RAM per ulanish)
Masshtablash	Gorizontal + ⚡ **ADR-005** (mijozga WS faqat buyurtmadan keyin)
Sog'liq tekshiruvi	`GET /ws-health` (HTTP) + `armenu_ws_connections` metrikasi

Ulanish byudjeti (S5 — 1 mln bir vaqtda foydalanuvchi):

Guruh	Ulanish soni	Hisob
Mijoz (buyurtma bergan)	~50 000	1 mln × 5%
Ofitsiant	~25 000	5 000 rest × 5
KDS ekran	~10 000	5 000 rest × 2
Admin	~5 000	5 000 rest × 1
JAMI	**~90 000**	**→ 2-3 instans** ✅

WS guruhlari (⚡ hammasi tenant-scoped):

```
# apps/orders/consumers.py
class KDSConsumer(AsyncJsonWebsocketConsumer):
    async def connect(self):
        rid = await self.authenticate_staff()          # PIN sessiya
        if rid is None:
            return await self.close(code=4401)
        self.rid = rid
        self.group = f'rest_{rid}_kds'                 # ⚡ HECH QACHON 'kds' emas (AP-17)
        await self.channel_layer.group_add(self.group, self.channel_name)
        await self.accept()
        metrics.inc('ws.connect', group='kds')

    async def order_new(self, event):
        await self.send_json({'type': 'order.new', 'order': event['order']})
```

Guruh nomi	Kim	Nima keladi
`rest_<rid>_kds`	Oshxona ekrani	Yangi buyurtma, bekor qilish, prioritet
`rest_<rid>_waiters`	Ofitsiantlar	Chaqiruv, buyurtma tayyor, stol holati
`rest_<rid>_admin`	Restoran admini	Barcha hodisa, alert, statistika
`guest_<session_id>`	Bitta mijoz	⚡ Faqat **o'z** buyurtmasi holati (ADR-005)

Channel layer sozlamalari:

```
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
            'hosts': [('127.0.0.1', 6379, 6)],       # ⚡ db=6 (kesh db=5 dan alohida)
            'capacity': 1500,                         # guruh bo'yicha navbat
            'expiry': 20,                             # eskirgan xabar tashlanadi
            'group_expiry': 86400,
        },
    },
}
```

⛔ Yiqilsa nima bo'ladi:
🟡 **O'rta ta'sir, biznes to'xtamaydi.**
- ✅ Ishlaydi: menyu, AR, savat, **buyurtma yaratish**, to'lov, admin panel
- ❌ Yo'q: real-time push
- 🔄 **Avtomatik fallback (P3):** klient 3 marta qayta ulanishga urinadi (1s, 3s, 9s), keyin **SSE**ga, u ham bo'lmasa **polling 10s**ga o'tadi
- KDS: `GET /v1/orders/?status=pending&since=<ts>` har 10s

Tiklanish: systemd 5s; ulanishlar klient tomonidan qayta tiklanadi (exponential backoff).

C5 — 🐘 POSTGRESQL 16 + pgvector

Maydon	Qiymat
Vazifasi	Yagona haqiqat manbai: restoran, menyu, buyurtma, to'lov, xodim, 3D metama'lumot, vektor
Texnologiya	PostgreSQL 16, `pgvector`, `pg_trgm`, `btree_gin`; PgBouncer (transaction mode)
Portlar	5432 (PgBouncer 6432)
Resurs	S0: 2 CPU / 4 GB → S5: primary 16 CPU / 64 GB NVMe + 3 replica
Sig'im	Primary ~3 000 write RPS (16 CPU/64 GB)
Masshtablash	Vertikal → read replica → partition → sharding (`restaurant_id` hash)
Sog'liq tekshiruvi	`SELECT 1` (3s) · replication lag < 5s · connection pool < 85%

Topologiya (S3+):



Partition strategiyasi:

Jadval	Partition	Saqlash	Sabab
`orders_order`	RANGE, oy bo'yicha	24 oy issiq, keyin S3 arxiv	Eng katta yozuv jadvali
`orders_orderitem`	RANGE, oy (parent bilan)	24 oy	Order bilan birga
`payments_payment`	RANGE, oy	60 oy (moliyaviy)	Audit talabi
`analytics_event`	RANGE, kun	90 kun (S2'dan ClickHouse)	ADR-013
`core_job`	RANGE, hafta	4 hafta	Navbat tarixi

⛔ Yiqilsa nima bo'ladi:
- **Primary yiqiladi:** 🔴 **Yozish yo'q** (buyurtma, to'lov). ✅**O'qish ishlaydi** (replica'dan: menyu, AR, taom). Patroni 30-60s da replica'ni primary qiladi.
Mijoz bu vaqtda menyu ko'raveradi, buyurtma tugmasi "Bir daqiqadan keyin qayta urinib ko'ring".
- **Replica yiqiladi:** 🟩 DB router primary'ga o'tadi (yuklama ortadi, lekin ishlaydi).
- **To'liq yiqilish:** 🔴 Faqat Cloudflare keshidagi statik menyu.
Tiklanish: WAL + kunlik backup'dan (RTO 5 daqiqa buyurtma uchun, RPO 0).

C6 — 🔴 REDIS 7

Maydon	Qiymat
Vazifasi	Kesh (menyu, taom) · sessiya · WS channel layer · navbat · idempotency · rate limit · distributed lock
Texnologiya	Redis 7, `maxmemory-policy allkeys-lru` (db=5), `noeviction` (db=6)
Portlar	6379
Resurs	S0: 1 CPU / 2 GB → S5: cluster 6 node × 4 CPU / 16 GB
Masshtablash	Sentinel (S2) → Cluster (S4)
Sog'liq tekshiruvi	`PING` · `INFO memory` (used < 80%) · replikatsiya holati

⚠ DB ajratish (muhim):

DB	Vazifa	Eviction	Yo'qolsa
db=5	Kesh, sessiya, rate limit, idempotency	`allkeys-lru`	🟡 Qayta quriladi (DB'dan)
db=6	Channels layer (WS pub/sub)	`noeviction`	🟡 WS uziladi, qayta ulanadi

> ⚡ **Nega ajratilgan:** kesh to'lib ketganda LRU eski kalitlarni o'chiradi. Agar WS channel'lari ham shu DB'da bo'lsa — faol WS ulanishlari **jimgina o'ladi**.

Kalitlar sxemasi:

```
menu:<rid>:<lang>:v<n>          → menyu payload          TTL 300s
menuver:<rid>                   → menyu versiya raqami   TTL 3600s
dish:<rid>:<did>:<lang>         → taom kartochkasi       TTL 300s
stoplist:<rid>                  → tugagan taom ID'lari   TTL 5s   ⚡ qisqa
rest:<slug>                     → restoran meta          TTL 600s
qrtoken:<token>                 → stol → restaurant_id   TTL 600s
sess:<session_key>              → Django sessiya         TTL 3600s
idem:<scope>:<rid>:<key>:done   → idempotent javob       TTL 24h  ⚡ P4
idem:<scope>:<rid>:<key>:lock   → parallel qulf          TTL 30s
rl:<zone>:<ident>:<window>      → rate limit hisoblagich   TTL window
cb:<service>:state              → circuit breaker holati  TTL 300s ⚡ P8
lock:<resource>                 → distributed lock       TTL 30s
queue:<name>                    → navbat (LIST)          ♾ TTL yo'q
delayed:<name>                  → kechiktirilgan (ZSET)  ♾ TTL yo'q
```

⛔ Yiqilsa nima bo'ladi:
🟩 **Katta ta'sir, lekin biznes to'xtamaydi.**
- ✅ Ishlaydi: menyu (DB'dan, sekinroq ~3×), buyurtma yaratish, to'lov
- ❌ Yo'q: WS (channel layer), sessiya (mijoz savati yo'qoladi), rate limit
- ⚠ **Navbatdagi job'lar:** Redis LIST'dagi `job_id`lar yo'qoladi — LEKIN `Job` yozuvlari **PostgreSQL'da** (ADR-008), healer C9 `status='queued'` job'larni qayta navbatga qo'yadi ✅
- ⚠ Idempotency yo'qoladi → DB `UniqueConstraint` ikkinchi qatlam sifatida ushlaydi (P12)
Tiklanish: Sentinel failover 10s; kesh o'z-o'zidan qayta quriladi (cache stampede'ga qarshi `proxy_cache_lock` va Redis `SET NX`).

C7 — 🪣 S3 / MinIO

Maydon	Qiymat
Vazifasi	3D modellar (`.glb` , `.usdz`), taom rasmlari (`.webp`), QR PDF, backup, eksport
Texnologiya	MinIO (S0-S2, o'z serverimizda) → AWS S3 / Backblaze B2 (S3+)
Portlar	9000 (API), 9001 (konsol)
Resurs	S0: 100 GB → S5: 5 TB
Masshtablash	Ob'ekt saqlash — tabiiy cheksiz; MinIO distributed rejim (4+ node)
Sog'liq tekshiruvi	`HEAD /minio/health/live`

Bucket tuzilishi:

```
armenu-models/          ⚡ ADR-011 — kutubxona umumiy
├── library/
│   ├── osh_toy/
│   │   ├── v1/model.glb          (2,1 MB)
│   │   ├── v1/model.usdz         (2,8 MB)
│   │   └── v1/poster.webp        (45 KB)
│   └── somsa_gosht/v1/...
└── custom/
    └── <restaurant_id>/<dish_id>/v1/...

armenu-images/
└── <restaurant_id>/dishes/<dish_id>_<hash>.webp

armenu-qr/
└── <restaurant_id>/tables/<table_id>.pdf

armenu-backups/         (boshqa region)
├── pg/YYYY-MM-DD/
└── redis/YYYY-MM-DD/
```

Kirish siyosati:

Bucket	Ommaviy o'qish	Yozish	CDN origin
`armenu-models`	✅ ha (CDN orqali)	faqat worker	✅
`armenu-images`	✅ ha	web + worker	✅
`armenu-qr`	⛔ presigned URL (15 daq)	web	⛔
`armenu-backups`	⛔	backup skripti	⛔

⛔ Yiqilsa nima bo'ladi:
🟡 **O'rta ta'sir.**
- ✅ Ishlaydi: menyu (matn+narx), buyurtma, to'lov
- ⚠ 3D/AR: Cloudflare keshidan 30 kun davomida ko'rinadi (hit bo'lganlar).
Kesh miss → **rasm fallback** (P3 zanjiri)
- ✖ Yangi rasm/model yuklash yo'q → `worker:model3d` job'lari navbatda kutadi
Tiklanish: MinIO distributed rejimda 1 node yiqilsa avtomatik; to'liq — replica bucket'dan.

C8 — ⚙ WORKER (5 navbat)

Maydon	Qiymat
Vazifasi	Fon vazifalar: 3D generatsiya, fiskal chek, bildirishnoma, AI so'rov, analitika ETL
Texnologiya	Python 3.12, `manage.py run_worker --queue=<name>` , Redis `BLPOP`
Portlar	— (tashqi port yo'q)
Resurs	Har navbat uchun alohida systemd unit + `MemoryMax` + `CPUQuota` (P9)
Masshtablash	Gorizontal — navbat bo'yicha mustaqil (`armenu-worker@fiscal` × N)
Sog'liq tekshiruvi	`Job.heartbeat_at` (10s) + `armenu_queue_depth{queue}` metrikasi

Batafsil: **§7 — Worker turlari**.

⛔ Yiqilsa nima bo'ladi (navbatga qarab):

Navbat	Ta'sir	Nima ishlaydi
`model3d`	🟢 Kichik	Hammasi. Yangi 3D navbatda kutadi
`fiscal`	🟡 O'rta	✅ Buyurtma, to'lov. Chek navbatda (soliq muddati 24 soat — yetadi)
`notify`	🟢 Kichik	Hammasi. Telegram/push kechikadi
`ai`	🟢 Yo'q	AI chat "hozir mavjud emas, ofitsiantni chaqiring" (P3)
`analytics`	🟢 Yo'q	Hammasi. Statistika kechikadi

Tiklanish: systemd `Restart=always` 5s; healer C9 orphan job'larni (heartbeat > 15 daq) qayta navbatga qo'yadi.

C9 — 🔧 HEALER (self-healing daemon)

Maydon	Qiymat
Vazifasi	Har 60s tizimni tekshiradi, muammoni **o'zi tuzatadi**, tuzata olmasa alert yuboradi
Texnologiya	Python, `manage.py run_healer`, 15 mexanizm
Portlar	—
Resurs	0,5 CPU / 512 MB
Masshtablash	⛔ **Faqat 1 instans** (distributed lock bilan kafolatlanadi)
Sog'liq tekshiruvi	`armenu_healer_last_cycle_ts` metrikasi (> 180s → alert)

```
# core/healing/daemon.py
HEALER_LOCK = 'lock:healer'

def run_cycle():
    # ⚡ Faqat bitta healer ishlaydi (ko'p node bo'lsa ham)
    if not cache.add(HEALER_LOCK, socket.gethostname(), timeout=90):
        return
    try:
        for name, check, heal in CHECKS:
            ...
    finally:
        metrics.gauge('armenu_healer_last_cycle_ts', time.time())
```

⛔ **Yiqilsa nima bo'ladi:**
🟡 **Kechiktirilgan ta'sir.** Darhol hech narsa buzilmaydi, lekin avtomatik tiklash to'xtaydi: orphan job'lar `running`da qoladi, disk to'lishi tuzatilmaydi, stuck order alerti kelmaydi. **24 soat sezilmasa** — muammolar to'planadi.
Tiklanish: systemd 5s; `armenu_healer_last_cycle_ts` metrikasi 180s da alert beradi.

C10 — 📊 CLICKHOUSE (S2 dan)

Maydon	Qiymat
Vazifasi	Analitik event saqlash va tahlil (ADR-013)
Texnologiya	ClickHouse, `MergeTree`, oy bo'yicha partition, TTL 730 kun
Portlar	8123 (HTTP), 9000 (native)
Resurs	2 node × 8 CPU / 32 GB / 1 TB
Masshtablash	Shard + replica (Distributed table)
Sog'liq tekshiruvi	`SELECT 1` + `system.replication_queue` uzunligi

⛔ **Yiqilsa nima bo'ladi:**
🟠 **Biznesga ta'sir yo'q.** `worker:analytics` batch'lari DLQ'ga tushadi (yo'qolmaydi), ClickHouse tiklanganda qayta yoziladi. Admin hisobotlari vaqtincha mavjud emas (oxirgi materialized view snapshot ko'rsatiladi). Buyurtma, to'lov, menyu — ta'sirlanmaydi.

2.1 Konteynerlar yiqilish matritsasi (qisqacha)

Yiqilgan	Menyu	AR	Buyurtma	To'lov	Real-time	Admin	Daraja
Cloudflare	✓ sekin	✓ sekin	✓	✓	✓	✓	🟡
nginx (1/2)	✓	✓	✓	✓	✓	✓	🟠
nginx (barcha)	✗	✗	✗	✗	✗	✗	🔴
web (1/N)	✓	✓	✓	✓	✓	✓	🟠
web (barcha)	⚠ kesh	⚠ CDN	✗	✗	✗	✗	🔴
ws	✓	✓	✓	✓	⚠ polling	✓	🟡
PG primary	✓ replica	✓	✗ 30-60s	✗	✓	⚠ o'qish	🔴
PG replica	✓	✓	✓	✓	✓	✓	🟠
Redis	✓ sekin	✓	✓	✓	✗	✓	🟤
S3	✓	⚠ rasm	✓	✓	✓	✓	🟡
worker:fiscal	✓	✓	✓	✓	✓	✓	🟡 chek navbatda
worker:model3d	✓	✓ mavjud	✓	✓	✓	✓	🟠
healer	✓	✓	✓	✓	✓	✓	🟡 kechiktirilgan
ClickHouse	✓	✓	✓	✓	✓	⚠ hisobot	🟠

---

§3. DJANGO MODULLAR (C4 — Level 3)

```
armenu_project/
├── config/                # settings, urls, asgi, wsgi
├── core/                  # umumiy qatlam (§5)
├── apps/
│   ├── tenants/       1   | Restaurant, tarif, obuna
│   ├── accounts/      2   | User, Staff, rol, PIN, JWT
│   ├── menu/          3   | Category, Dish, Modifier, stop-list
│   ├── ar3d/          4 ⚡ | DishLibrary, DishModel3D, pipeline
│   ├── tables/        5   | Hall, Table, QR token
│   ├── guest/         6   | Guest, GuestSession, savat
│   ├── orders/        7 ⚡ | Order, holat mashinasi, KDS
│   ├── payments/      8   | Payment, Tip, gateway adapterlar
│   ├── fiscal/        9   | OFD, IKPU, chek
│   ├── loyalty/      10   | Bonus, keshbek, promo
│   ├── reviews/      11   | Review, Google yo'naltirish
│   ├── analytics/    12   | Event, hisobot, menyu-injiniring
│   ├── ai/           13 ⚡ | MCP qatlam, chatbot, tarjima
│   ├── notifications/     | (yordamchi) Telegram, push
│   └── platform_admin/    | (yordamchi) super-admin
├── static/ · templates/ · locale/ · deploy/
```

> **13 asosiy modul** raqamlangan. `notifications` va `platform_admin` — yordamchi
> (o'z domen modeliga ega emas, boshqalarga xizmat qiladi).

3.1 `apps/tenants` — Restoran va obuna

Maydon	Qiymat
Mas'uliyat chegarasi	Restoran identifikatori, tarif, obuna, sozlamalar. ⚡ Barcha tenant izolyatsiyasining manbai
Egalik qiladigan jadvallar	`tenants_restaurant`, `tenants_subscription`, `tenants_plan`, `tenants_settings`, `tenants_branch`
Bog'liqliklari	⛔ **Hech kimga** (eng past qatlam, faqat `core`)
Ajratish mumkinmi	⛔ Yo'q — hamma unga bog'liq

Ochiq interfeys:

```
# apps/tenants/services.py
def resolve_by_slug(slug: str) -> int | None:          # kesh bilan, 600s
def resolve_by_token(qr_token: str) -> int | None:     # kesh bilan, 600s
def get_settings(restaurant_id: int) -> RestaurantSettings
def is_feature_enabled(restaurant_id: int, feature: str) -> bool   # tarif gate
def check_quota(restaurant_id: int, resource: str) -> QuotaResult  # 3D model, AI so'rov
def purge_restaurant(restaurant_id: int, *, confirm: str) -> None  # GDPR
```

3.2 `apps/accounts` — Foydalanuvchi va rol

Maydon	Qiymat
Mas'uliyat chegarasi	Autentifikatsiya (3 rol), avtorizatsiya, PIN, JWT, sessiya
Egalik qiladigan jadvallar	`accounts_user`, `accounts_staff`, `accounts_role`, `accounts_permission`, `accounts_auditlog`
Bog'liqliklari	`tenants`
Ajratish mumkinmi	🟡 Nazariy (SSO servis), lekin foyda yo'q

3 xil autentifikatsiya:

Rol	Usul	Sessiya muddati	Saqlash
🧍 Mijoz	⛔ **Auth yo'q** — QR token → anonim sessiya	4 soat	Redis `sess:`
👨‍🍳 Ofitsiant	4-6 raqamli PIN (bcrypt)	Smena oxirigacha (12 soat)	Redis + `Staff.last_seen`
🧑‍💼 Admin	Login + parol (+ 2FA super-admin)	JWT 15 daq + refresh 7 kun	JWT

```
# apps/accounts/services.py
def authenticate_staff(restaurant_id: int, pin: str) -> Staff | None   # rate limit 5/15daq
def issue_guest_session(table_id: int) -> GuestSession
def has_permission(user, perm: str, restaurant_id: int) -> bool
def audit(actor, action: str, target, **meta) -> None                  # AuditLog
```

3.3 `apps/menu` — Menyu

Maydon	Qiymat
Mas'uliyat chegarasi	Kategoriya, taom, modifikator, stop-list, tarjima, narx. ⚡ **Narx yagona manbai** (ADR-006)
Egalik qiladigan jadvallar	`menu_category`, `menu_dish`, `menu_dishtranslation`, `menu_modifier`, `menu_modifiergroup`, `menu_stoplist`, `menu_allergen`
Bog'liqliklari	`tenants`
Ajratish mumkinmi	🟡 Nazariy, lekin `orders` bilan zich bog'langan (narx)

```
# apps/menu/services.py
def get_menu_payload(restaurant_id: int, lang: str) -> dict      # ⚡ keshlanadi (P7)
def get_dish(restaurant_id: int, dish_id: int, lang: str) -> dict
def stoplist_ids(restaurant_id: int) -> set[int]                 # TTL 5s
def set_stoplist(restaurant_id: int, dish_id: int, on: bool) -> None
def bump_menu_version(restaurant_id: int) -> None                # kesh invalidatsiya
def search_semantic(restaurant_id: int, query: str, lang: str, limit: int) -> list
```

⚡ Kesh invalidatsiya oqimi:

```
Admin narxni o'zgartiradi
   │
   ▼
post_save(Dish)  →  on_commit  →  bump_menu_version(rid)
   │                                  │
   │                                  ├─▶ Restaurant.menu_version += 1
   │                                  ├─▶ cache.delete('menuver:<rid>')
   │                                  └─▶ Cloudflare purge Cache-Tag: menu-<rid>
   ▼
Barcha `menu:<rid>:*:v<eski>` kalitlari o'z-o'zidan foydalanilmaydi
(TTL bilan o'ladi — DELETE kerak emas)
```

3.4 `apps/ar3d` — 3D va AR ⚡

Maydon	Qiymat
Mas'uliyat chegarasi	3D model kutubxonasi, restoran modellari, generatsiya pipeline'i, AR ko'rish, format konversiyasi
Egalik qiladigan jadvallar	`ar3d_dishlibrary`, `ar3d_dishmodel3d`, `ar3d_modeljob`, `ar3d_arview`, `ar3d_modelversion`
Bog'liqliklari	`tenants`, `menu`
Ajratish mumkinmi	✅ **HA — birinchi nomzod** (og'ir, mustaqil, GPU kerak bo'lishi mumkin)

```
# apps/ar3d/services.py
def get_ar_payload(restaurant_id: int, dish_id: int) -> ARPayload
    # → {glb_url, usdz_url, poster_url, scale, placement}  ⚡ keshlanadi

def request_model(dish_id: int, *, source: str, ref_images: list) -> int
    # → enqueue('model3d', ...), Job ID qaytaradi

def match_library(dish: Dish, threshold: float = 0.82) -> DishLibrary | None
    # ⚡ ADR-011 — semantik moslash (pgvector), TAKLIF sifatida

def track_ar_view(restaurant_id, dish_id, session_id, duration_ms) -> None
    # → enqueue('analytics', ...)
```

3D generatsiya pipeline'i:

```
[1] Admin taom rasmini yuklaydi (2-6 ta rakurs)
     │
     ▼
[2] match_library(dish)  ── pgvector cosine ──▶  o'xshashlik ≥ 0,82?
     │                                                  │
     │ yo'q                                       ha ──┴──▶ TAKLIF ko'rsatiladi
     ▼                                                       admin tasdiqlaydi ✅ TUGADI
[3] enqueue('model3d', 'generate', dish_id=...)
     │
     ▼
[4] worker:model3d
     ├── rasm tayyorlash (fon olib tashlash, normalizatsiya)
     ├── AI provayder zanjiri:  Meshy → Tripo → Rodin      (breaker + fallback)
     ├── GLB yuklab olish
     ├── optimizatsiya (Draco siqish, tekstura → KTX2, < 4 MB)
     ├── USDZ konversiya (usdzconvert / Reality Converter CLI)
     ├── poster render (Blender headless yoki model-viewer screenshot)
     ├── S3'ga yuklash (versiyalangan yo'l)
     └── DishModel3D.objects.create(...)  →  cache invalidatsiya
     │
     ▼
[5] Admin ko'rib chiqadi → tasdiqlaydi / rad etadi
     │
     ▼
[6] ✅ Mijozga AR ko'rinadi

O'rtacha vaqt: 3-8 daqiqa · Narx: $0,20-0,50 · Muvaffaqiyat: ~78%
```

3.5 `apps/tables` — Zal va stol

Maydon	Qiymat
Mas'uliyat chegarasi	Zal, stol, QR token, stol holati, zal plani
Egalik qiladigan jadvallar	`tables_hall`, `tables_table`, `tables_qrtoken`, `tables_tablesession`
Bog'liqliklari	`tenants`
Ajratish mumkinmi	⛔ Yo'q (kichik, `orders` bilan zich)

```
# apps/tables/services.py
def resolve_token(token: str) -> TableContext | None    # → (restaurant_id, table_id)
def rotate_token(table_id: int) -> str                  # QR yangilash (xavfsizlik)
def set_status(table_id: int, status: str) -> None      # free|ordering|served|bill
def generate_qr_pdf(restaurant_id: int, hall_id: int) -> str   # presigned S3 URL
```

QR token xavfsizligi:

Xususiyat	Qiymat
Format	22 belgi, `secrets.token_urlsafe(16)`
Ma'no	⛔ Ketma-ket ID emas (brute-force himoyasi)
Aylantirish	Qo'lda (stol o'zgarsa) yoki 12 oyda
Rate limit	60/daq per IP
Kesh	`qrtoken:<token>` → `restaurant_id`, TTL 600s

3.6 `apps/guest` — Mijoz sessiyasi

Maydon	Qiymat
Mas'uliyat chegarasi	Anonim mijoz sessiyasi, savat, til tanlovi, telefon (ixtiyoriy)
Egalik qiladigan jadvallar	`guest_guest`, `guest_guestsession`, `guest_cart`, `guest_cartitem`
Bog'liqliklari	`tenants`, `tables`
Ajratish mumkinmi	⛔ Yo'q

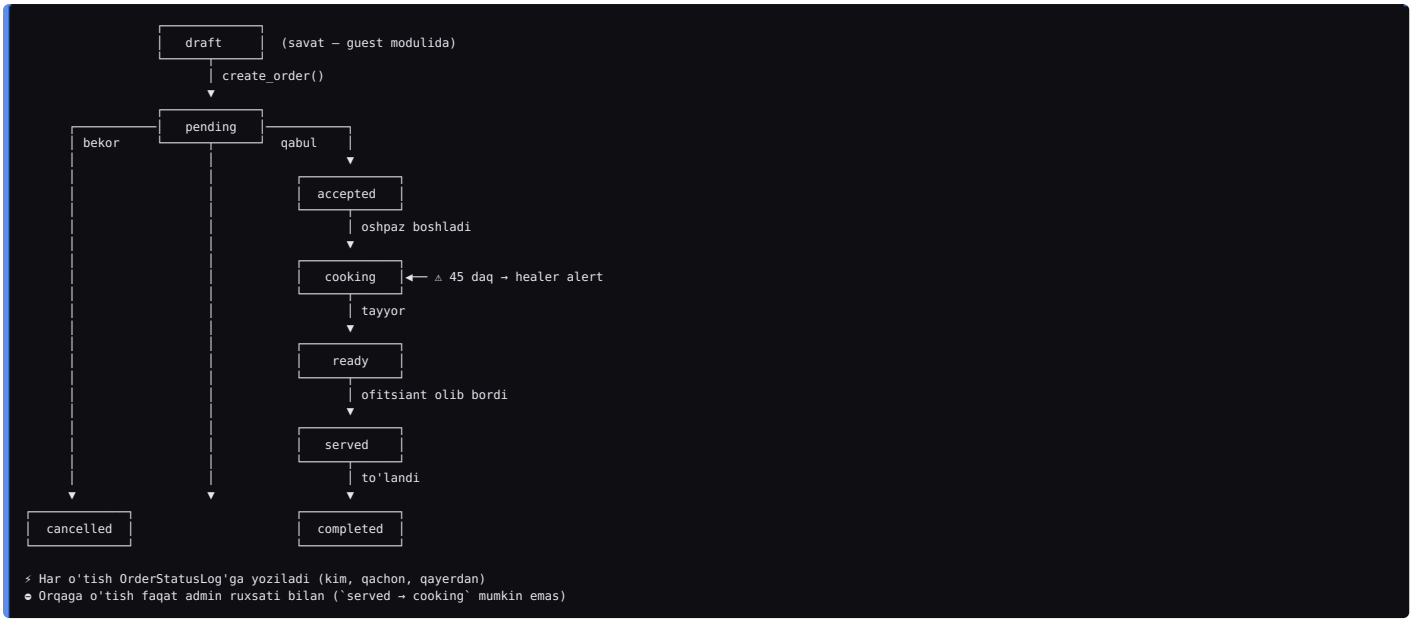
> ⚡ **Savat serverda saqlanadi** (`guest_cart`), klientda emas. Sabab: mijoz sahifani yopib qayta ochsa savat qolishi kerak; ofitsiant mijoz savatini ko'rishi mumkin.
> Narx savatda **saqlanmaydi** — buyurtma paytida qayta hisoblanadi (ADR-006).

```
# apps/guest/services.py
def get_or_create_session(table_token: str, lang: str) -> GuestSession
def add_to_cart(session_id: str, dish_id: int, qty: int, modifier_ids: list) -> Cart
def get_cart(session_id: str) -> CartView            # narx SERVERDA hisoblanadi
def clear_cart(session_id: str) -> None
def attach_phone(session_id: str, phone: str) -> Guest   # 998XXXXXXXX normalizatsiya
```

3.7 `apps/orders` — Buyurtma ⚡

Maydon	Qiymat
Mas'uliyat chegarasi	Buyurtma hayotiy sikli, holat mashinasi, KDS, ofitsiant chaqiruvi. ⚡ **Eng kritik modul**
Egalik qiladigan jadvallar	`orders_order` (partition), `orders_orderitem` (partition), `orders_orderstatuslog`, `orders_waitercall`
Bog'liqliklari	`tenants`, `menu`, `tables`, `guest`, `accounts`
Ajratish mumkinmi	⛔ Yo'q — tranzaksion yadro (ADR-001 sababi)

Holat mashinasi:




```
# apps/orders/services.py
def create_order(*, restaurant, table, items, session, idem_key) -> Order   # ⚡ atomic
def transition(order: Order, to_status: str, *, actor, reason='') -> Order  # holat mashinasi
def cancel_order(order: Order, *, actor, reason: str) -> Order
def call_waiter(table_id: int, reason: str) -> WaiterCall                   # rate limit 1/daq
def kds_queue(restaurant_id: int) -> list[dict]                            # KDS ro'yxati
def estimate_eta(restaurant_id: int, items) -> int                         # daqiqa
```

Buyurtma yaratish — to'liq oqim:

```
@api_view(['POST'])
@idempotent('order.create')                        # ⚡ P4
def create_order_view(request):
    ser = OrderInSerializer(data=request.data)    # ⛔ price maydoni YO'Q (ADR-006)
    ser.is_valid(raise_exception=True)

    with transaction.atomic():
        # 1) Validatsiya (7 bosqich) — narx SERVERDA
        price = calc_order_total(
            restaurant=request.restaurant,
            items_in=ser.validated_data['items'],
            promo_code=ser.validated_data.get('promo_code'),
        )
        # 2) Yozuv
        order = Order.objects.create(
            restaurant_id=request.restaurant_id,
            table_id=ser.validated_data['table_id'],
            session_id=request.session_id,
            total=price.total,
            price_breakdown=price.breakdown,       # ⚡ audit izi
            idempotency_key=request.headers['X-Idempotency-Key'],
        )
        OrderItem.objects.bulk_create(build_items(order, price))
        Table.objects.filter(pk=order.table_id).update(status='ordering')

    # 3) ⚡ COMMIT'dan keyin — WS va navbat (AP-09)
    transaction.on_commit(lambda: order_created.send(sender=Order, order=order))
    return Response(OrderOutSerializer(order).data, status=201)
```

3.8 `apps/payments` — To'lov

Maydon	Qiymat
Mas'uliyat chegarasi	To'lov yaratish, webhook, chaevoy, qaytarish, gateway adapterlar (ADR-014)
Egalik qiladigan jadvallar	`payments_payment` (partition), `payments_tip`, `payments_refund`, `payments_webhookevent`
Bog'liqliklari	`tenants`, `orders`
Ajratish mumkinmi	🟡 Nazariy (PCI izolyatsiya foydasi), lekin karta ma'lumoti bizda saqlanmaydi

```
# apps/payments/services.py
def create_payment(*, order, method, preferred=None) -> PaymentIntent    # fallback zanjiri
def handle_webhook(provider: str, request) -> WebhookResult              # HMAC tekshiruvi
def refund(payment: Payment, *, amount, reason, actor) -> Refund
def add_tip(order: Order, amount: Decimal, staff_id: int) -> Tip
```

Webhook xavfsizligi (P12 — 4 qatlam):

#	Qatlam	Tekshiruv
1	Imzo	HMAC / Basic auth (provayderga qarab)
2	Idempotency	`WebhookEvent.provider_event_id` unique
3	Summa	`webhook.amount == payment.amount` (aks holda ☎ alert)
4	Holat	Holat o'tishi qonuniymi (`created → paid` ✅, `paid → paid` — jim o'tkazish)

3.9 `apps/fiscal` — Fiskal (OFD)

Maydon	Qiymat
Mas'uliyat chegarasi	Soliq cheki, IKPU kodlar, OFD integratsiya, chek arxivi
Egalik qiladigan jadvallar	`fiscal_receipt`, `fiscal_ikpu`, `fiscal_ofdlog`, `fiscal_vatrate`
Bog'liqliklari	`tenants`, `orders`, `payments`
Ajratish mumkinmi	🟡 Ha (mamlakatga xos mantiq — xalqaro kengayishda foydali)

```
# apps/fiscal/services.py
def enqueue_receipt(order: Order) -> int                  # ⚡ HECH QACHON sinxron emas
def send_receipt(order_id: int) -> Receipt                # worker:fiscal ichida
def resolve_ikpu(dish: Dish) -> str                       # tasnif.soliq.uz kesh
def get_receipt_url(order: Order) -> str | None
```

⚠ **Kritik qoida:** fiskal chek **hech qachon** web so'rovda yuborilmaydi (AP-04).
Navbat `fiscal` — kafolatlangan 5 worker, retry 10 marta, 48 soat davomida, keyin DLQ + alert P1.

3.10 `apps/loyalty` — Sodiqlik

Maydon	Qiymat
Mas'uliyat chegarasi	Bonus, keshbek, promo-kod, sodiqlik darajalari
Egalik qiladigan jadvallar	`loyalty_program`, `loyalty_bonusaccount`, `loyalty_bonustransaction`, `loyalty_promocode`, `loyalty_tier`
Bog'liqliklari	`tenants`, `guest`, `orders`
Ajratish mumkinmi	✅ Ha (mustaqil domen)

```
# apps/loyalty/services.py
def apply_promo(restaurant, code: str | None, subtotal: Decimal) -> Decimal  # ⚡ narx zanjirida
def accrue_cashback(order: Order) -> BonusTransaction                        # event handler
def redeem_bonus(guest, amount: Decimal, order: Order) -> BonusTransaction
def get_balance(guest_id: int, restaurant_id: int) -> Decimal
```

3.11 `apps/reviews` — Sharhlar

Maydon	Qiymat
Mas'uliyat chegarasi	Ichki sharh, baho, Google/Yandex yo'naltirish, negativ filtr
Egalik qiladigan jadvallar	`reviews_review`, `reviews_reviewphoto`, `reviews_externallink`
Bog'liqliklari	`tenants`, `guest`, `orders`
Ajratish mumkinmi	✅ Ha

Smart yo'naltirish mantiqi:

```
Mijoz baho beradi
   │
   ├─ 4-5 ★ ──► "Google'da ham yozib qo'ysangiz?" → Google Maps havolasi
   │
   └─ 1-3 ★ ──► Ichki forma: "Nima yoqmadi?" → restoran admini darhol xabar oladi
                (⛔ tashqi platformaga yo'naltirilmaydi)
```

3.12 `apps/analytics` — Analitika

Maydon	Qiymat
Mas'uliyat chegarasi	Event yig'ish, hisobot, menyu-injiniring, AR konversiya
Egalik qiladigan jadvallar	`analytics_event` (partition, S0-S1) · ClickHouse `events` (S2+) · `analytics_report`
Bog'liqliklari	Hech kimga (faqat event **qabul qiladi**)
Ajratish mumkinmi	✅ **HA — uchinchi nomzod** (ADR-013 bilan birga)

```
# apps/analytics/services.py
def track(event_type: str, *, restaurant_id: int, **props) -> None   # ⚡ faqat enqueue
def menu_engineering(restaurant_id: int, days: int = 30) -> dict      # Star/Plow/Puzzle/Dog
def ar_conversion(restaurant_id: int, days: int = 30) -> dict         # AR ko'rgan → buyurtma
def revenue_report(restaurant_id: int, period) -> dict
```

Menyu-injiniring matritsasi (asosiy hisobot):

```
              Yuqori marjinallik        Past marjinallik
            ┌──────────────────────┬──────────────────────┐
  Ko'p      │  ⭐ STARS            │  🐎 PLOWHORSES       │
  sotiladi  │  Saqlash, ko'rsatish │  Narx oshirish yoki  │
            │  AR majburiy         │  tannarx pasaytirish │
            ├──────────────────────┼──────────────────────┤
  Kam       │  🧩 PUZZLES          │  🐕 DOGS             │
  sotiladi  │  Reklama, AR qo'shish,  │  Menyudan olib tashlash│
            │  ofitsiant tavsiyasi │                      │
            └──────────────────────┴──────────────────────┘
⚡ ARMENU farqi: har kvadrantga "AR ko'rish → buyurtma konversiyasi" qo'shiladi
```

3.13 `apps/ai` — AI va MCP ⚡

Maydon	Qiymat
Mas'uliyat chegarasi	MCP tool registry, chatbot, tarjima, taom tavsifi, provayder adapterlar (ADR-009)
Egalik qiladigan jadvallar	`ai_conversation`, `ai_message`, `ai_toolcall`, `ai_usagelog`, `ai_providerstate`
Bog'liqliklari	`tenants`, `menu`, `ar3d` (⚡ faqat servis funksiyalari orqali)
Ajratish mumkinmi	✅ **HA — ikkinchi nomzod** (tashqi API, sekin, mustaqil)

```
apps/ai/
├── mcp/
│   ├── registry.py      # @mcp_tool dekoratori, tool ro'yxati
│   ├── guard.py         # ⚡ tenant filtri + ruxsat + rate limit
│   ├── audit.py         # har chaqiruv AuditLog'ga
│   └── tools/
│       ├── menu.py      # search_menu, get_dish, check_stoplist
│       ├── orders.py    # get_order_status  (faqat O'Z buyurtmasi)
│       └── actions.py   # call_waiter       (⛔ yozuv tool'lari YO'Q)
├── providers/
│   ├── base.py          # Protocol: chat(msgs, tools) -> Response
│   ├── gemini.py · claude.py · gpt.py · groq.py · deepseek.py
│   ├── meshy.py · tripo.py   # 3D generatsiya
│   └── chain.py         # fallback zanjiri + circuit breaker
└── services.py
```

```
# apps/ai/services.py
def chat(session_id: str, message: str, lang: str) -> AIResponse   # MCP tool'lar bilan
def translate_menu(restaurant_id: int, target_langs: list) -> int  # → enqueue('ai', ...)
def generate_description(dish_id: int, tone: str) -> str
def embed_dish(dish_id: int) -> None                               # pgvector uchun
```

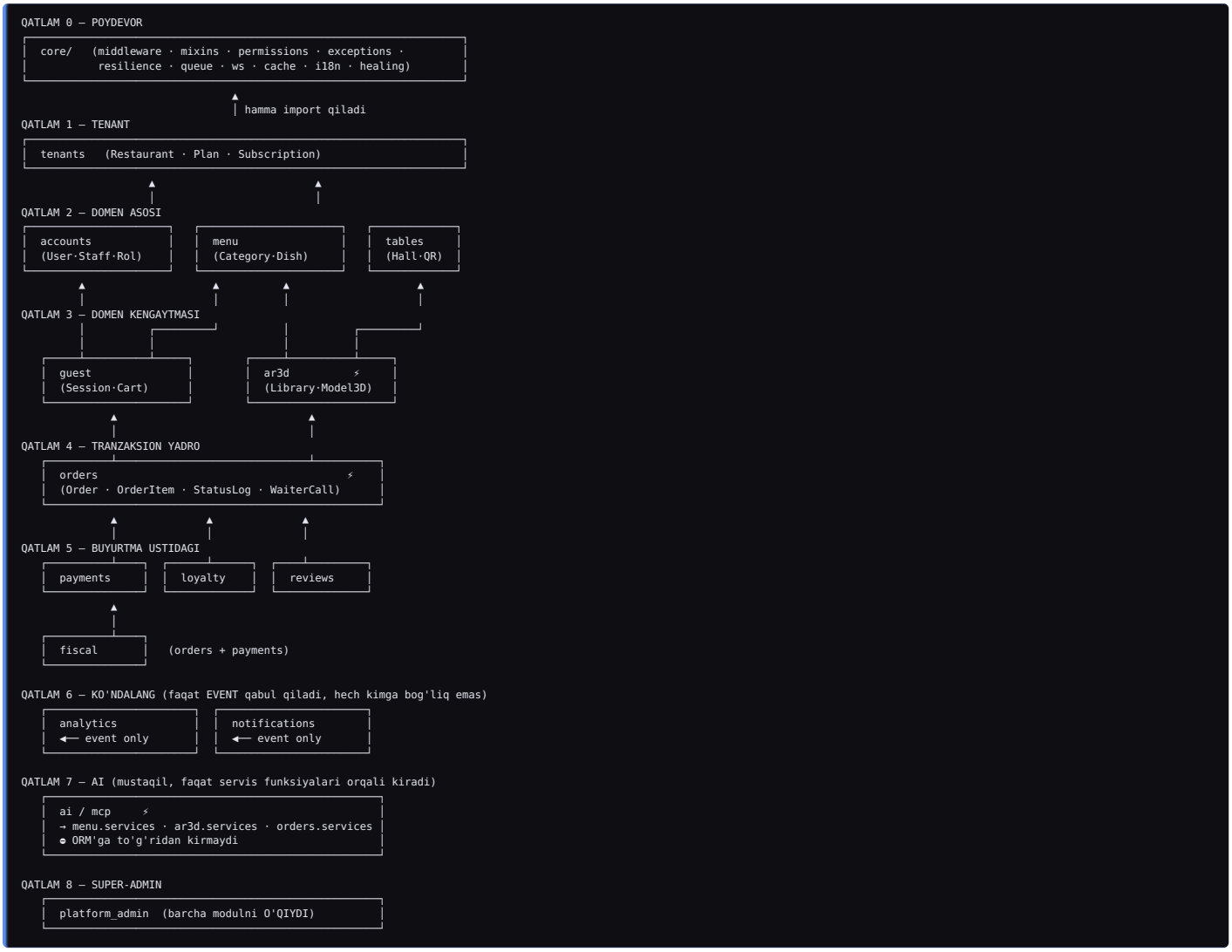
3.14 Modullar qisqacha jadvali

#	Modul	Jadval	Faza	Ajratish	Kritiklik
1	`tenants`	5	F1	⛔	🔴
2	`accounts`	5	F1	🟡	🔴
3	`menu`	7	F2	🟡	🔴
4	`ar3d` ⚡	5	F5	✅ 1-nomzod	🟡
5	`tables`	4	F3	⛔	🔴
6	`guest`	4	F3-F4	⛔	🔴
7	`orders` ⚡	4	F6	⛔	🔴
8	`payments`	4	F7	🟡	🔴
9	`fiscal`	4	F8	🟡	🟡
10	`loyalty`	5	F10	✅	🟢
11	`reviews`	3	F10	✅	🟢
12	`analytics`	2	F11	✅ 3-nomzod	🟢
13	`ai` ⚡	5	F9	✅ 2-nomzod	🟢
—	`notifications`	3	F6	✅	🟡
—	`platform_admin`	2	F1	🟡	🟢

---

§4. MODULLAR BOG'LIQLIK GRAFI

> ⚡ **Qoida:** bog'liqlik faqat **pastdan yuqoriga**. Teskari yo'nalish — **faqat event** (P5). Grafda **sikl yo'q** — bu CI'da tekshiriladi (`test_no_cross_import.py`).



4.1 Ruxsat etilgan bog'liqlik matritsasi

> Satr = kim import qiladi · Ustun = kimni import qiladi · ✅ ruxsat · ⛔ taqiq · 📨 faqat event

↓ importer / →	core	tenants	accounts	menu	tables	guest	ar3d	orders	payments	fiscal	loyalty	reviews	analytics	ai
tenants	✅	—	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔
accounts	✅	✅	—	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔
menu	✅	✅	⛔	—	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔
tables	✅	✅	⛔	⛔	—	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔
guest	✅	✅	⛔	✅	✅	—	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔
ar3d	✅	✅	⛔	✅	⛔	⛔	—	⛔	⛔	⛔	⛔	⛔	⛔	⛔
orders	✅	✅	✅	✅	✅	✅	⛔	—	⛔	⛔	✅[1]	⛔	📨	⛔
payments	✅	✅	✅	⛔	⛔	⛔	⛔	✅	—	⛔	⛔	⛔	📨	⛔
fiscal	✅	✅	⛔	✅	⛔	⛔	⛔	✅	✅	—	⛔	⛔	📨	⛔
loyalty	✅	✅	⛔	⛔	⛔	✅	⛔	✅	⛔	⛔	—	⛔	📨	⛔
reviews	✅	✅	⛔	⛔	⛔	✅	⛔	✅	⛔	⛔	⛔	—	📨	⛔
analytics	✅	✅	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	—	⛔
notifications	✅	✅	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔	⛔
ai	✅	✅	⛔	✅[2]	⛔	⛔	✅[2]	✅[2]	⛔	⛔	⛔	⛔	⛔	—
platform_admin	✅	✅	✅	✅	✅	✅	✅	✅	✅	✅	✅	✅	✅	✅

[1] `orders` → `loyalty.services.apply_promo()` — narx hisoblash zanjirida (faqat servis funksiyasi)

[2] `ai` → faqat `services.py` funksiyalari (⛔ modellar/ORM emas)

4.2 Siklsizlik isboti (topologik tartib)

```
core → tenants → {accounts, menu, tables} → {guest, ar3d} → orders
     → {payments, loyalty, reviews} → fiscal → {analytics, notifications} → ai → platform_admin

Har o'q chapdan o'ngga. Teskari o'q YO'Q. ∎ Yo'naltirilgan asiklik graf (DAG) ✓
```

CI tekshiruvi:

```
# tests/architecture/test_no_cross_import.py
import ast, pathlib

ALLOWED = {                      # §4.1 matritsasi kodda
    'tenants':   {'core'},
    'accounts':  {'core', 'tenants'},
    'menu':      {'core', 'tenants'},
    'tables':    {'core', 'tenants'},
    'guest':     {'core', 'tenants', 'menu', 'tables'},
    'ar3d':      {'core', 'tenants', 'menu'},
    'orders':    {'core', 'tenants', 'accounts', 'menu', 'tables', 'guest', 'loyalty'},
    'payments':  {'core', 'tenants', 'accounts', 'orders'},
    'fiscal':    {'core', 'tenants', 'menu', 'orders', 'payments'},
    'loyalty':   {'core', 'tenants', 'guest', 'orders'},
    'reviews':   {'core', 'tenants', 'guest', 'orders'},
    'analytics': {'core', 'tenants'},
    'notifications': {'core', 'tenants'},
    'ai':        {'core', 'tenants', 'menu', 'ar3d', 'orders'},
}

def test_module_dependencies():
    for app, allowed in ALLOWED.items():
        for py in pathlib.Path(f'apps/{app}').rglob('*.py'):
            tree = ast.parse(py.read_text())
            for node in ast.walk(tree):
                mods = []
                if isinstance(node, ast.ImportFrom) and node.module:
                    mods = [node.module]
                elif isinstance(node, ast.Import):
                    mods = [a.name for a in node.names]
                for m in mods:
                    if m.startswith('apps.'):
                        target = m.split('.')[1]
                        assert target == app or target in allowed, (
                            f'⛔ {app} → {target} taqiqlangan ({py}). '
                            f'Event ishlating (P5).')
```

---

§5. `core/` QATLAMI

> `core/` — barcha modul foydalanadigan **umumiy infratuzilma**. Domen mantiqi **⛔ bu yerda bo'lmaydi** (`Order`, `Dish` haqida bilmaydi).

```
core/
├── models.py          # BaseModel, TenantModel, TimeStampedModel, Job
├── middleware.py      # TenantMiddleware, ObservabilityMiddleware, ReadYourWrites
├── mixins.py          # TenantScopedMixin, IdempotentMixin, CachedListMixin
├── permissions.py     # IsGuest, IsWaiter, IsRestaurantAdmin, IsPlatformAdmin
├── exceptions.py      # ARMENUError iyerarxiyasi + DRF handler
├── resilience.py      # CircuitBreaker, retry, with_fallback_chain, timeout
├── queue.py           # enqueue, dequeue, DLQ, QUEUE_LIMITS
├── ws.py              # ws_send, group nomlari, tenant guard
├── cache.py           # versiyalangan kalit, invalidatsiya, stampede himoyasi
├── i18n.py            # 3 til, tarjima fallback, lokalizatsiya
├── idempotency.py     # @idempotent dekoratori
├── ratelimit.py       # zona-asosli rate limiter
├── events.py          # domain signal'lar
├── metrics.py         # Prometheus wrapper
├── health.py          # /healthz, /readyz, /metrics
├── storage.py         # S3 wrapper, presigned URL
└── healing/
    ├── daemon.py      # 60s sikl, distributed lock
    └── checks.py      # 15 mexanizm
```

5.1 Fayl kartochkalari

`core/models.py`

```
class TimeStampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    updated_at = models.DateTimeField(auto_now=True)
    class Meta: abstract = True


class TenantModel(TimeStampedModel):
    """⚡ Tenant'ga tegishli har model shundan meros oladi."""
    restaurant = models.ForeignKey('tenants.Restaurant', on_delete=models.CASCADE,
                                   db_index=True, related_name='+')
    class Meta:
        abstract = True
        indexes = [models.Index(fields=['restaurant', '-created_at'])]


class Job(TimeStampedModel):
    """Navbat yozuvi (ADR-008). PostgreSQL'da — Redis yiqilsa yo'qolmaydi."""
    # ... §2 C6 va ADR-008 da to'liq
```

Nima beradi	Kim ishlatadi
`TenantModel` — `restaurant` FK + indeks avtomatik	Barcha domen model
`Job` — navbat holati, heartbeat, trace_id	`core/queue.py`, healer

`core/middleware.py`

Middleware	Tartib	Vazifasi
`ObservabilityMiddleware`	1	`trace_id` yaratish, davomiylik metrikasi, sekin so'rov log
`TenantMiddleware`	2	`restaurant_id` aniqlash (QR token / subdomain / staff sessiya)
`ReadYourWritesMiddleware`	3	Yozuv so'rovida primary'dan o'qish (P6)
`RateLimitMiddleware`	4	Zona-asosli limit (ADR jadvali)
`SecurityHeadersMiddleware`	5	CSP, HSTS, X-Frame-Options

> ⚠ **Tartib muhim:** `TenantMiddleware` `RateLimitMiddleware` dan **oldin** —
> rate limit kaliti `restaurant_id` ni ishlatadi.

`core/mixins.py`

```
class TenantScopedMixin:
    """⚡ Har tenant ViewSet'da MAJBURIY (ADR-007, P12 qatlam 1)."""
    tenant_field = 'restaurant_id'
    def get_queryset(self): ...          # §ADR-007 da to'liq
    def perform_create(self, serializer): ...


class CachedListMixin:
    """GET list javobini versiyalangan kalit bilan keshlaydi (P7)."""
    cache_ttl = 300
    cache_prefix: str

    def list(self, request, *a, **kw):
        key = f'{self.cache_prefix}:{request.restaurant_id}:{get_language()}:v{self.version()}'
        data = cache.get(key)
        if data is None:
            data = super().list(request, *a, **kw).data
            cache.set(key, data, self.cache_ttl)
        return Response(data)


class SoftDeleteMixin:
    """O'chirish = is_deleted=True. ⚡ MEMORY qoidasi: 'o'chir' = yashirish."""
    def perform_destroy(self, instance):
        instance.is_deleted = True
        instance.deleted_at = timezone.now()
        instance.save(update_fields=['is_deleted', 'deleted_at'])
```

`core/permissions.py`

Sinf	Kim	Tekshiruv
`IsGuest`	Mijoz	Amal qiluvchi `GuestSession` + `restaurant_id` mos
`IsWaiter`	Ofitsiant	`Staff.role='waiter'` + smena ochiq
`IsKitchen`	Oshpaz	`Staff.role='kitchen'`
`IsRestaurantAdmin`	Restoran admini	`Staff.role='admin'` + `restaurant_id` mos
`IsPlatformAdmin`	Super-admin	`User.is_platform_admin` + 2FA
`ReadOnlyOrAdmin`	Aralash	GET hammaga, yozuv adminga

`core/exceptions.py`

```
class ARMENUError(APIException):
    """Barcha xatolarning ildizi. Har biri: kod, xabar (3 til), HTTP status."""
    code_name = 'armenu_error'

class TenantNotResolved(ARMENUError):     status_code = 400; code_name = 'tenant_not_resolved'
class QuotaExceeded(ARMENUError):         status_code = 402; code_name = 'quota_exceeded'
class DishUnavailable(ARMENUError):       status_code = 409; code_name = 'dish_unavailable'
class ServiceUnavailable(ARMENUError):    status_code = 503; code_name = 'service_unavailable'
class AllProvidersDown(ARMENUError):      status_code = 503; code_name = 'all_providers_down'
class QueueFull(ARMENUError):             status_code = 503; code_name = 'queue_full'
class Conflict(ARMENUError):              status_code = 409; code_name = 'conflict'

# Yagona javob formati (barcha xato uchun)
# {"error": {"code": "dish_unavailable", "message": "Somsa hozircha yo'q",
#            "detail": {...}, "trace_id": "a1b2c3d4"}}
```

`core/resilience.py`

Funksiya	Vazifasi
`CircuitBreaker`	3 holatli avtomat (P8), holat Redis'da
`retry(attempts, backoff, jitter)`	Eksponensial qayta urinish dekoratori
`with_fallback_chain(providers, ...)`	Zanjir bo'ylab yurish (P3)
`timeout_guard(seconds)`	Kontekst menejer — timeout majburlash
`get_breaker(name)`	Ro'yxatdan breaker olish

`core/queue.py`

Funksiya	Vazifasi
`enqueue(queue, task, *, delay=0, **payload) -> int`	Navbatga qo'yish; `trace_id` avtomatik
`dequeue(queue, timeout=5) -> Job | None`	`BLPOP` + `Job` yuklash
`move_to_dlq(queue, job, reason)`	Dead Letter Queue
`promote_delayed(queue)`	Kechiktirilganlarni (ZSET) navbatga ko'chirish
`queue_depth(queue) -> int`	Metrik uchun
`QUEUE_LIMITS`	Bulkhead konfiguratsiyasi (P9)

`core/ws.py`

```
def group_name(restaurant_id: int, channel: str) -> str:
    """⚡ Yagona nuqta — tenant'siz guruh yaratish imkonsiz (AP-17)."""
    assert channel in ('kds', 'waiters', 'admin'), f'noma\'lum kanal: {channel}'
    assert restaurant_id, 'restaurant_id majburiy'
    return f'rest_{restaurant_id}_{channel}'

def guest_group(session_id: str) -> str:
    return f'guest_{session_id}'

def ws_send(group: str, payload: dict) -> None:
    """⚡ Faqat transaction.on_commit ichida chaqiriladi (AP-09)."""
    async_to_sync(get_channel_layer().group_send)(group, {'type': 'broadcast', **payload})
```

`core/cache.py`

Funksiya	Vazifasi
`versioned_key(prefix, rid, *parts) -> str`	`menu:12:uz:v7`
`get_version(rid, kind) -> int`	Versiya raqami (kesh + DB)
`bump_version(rid, kind)`	Invalidatsiya (DELETE emas)
`cache_or_build(key, builder, ttl)`	Stampede himoyasi (`SET NX` lock)
`warm_up(restaurant_id)`	Restart'dan keyin issiq kesh (healer)

`core/i18n.py`

Funksiya	Vazifasi
`resolve_language(request) -> str`	`?lang=` → `Accept-Language` → restoran default → `uz`
`translate_field(obj, field, lang) -> str`	Tarjima fallback: `lang` → `uz` → `ru` → bo'sh
`format_price(amount, lang) -> str`	`45 000 so'm` / `45 000 сум` / `45,000 UZS`
`SUPPORTED = ('uz', 'ru', 'en')`	⚡ Kesh kalitida ishtirok etadi

`core/healing/`

15 mexanizm (`checks.py`):

#	Nom	Tekshiradi	Tuzatadi
1	`orphan_jobs`	`running` + heartbeat > 15 daq	Qayta navbatga / DLQ
2	`stuck_orders`	`cooking` > 45 daq	⚠ Alert (odam kerak)
3	`db_connections`	Pool > 85%	Idle conn yopish
4	`redis_memory`	`used_memory` > 80%	Eski kalit tozalash
5	`disk_space`	> 85%	Log/temp o'chirish
6	`worker_alive`	Har navbatda ≥1 worker	systemd restart
7	`fiscal_queue`	depth > 50, 10 daq	Worker qo'shish + alert P1
8	`ws_connections`	Keskin tushish > 20%	Daphne restart
9	`cdn_health`	Origin pull xatosi	Cache-Tag purge
10	`ai_providers`	Breaker OPEN > 30 daq	HALF_OPEN majburlash
11	`replication_lag`	> 30s	Replica'dan o'qishni to'xtatish
12	`cache_hit_ratio`	< 70% (10 daq)	Kesh warm-up
13	`dlq_depth`	> 100	Alert + avto-retry
14	`cert_expiry`	< 14 kun	certbot renew
15	`backup_verify`	Kunlik tiklash sinovi	⚠ Alert

---

§6. FRONTEND KOMPONENTLARI

> ADR-003: Vanilla JS + PWA. 4 ta mustaqil ilova, umumiy `core.js`.

```
static/js/
├── core/
│   ├── api.js          # fetch wrapper: trace_id, idempotency key, xato
│   ├── store.js        # observable holat (~80 qator)
│   ├── realtime.js     # WS → SSE → polling (avtomatik fallback)
│   ├── i18n.js         # 3 til, `t('key')`
│   └── ui.js           # toast, modal, skeleton, spinner
├── guest/              # mijoz PWA
├── staff/              # ofitsiant PWA
├── kds/                # oshxona ekrani
└── admin/              # admin panel
```

6.1 🧑 GUEST PWA (mijoz)

Maydon	Qiymat
Kirish	QR skan → `https://<slug>.armenu.uz/t/<token>`
Auth	⛔ Yo'q (anonim sessiya, 4 soat)
Bundle	~45 KB gzip (+ `model-viewer` 250 KB, faqat AR ochilganda)
Maqsad	FCP < 1,2s, to'liq menyu < 2s (4G)

Sahifalar:

Sahifa	URL	Nima
Menyu	`/t/<token>/`	Kategoriyalar, taomlar, qidiruv, filtr
Taom	`/t/<token>/d/<id>/`	Tavsif, allergen, modifikator, ⚡ **AR tugmasi**
AR ko'rish	modal	`<model-viewer>` + AR rejim
Savat	`/t/<token>/cart/`	Ro'yxat, miqdor, izoh, jami (serverdan)
Buyurtma holati	`/t/<token>/o/<id>/`	⚡ Bu yerda WS ochiladi (ADR-005)
To'lov	`/t/<token>/pay/<id>/`	Provayder tanlash, chaevoy
Sharh	`/t/<token>/review/<id>/`	Baho + smart yo'naltirish

JS modullar:

Modul	Vazifasi
`guest/menu.js`	Kategoriya navigatsiya, virtual scroll (200+ taom), qidiruv
`guest/dish.js`	Taom modali, modifikator tanlash, narx **oldindan ko'rsatish** (server tasdiqlaydi)
`guest/ar.js`	`<model-viewer>` lazy yuklash, AR tugmasi, fallback zanjiri
`guest/cart.js`	Savat CRUD, serverga sinxronizatsiya (debounce 400ms)
`guest/order.js`	Buyurtma yuborish (idempotency key), holat kuzatuvi
`guest/pay.js`	To'lov provayder redirect, natija kutish

Holat boshqaruvi:

```
// static/js/core/store.js — ~80 qator, framework o'rniga
export function createStore(initial) {
  let state = { ...initial };
  const listeners = new Set();
  return {
    get: () => state,
    set(patch) {
      const prev = state;
      state = { ...state, ...patch };
      listeners.forEach(fn => fn(state, prev));
    },
    subscribe(fn) { listeners.add(fn); return () => listeners.delete(fn); },
  };
}

// static/js/guest/cart.js
export const cart = createStore({ items: [], total: 0, syncing: false });

cart.subscribe((s) => {
  document.querySelector('#cart-badge').textContent = s.items.length || '';
  document.querySelector('#cart-total').textContent = formatPrice(s.total);
});

// ⚡ Jami SERVERDAN keladi (ADR-006) — klient faqat ko'rsatadi
async function syncCart() {
  cart.set({ syncing: true });
  const data = await api.post('/v1/cart/sync/', { items: cart.get().items });
  cart.set({ total: data.total, items: data.items, syncing: false });
}
```

Offline strategiya (Service Worker):

Resurs	Strategiya	Sabab
App shell (HTML/CSS/JS)	**Cache-first**	Darhol ochiladi
Menyu JSON	**Stale-while-revalidate**	Tarmoq sekin bo'lsa eski menyu ko'rinadi
Taom rasmlari	**Cache-first**, LRU 50 MB	Qayta ko'rishda tez
3D modellar	**Network-first**, kesh 3 ta oxirgi	Katta fayl, joy tejash
`POST /v1/orders/`	⛔ **Network-only**	Offline buyurtma **yo'q** (ADR-006 + fiskal)
Buyurtma holati	Network-first, `stale` bannerli	Offline'da "oxirgi ma'lum holat"

```
// static/sw.js — buyurtma hech qachon keshlanmaydi
self.addEventListener('fetch', (e) => {
  const url = new URL(e.request.url);
  if (e.request.method !== 'GET') return;                    // ⛔ POST kesh yo'q
  if (url.pathname.startsWith('/v1/orders')) return;         // ⛔ hech qachon
  if (url.pathname.startsWith('/v1/menu')) {
    e.respondWith(staleWhileRevalidate(e.request, 'menu-v1'));
  } else if (url.pathname.startsWith('/static/')) {
    e.respondWith(cacheFirst(e.request, 'shell-v1'));
  }
});
```

6.2 👨‍🍳 STAFF PWA (ofitsiant)

Maydon	Qiymat
Kirish	`https://<slug>.armenu.uz/staff/` → PIN
Auth	4-6 raqamli PIN (bcrypt), smena sessiyasi 12 soat
Qurilma	Telefon (asosan), planshet
Real-time	⚡ WS **doim ochiq** (`rest_<rid>_waiters`)

Sahifalar:

Sahifa	Nima
Zal plani	Stollar holati (bo'sh/buyurtma/xizmatda/hisob), rang bilan
Chaqiruvlar	Ofitsiant chaqiruvlari (ovoz + vibratsiya + qizil badge)
Buyurtmalar	O'z stollari buyurtmalari, holat o'zgartirish
Buyurtma qo'shish	Ofitsiant mijoz nomidan buyurtma kiritadi
Hisob	Chek, to'lov qabul qilish (naqd/karta/QR)
Smena	Ochish/yopish, statistika, chaevoy

JS modullar:

Modul	Vazifasi
`staff/auth.js`	PIN kiritish, sessiya saqlash, avto-lock 10 daq
`staff/hall.js`	Zal plani (SVG), stol holati real-time
`staff/calls.js`	Chaqiruv navbati, ovozli signal
`staff/order.js`	Buyurtma kiritish/o'zgartirish
`staff/bill.js`	Hisob, bo'lish, chaevoy

Offline strategiya:

Resurs	Strategiya
App shell	Cache-first
Menyu (buyurtma kiritish uchun)	Cache-first, 5 daq da yangilanadi
Zal holati	⛔ Network-only (real-time bo'lishi shart)
Harakatlar (holat o'zgartirish)	🟢 **Navbat** — offline'da `localStorage`, tarmoq kelganda yuboriladi (idempotency key bilan)

6.3 🍽️ KDS (oshxona ekrani)

Maydon	Qiymat
Kirish	`https://<slug>.armenu.uz/kds/` → PIN (bir marta, doimiy sessiya)
Qurilma	TV / katta monitor, ⚡ **teginish yo'q ham bo'lishi mumkin**
Real-time	⚡ WS majburiy (`rest_<rid>_kds`) + polling fallback 10s
Alohida talab	Ekran **hech qachon o'chmaydi** (Wake Lock API), avto-qayta yuklash 6 soatda

Ekran maketi:



JS modullar:

Modul	Vazifasi
`kds/board.js`	Kartochkalar grid, avtomatik joylashish, taymer
`kds/sound.js`	Yangi buyurtma signali (ovoz + flash)
`kds/wake.js`	Wake Lock API — ekran o'chmaydi
`kds/reconnect.js`	WS uzilsa: 1s→3s→9s→SSE→polling 10s + 🔴 indikator

Offline strategiya: ⛔ Offline rejim**yo'q** — KDS tarmoqsiz ma'nosiz.
Uzilganda: qizil banner "Ulanish yo'q — oxirgi yangilanish 14:29", oxirgi holat ekranda qoladi.

6.4 🧑‍💼 ADMIN (restoran boshqaruvi)

Maydon	Qiymat
Kirish	`https://<slug>.armenu.uz/admin/` → login + parol
Qurilma	Desktop (asosiy), planshet
Texnologiya	Django template + server render + kichik JS (⛔ SPA emas)
Real-time	WS (`rest_<rid>_admin`) — dashboard jonli raqamlar

Bo'limlar:

Bo'lim	Nima
Dashboard	Bugungi savdo, faol stol, o'rtacha chek, ⚡ AR konversiya
Menyu	Kategoriya/taom CRUD, narx, stop-list, tarjima, rasm
3D / AR	Model kutubxonasi, generatsiya so'rovi, tasdiqlash navbati
Zal va stol	Zal plani muharriri, QR generatsiya (PDF)
Buyurtmalar	Tarix, filtr, tafsilot, bekor qilish
To'lovlar	To'lovlar, qaytarishlar, chaevoy taqsimoti
Xodimlar	Staff CRUD, PIN, rol, smena
Hisobotlar	Savdo, menyu-injiniring, AR konversiya, eksport (Excel/PDF)
Sozlamalar	Restoran, tarif, integratsiyalar, i18n

JS modullar:

Modul	Vazifasi
`admin/dashboard.js`	Jonli raqamlar (WS), grafiklar (kichik SVG, kutubxonasiz)
`admin/menu-editor.js`	Inline tahrirlash, drag-drop tartib, rasm yuklash
`admin/model-review.js`	3D tasdiqlash: `<model-viewer>` + tasdiqla/rad et
`admin/hall-editor.js`	Zal plani (SVG drag-drop)
`admin/reports.js`	Filtr, jadval, eksport

⚡ **Dark + Light mavzu** — ikkalasi ham majburiy `prefers-color-scheme` + qo'lda almashtirish, tanlov `localStorage` da).

6.5 `core/realtime.js` — avtomatik fallback (P3)

```
// static/js/core/realtime.js
const BACKOFF = [1000, 3000, 9000];

export function connect(group, onMessage) {
  let attempt = 0, mode = 'ws', es = null, poller = null;

  function openWS() {
    const ws = new WebSocket(`wss://${location.host}/ws/${group}/`);
    ws.onopen    = () => { attempt = 0; mode = 'ws'; setIndicator('🟢'); };
    ws.onmessage = (e) => onMessage(JSON.parse(e.data));
    ws.onclose   = () => {
      if (attempt < BACKOFF.length) {
        setIndicator('🟡');
        setTimeout(openWS, BACKOFF[attempt++]);
      } else {
        openSSE();                      // ⚡ 2-qatlam
      }
    };
  }

  function openSSE() {
    mode = 'sse'; setIndicator('🟡');
    es = new EventSource(`/sse/${group}/`);
    es.onmessage = (e) => onMessage(JSON.parse(e.data));
    es.onerror   = () => { es.close(); startPolling(); };   // ⚡ 3-qatlam
  }

  function startPolling() {
    mode = 'poll'; setIndicator('🔴');
    poller = setInterval(async () => {
      const data = await api.get(`/v1/updates/?group=${group}&since=${lastTs}`);
      data.events.forEach(onMessage);
    }, 10000);
  }

  openWS();
  return { close: () => { /* ... */ }, get mode() { return mode; } };
}
```

---

§7. WORKER TURLARI (5 navbat)

> ADR-008 (Redis navbat) + P9 (bulkhead). Har navbat — alohida systemd unit, alohida resurs chegarasi, alohida retry siyosati.

```
┌──────────────────┬────────┬──────┬──────┬──────────┬──────────────┬─────────────┐
│ NAVBAT           │ WORKER │ MEM  │ CPU  │ O'RTACHA │ RETRY        │ MAX DEPTH   │
├──────────────────┼────────┼──────┼──────┼──────────┼──────────────┼─────────────┤
│ 🎨 model3d       │      3 │ 2 GB │ 150% │ 3-8 daq  │ 3× (1/5/15m) │   5 000     │
│ 🧾 fiscal        │      5 │ 512M │  50% │ 2-5 s    │ 10× (48s)    │     500     │
│ 🔔 notify        │      8 │ 256M │  30% │ 0,5-2 s  │ 5× (1s→10m)  │  20 000     │
│ 🤖 ai            │      4 │ 1 GB │ 100% │ 3-15 s   │ 2× (0,5/2s)  │   2 000     │
│ 📊 analytics     │      2 │ 512M │  50% │ 50-200 ms│ 3× (batch)   │  50 000     │
└──────────────────┴────────┴──────┴──────┴──────────┴──────────────┴─────────────┘

   ⚡ model3d 5 000 job bilan to'lca — fiscal, notify ta'sirlanmaydi
```

7.1 🎨 `queue:model3d` — 3D generatsiya

Maydon	Qiymat
Nima qiladi	Rasmdan 3D model yaratish, optimizatsiya, USDZ konversiya, S3 yuklash
Vazifalar	`generate` · `optimize` · `convert_usdz` · `render_poster` · `reoptimize`
O'rtacha vaqt	**3-8 daqiqa** (AI provayder kutish 2-6 daq)
Job timeout	900s (15 daq)
Retry	3 marta — 1 daq, 5 daq, 15 daq
Bulkhead	3 worker · `MemoryMax=2G` · `CPUQuota=150%` · max depth 5 000
Tashqi API	Meshy → Tripo → Rodin (breaker + fallback zanjiri)
Narx	~$0,20-0,50 per model
Muvaffaqiyat	~78% (qolgani qo'lda tuzatish yoki kutubxonadan)

```python
# apps/ar3d/tasks.py
@task('model3d', 'generate')
def generate_model(job, dish_id: int, ref_images: list[str]):
    dish = Dish.objects.select_related('category__restaurant').get(pk=dish_id)

    prepared = prepare_images(ref_images)          # fon olib tashlash, 1024×1024
    job.heartbeat()

    glb_bytes = with_fallback_chain(
        [MeshyProvider(), TripoProvider(), RodinProvider()],
        'image_to_3d', prepared, prompt=build_prompt(dish),
    )
    job.heartbeat()

    glb  = optimize_glb(glb_bytes, target_mb=4)    # Draco + KTX2
    usdz = convert_usdz(glb)                       # iOS uchun
    poster = render_poster(glb)

    urls = upload_model(dish.category.restaurant_id, dish_id, glb, usdz, poster)
    DishModel3D.objects.update_or_create(
        dish=dish, defaults={**urls, 'source': 'ai', 'status': 'review'})

    bump_version(dish.category.restaurant_id, 'menu')
    notify_admin(dish, '3D model tayyor — ko'rib chiqing')
```

⛔ **Yiqilsa:** 🟢 Kichik ta'sir. Mavjud 3D modellar ishlaydi, yangilari navbatda kutadi.

7.2 🧾 `queue:fiscal` — Fiskal chek

Maydon	Qiymat
Nima qiladi	OFD'ga soliq cheki yuborish, IKPU aniqlash, chek arxivi
Vazifalar	`send_receipt` · `send_refund` · `sync_ikpu` · `close_shift`
O'rtacha vaqt	**2-5 soniya**
Job timeout	30s
Retry	⚡ **10 marta** — 5 daq, 15 daq, 1 soat, 6 soat... **48 soat davomida**
Bulkhead	5 worker · `MemoryMax=512M` · max depth 500
Tashqi API	UZKASSA → e-POS (breaker `fail_threshold=3, timeout=120`)
Kritiklik	🟠 **Eng yuqori** — soliq javobgarligi

```
@task('fiscal', 'send_receipt', max_attempts=10)
def send_receipt(job, order_id: int):
    order = Order.objects.select_related('restaurant').prefetch_related('items__dish').get(pk=order_id)

    if Receipt.objects.filter(order_id=order_id, status='sent').exists():
        return                                  # ⚡ idempotent (P4)

    payload = build_fiscal_payload(order)          # IKPU, QQS, summa
    result = with_fallback_chain(
        [UzkassaOFD(order.restaurant), EposOFD(order.restaurant)],
        'send', payload,
    )
    Receipt.objects.create(order=order, provider=result.provider,
                           fiscal_sign=result.sign, url=result.url, status='sent')
    ws_send(group_name(order.restaurant_id, 'admin'),
            {'type': 'receipt.sent', 'order_id': order_id})
```

⛔ **Yiqilsa:** 🟡 O'rta. Buyurtma va to'lov ishlaydi, chek navbatda.
Soliq muddati 24 soat — 48 soatlik retry byudjeti yetarli.
Alert: depth > 50 (10 daq) → 🟡 **P1** (Telegram + qo'ng'iroq).
DLQ'ga tushsa → darhol P1 + qo'lda chek chiqarish ko'rsatmasi.

7.3 🪫 `queue:notify` — Bildirishnoma

Maydon	Qiymat
Nima qiladi	Telegram, web-push, SMS, email, oshxona printeri
Vazifalar	`telegram_new_order` · `push_order_ready` · `kitchen_print` · `sms_otp` · `email_report`
O'rtacha vaqt	**0,5-2 soniya**
Job timeout	20s
Retry	5 marta — 1s, 5s, 30s, 2 daq, 10 daq (jami ~13 daq)
Bulkhead	8 worker · `MemoryMax=256M` · max depth 20 000
Tashqi API	Telegram Bot API, Web Push (VAPID), SMS gateway

⛔ **Yiqilsa:** 🟢 Kichik. Buyurtma ishlaydi, bildirishnoma kechikadi.
KDS baribir WS orqali ko'radi (bildirishnoma dublikat kanal).

7.4 🤖 `queue:ai` — AI so'rovlar

Maydon	Qiymat
Nima qiladi	Menyu tarjimasi, taom tavsifi, embedding, og'ir AI tahlil
Vazifalar	`translate_menu` · `generate_description` · `embed_dish` · `menu_insights`
O'rtacha vaqt	**3-15 soniya**
Job timeout	60s
Retry	2 marta — 0,5s, 2s (keyin **boshqa provayder**)
Bulkhead	4 worker · `MemoryMax=1G` · max depth 2 000
Tashqi API	Gemini → Haiku → GPT-mini → Groq → DeepSeek → Sonnet
Xarajat nazorati	`armenu_ai_cost_usd_total{provider,model}` · alert > $50/kun

> ⚡ **Mijoz chatbot bu navbatda EMAS.** Chat — sinxron (web so'rovda), 8s timeout bilan.
> Sabab: mijoz javob kutadi. Navbat — faqat **fon** AI ishlari uchun.

⛔ **Yiqilsa:** 🟢 Ta'sir yo'q. Tarjima/tavsif kechikadi. Chatbot alohida yo'ldan ishlaydi.

7.5 📊 `queue:analytics` — Analitika

Maydon	Qiymat
Nima qiladi	Event'larni batch qilib ClickHouse'ga yozish, materialized view yangilash
Vazifalar	`ingest` (batch) · `refresh_mv` · `daily_rollup` · `export_report`
O'rtacha vaqt	**50-200 ms** (batch 1 000 ta)
Job timeout	120s
Retry	3 marta (batch bilan) → DLQ
Bulkhead	2 worker · `MemoryMax=512M` · max depth 50 000
Prioritet	🟢 Eng past
Batch	1 000 ta yoki 5 soniya (qaysi biri oldin)

⛔ **Yiqilsa:** 🟩 Biznesga ta'sir yo'q. Event'lar navbatda to'planadi (max depth 50 000 — ~2 soatlik zaxira), keyin yoziladi.

7.6 Worker umumiy jadvali

Navbat	Worker	RAM	CPU	Vaqt	Retry	Depth	Timeout	Yiqilsa
`model3d`	3	2 GB	150%	3-8 daq	3× (1/5/15 daq)	5 000	900s	🟩
`fiscal`	5	512 MB	50%	2-5 s	10× (48 soat)	500	30s	🟩
`notify`	8	256 MB	30%	0,5-2 s	5× (13 daq)	20 000	20s	🟩
`ai`	4	1 GB	100%	3-15 s	2× + provayder	2 000	60s	🟩
`analytics`	2	512 MB	50%	50-200 ms	3× (batch)	50 000	120s	🟩

systemd shabloni:

```
# deploy/systemd/armenu-worker@.service
[Unit]
Description=ARMENU worker (%i)
After=network.target postgresql.service redis.service

[Service]
Type=simple
User=armenu
WorkingDirectory=/opt/armenu
EnvironmentFile=/opt/armenu/.env
ExecStart=/opt/armenu/venv/bin/python manage.py run_worker --queue=%i
Restart=always
RestartSec=5
# ⚡ Bulkhead (P9) — qiymatlar drop-in fayldan keladi
# /etc/systemd/system/armenu-worker@model3d.service.d/limits.conf:
#   [Service]
#   MemoryMax=2G
#   CPUQuota=150%

[Install]
WantedBy=multi-user.target
```

```
# Ishga tushirish
systemctl enable --now armenu-worker@model3d.service
systemctl enable --now armenu-worker@fiscal.service
systemctl enable --now armenu-worker@notify.service
systemctl enable --now armenu-worker@ai.service
systemctl enable --now armenu-worker@analytics.service
```

---

§8. KOMPONENT ↔ ADR ↔ FAZA MATRITSASI

Komponent	Amalga oshiruvchi ADR	Printsip	Faza
C2 nginx	010, 012	P7	F0
C3 web	001, 010, 012	P1, P2	F0
C4 ws	005	P3, P12	F6
C5 db	002, 007	P6, P12	F0
C6 redis	008	P1, P4, P7, P8	F0
C7 s3	011	P3	F5
C8 worker	008	P9	F0 (skelet), F5+
C9 healer	—	P3, P10	F12
C10 clickhouse	013	P9	F11 (S2)
`apps/tenants`	007	P12	F1
`apps/accounts`	007	P12	F1
`apps/menu`	002, 006, 012	P7, P11	F2
`apps/tables`	007	P11	F3
`apps/guest`	003	P1	F3-F4
`apps/ar3d`	004, 011	P3, P9	F5
`apps/orders`	001, 006	P4, P5, P11	F6
`apps/payments`	014	P4, P8, P12	F7
`apps/fiscal`	—	P8, P9	F8
`apps/ai`	009	P3, P8, P12	F9
`apps/loyalty`	—	P5	F10
`apps/reviews`	—	P5	F10
`apps/analytics`	013	P5, P9	F11
Guest PWA	003, 004, 005	P3	F4-F5
Staff PWA	003	P3, P4	F6
KDS	003, 005	P3	F6
Admin	003	P12	F2+
`core/`	001, 007, 008	P1-P12	F0